

Sistema RTOS para Control de Cuna Inteligente

1.0.0

Generado el Miércoles, 10 de diciembre de 2025 16:48:45 para Sistema RTOS para Control de Cuna Inteligente por Doxygen 1.15.0

Miércoles, 10 de diciembre de 2025 16:48:45

1 Lista de obsoletos	1
2 Topic Index	3
2.1 Topics	3
3 Jerarquía de directorios	5
3.1 Directorios	5
4 Índice de estructuras de datos	7
4.1 Estructuras de datos	7
5 Índice de archivos	9
5.1 Lista de archivos	9
6 Topic Documentation	11
6.1 Estados de actualización OTA	11
6.1.1 Descripción detallada	11
6.1.2 Documentación de «define»	11
6.1.2.1 OTA_UPDATE_FAILED	11
6.1.2.2 OTA_UPDATE_PENDING	11
6.1.2.3 OTA_UPDATE_SUCCESSFUL	12
6.2 Archivos embebidos en el firmware	12
6.2.1 Descripción detallada	12
6.2.2 Documentación de funciones	12
6.2.2.1 asm() [1/10]	12
6.2.2.2 asm() [2/10]	13
6.2.2.3 asm() [3/10]	13
6.2.2.4 asm() [4/10]	13
6.2.2.5 asm() [5/10]	13
6.2.2.6 asm() [6/10]	13
6.2.2.7 asm() [7/10]	13
6.2.2.8 asm() [8/10]	13
6.2.2.9 asm() [9/10]	14
6.2.2.10 asm() [10/10]	14
6.3 Configuración del Access Point	14
6.3.1 Descripción detallada	15
6.3.2 Documentación de «define»	15
6.3.2.1 MAX_CONNECTION_RETRIES	15
6.3.2.2 MAX_PASSWORD_LENGTH	15
6.3.2.3 MAX_SSID_LENGTH	15
6.3.2.4 WIFI_AP_BANDWIDTH	15
6.3.2.5 WIFI_AP_BEACON_INTERVAL	15
6.3.2.6 WIFI_AP_CHANNEL	16
6.3.2.7 WIFI_AP_GATEWAY	16

6.3.2.8 WIFI_AP_IP	16
6.3.2.9 WIFI_AP_MAX_CONNECTIONS	16
6.3.2.10 WIFI_AP_NETMASK	16
6.3.2.11 WIFI_AP_PASSWORD	17
6.3.2.12 WIFI_AP_SSID	17
6.3.2.13 WIFI_AP_SSID_HIDDEN	17
6.3.2.14 WIFI_STA_POWER_SAVE	17
6.4 Configuración del Sensor NTC	17
6.4.1 Descripción detallada	18
6.4.2 Documentación de «define»	18
6.4.2.1 CANAL_ADC_NTC	18
6.4.2.2 COEFICIENTE_BETA_NTC	18
6.4.2.3 RESISTENCIA_DIVISOR	19
6.4.2.4 RESISTENCIA_NOMINAL_NTC	19
6.4.2.5 TEMP_NOMINAL_NTC	19
6.4.2.6 VOLTAJE_REFERENCIA_MV	19
7 Documentación de directorios	21
7.1 Referencia del directorio webpage	21
8 Documentación de estructuras de datos	23
8.1 Referencia de la estructura http_server_queue_message	23
8.1.1 Descripción detallada	23
8.1.2 Documentación de campos	23
8.1.2.1 msgID	23
8.2 Referencia de la estructura http_server_queue_message_t	23
8.2.1 Descripción detallada	24
8.3 Referencia de la estructura pir_event_t	24
8.3.1 Descripción detallada	24
8.3.2 Documentación de campos	24
8.3.2.1 motion	24
8.4 Referencia de la estructura reg_t	25
8.4.1 Descripción detallada	25
8.4.2 Documentación de campos	25
8.4.2.1 day_count	25
8.4.2.2 days	25
8.4.2.3 hour	26
8.4.2.4 minute	26
8.5 Referencia de la estructura register_saved	26
8.5.1 Descripción detallada	27
8.5.2 Documentación de campos	27
8.5.2.1 friday	27
8.5.2.2 hour	27

8.5.2.3 min	27
8.5.2.4 monday	27
8.5.2.5 saturday	27
8.5.2.6 sunday	28
8.5.2.7 thursday	28
8.5.2.8 tuesday	28
8.5.2.9 wednesday	28
8.6 Referencia de la estructura register_saved_e	28
8.6.1 Descripción detallada	28
8.7 Referencia de la estructura sensor_data_t	29
8.7.1 Descripción detallada	29
8.7.2 Documentación de campos	29
8.7.2.1 temperature_celsius	29
8.7.2.2 temperature_valid	29
8.7.2.3 time_info	30
8.7.2.4 time_valid	30
8.8 Referencia de la estructura wifi_app_queue_message	30
8.8.1 Descripción detallada	30
8.8.2 Documentación de campos	30
8.8.2.1 msgID	30
8.9 Referencia de la estructura wifi_app_queue_message_t	31
8.9.1 Descripción detallada	31
9 Documentación de archivos	33
9.1 Referencia del archivo fan_control.c	33
9.1.1 Descripción detallada	34
9.1.2 Documentación de funciones	35
9.1.2.1 compute_auto_speed()	35
9.1.2.2 fan_control_get_mode()	35
9.1.2.3 fan_control_get_speed()	36
9.1.2.4 fan_control_init()	36
9.1.2.5 fan_control_on_register_tick()	37
9.1.2.6 fan_control_set_manual_speed()	37
9.1.2.7 fan_control_set_mode()	38
9.1.2.8 fan_control_update()	39
9.1.3 Documentación de variables	40
9.1.3.1 s_current_speed	40
9.1.3.2 s_manual_speed	40
9.1.3.3 s_mode	41
9.1.3.4 s_ntc_adc_handle	41
9.1.3.5 s_reg_active	41
9.2 fan_control.c	42

9.3 Referencia del archivo fan_control.h	44
9.3.1 Descripción detallada	44
9.3.2 Documentación de enumeraciones	45
9.3.2.1 fan_mode_t	45
9.3.3 Documentación de funciones	45
9.3.3.1 fan_control_get_mode()	45
9.3.3.2 fan_control_get_speed()	46
9.3.3.3 fan_control_init()	46
9.3.3.4 fan_control_on_register_tick()	47
9.3.3.5 fan_control_set_manual_speed()	48
9.3.3.6 fan_control_set_mode()	48
9.3.3.7 fan_control_update()	50
9.4 fan_control.h	51
9.5 Referencia del archivo fan_driver.c	51
9.5.1 Documentación de funciones	51
9.5.1.1 fan_init()	51
9.5.1.2 fan_off()	52
9.5.1.3 fan_set_speed_percent()	52
9.5.2 Documentación de variables	52
9.5.2.1 s_fan_initialized	52
9.6 fan_driver.c	52
9.7 Referencia del archivo fan_driver.h	53
9.7.1 Documentación de «define»	53
9.7.1.1 FAN_LEDC_CHANNEL	53
9.7.1.2 FAN_LEDC_DUTY_RES	54
9.7.1.3 FAN_LEDC_FREQ_HZ	54
9.7.1.4 FAN_LEDC_MODE	54
9.7.1.5 FAN_LEDC_TIMER	54
9.7.1.6 FAN_PWM_GPIO	54
9.7.2 Documentación de funciones	54
9.7.2.1 fan_init()	54
9.7.2.2 fan_off()	55
9.7.2.3 fan_set_speed_percent()	55
9.8 fan_driver.h	55
9.9 Referencia del archivo http_server.c	56
9.9.1 Descripción detallada	58
9.9.2 Documentación de funciones	60
9.9.2.1 fan_task()	60
9.9.2.2 http_server_app_css_handler()	60
9.9.2.3 http_server_app_js_handler()	61
9.9.2.4 http_server_configure()	61
9.9.2.5 http_server_fan_control_handler()	62

9.9.2.6 http_server_favicon_ico_handler()	63
9.9.2.7 http_server_fw_update_reset_callback()	63
9.9.2.8 http_server_fw_update_reset_timer()	64
9.9.2.9 http_server_get_dht_sensor_readings_json_handler()	64
9.9.2.10 http_server_index_html_handler()	65
9.9.2.11 http_server_jquery_handler()	65
9.9.2.12 http_server_monitor()	66
9.9.2.13 http_server_monitor_send_message()	66
9.9.2.14 http_server_OTA_status_handler()	67
9.9.2.15 http_server_OTA_update_handler()	67
9.9.2.16 http_server_read_register_handler()	68
9.9.2.17 http_server_register_erase_handler()	68
9.9.2.18 http_server_start()	69
9.9.2.19 http_server_stop()	70
9.9.2.20 http_server_time_json_handler()	70
9.9.2.21 http_server_toogle_led_handler()	71
9.9.2.22 http_server_wifi_connect_json_handler()	71
9.9.2.23 http_server_wifi_connect_status_json_handler()	72
9.9.2.24 log_error_if_nonzero()	72
9.9.2.25 sensors_init()	73
9.9.2.26 toogle_led()	73
9.9.3 Documentación de variables	74
9.9.3.1 fw_update_reset	74
9.9.3.2 fw_update_reset_args	74
9.9.3.3 g_fw_update_status	74
9.9.3.4 g_wifi_connect_status	74
9.9.3.5 http_server_handle	75
9.9.3.6 http_server_monitor_queue_handle	75
9.9.3.7 s_led_state	75
9.9.3.8 s_ntc_adc_handle	75
9.9.3.9 TAG	76
9.9.3.10 task_http_server_monitor	76
9.10 http_server.c	76
9.11 Referencia del archivo http_server.h	88
9.11.1 Descripción detallada	89
9.11.2 Documentación de «define»	90
9.11.2.1 BLINK_GPIO	90
9.11.3 Documentación de «typedef»	90
9.11.3.1 http_server_message_e	90
9.11.3.2 http_server_queue_message_t	90
9.11.3.3 http_server_wifi_connect_status_e	90
9.11.4 Documentación de enumeraciones	90

9.11.4.1 http_server_message	90
9.11.4.2 http_server_wifi_connect_status	90
9.11.5 Documentación de funciones	91
9.11.5.1 http_server_fw_update_reset_callback()	91
9.11.5.2 http_server_monitor_send_message()	91
9.11.5.3 http_server_start()	92
9.11.5.4 http_server_stop()	93
9.11.5.5 toggle_led()	94
9.12 http_server.h	94
9.13 Referencia del archivo ntc_driver.c	95
9.13.1 Descripción detallada	95
9.13.2 Documentación de funciones	96
9.13.2.1 inicializar_ntc()	96
9.13.2.2 leer_temperatura_celsius()	97
9.14 ntc_driver.c	98
9.15 Referencia del archivo ntc_driver.h	98
9.15.1 Descripción detallada	99
9.15.2 Documentación de funciones	100
9.15.2.1 inicializar_ntc()	100
9.15.2.2 leer_temperatura_celsius()	101
9.16 ntc_driver.h	103
9.17 Referencia del archivo pir_driver.c	103
9.17.1 Descripción detallada	103
9.17.2 Documentación de funciones	104
9.17.2.1 pir_init()	104
9.17.2.2 pir_is_motion_active()	105
9.17.2.3 pir_isr_handler()	106
9.17.3 Documentación de variables	106
9.17.3.1 s_pir_evt_queue	106
9.17.3.2 s_pir_gpio	107
9.18 pir_driver.c	107
9.19 Referencia del archivo pir_driver.h	108
9.19.1 Descripción detallada	108
9.19.2 Documentación de funciones	109
9.19.2.1 pir_init()	109
9.19.2.2 pir_is_motion_active()	111
9.20 pir_driver.h	112
9.21 Referencia del archivo registers.c	112
9.21.1 Descripción detallada	113
9.21.2 Documentación de funciones	114
9.21.2.1 api_delete_register()	114
9.21.2.2 api_get_registers()	115

9.21.2.3	api_post_register()	115
9.21.2.4	delete_register_from_nvs()	116
9.21.2.5	init_nvs()	116
9.21.2.6	load_register_from_nvs()	117
9.21.2.7	load_register_struct_from_nvs()	118
9.21.2.8	register_is_active_now()	119
9.21.2.9	register_registers_endpoints()	120
9.21.2.10	registers_scheduler_task()	121
9.21.2.11	save_register_to_nvs()	122
9.21.3	Documentación de variables	123
9.21.3.1	TAG	123
9.22	registers.c	123
9.23	Referencia del archivo registers.h	128
9.23.1	Descripción detallada	129
9.23.2	Documentación de funciones	129
9.23.2.1	api_delete_register()	129
9.23.2.2	api_get_registers()	130
9.23.2.3	api_post_register()	130
9.23.2.4	delete_register_from_nvs()	131
9.23.2.5	init_nvs()	131
9.23.2.6	load_register_from_nvs()	132
9.23.2.7	register_registers_endpoints()	133
9.23.2.8	save_register_to_nvs()	133
9.24	registers.h	135
9.25	Referencia del archivo sensor_task.c	135
9.25.1	Descripción detallada	136
9.25.2	Documentación de funciones	137
9.25.2.1	sensor_task_get_data()	137
9.25.2.2	sensor_task_init()	138
9.25.2.3	sensor_task_update_now()	139
9.25.2.4	sensor_update_task()	139
9.25.3	Documentación de variables	140
9.25.3.1	adc_handle	140
9.25.3.2	sensor_mutex	140
9.25.3.3	shared_sensor_data	141
9.25.3.4	TAG	141
9.26	sensor_task.c	141
9.27	Referencia del archivo sensor_task.h	142
9.27.1	Descripción detallada	143
9.27.2	Documentación de funciones	143
9.27.2.1	sensor_task_get_data()	143
9.27.2.2	sensor_task_init()	145

9.27.2.3 sensor_task_update_now()	146
9.28 sensor_task.h	147
9.29 Referencia del archivo webpage/app.js	147
9.30 app.js	147
9.31 Referencia del archivo webpage/index.html	155
9.32 index.html	155
9.33 Referencia del archivo wifi_app.c	159
9.33.1 Descripción detallada	161
9.33.2 Documentación de funciones	162
9.33.2.1 check_sta_connection_state()	162
9.33.2.2 compare_hour_day_structs()	162
9.33.2.3 connect_to_wifi()	163
9.33.2.4 get_state_time_was_synchronized()	163
9.33.2.5 init_obtain_time()	163
9.33.2.6 initialize_registers()	164
9.33.2.7 load_wifi_credentials()	164
9.33.2.8 nvs_credentials_exist()	165
9.33.2.9 obtain_time()	165
9.33.2.10 read_reg_data()	165
9.33.2.11 save_reg_data()	166
9.33.2.12 save_wifi_credentials()	166
9.33.2.13 task_compare_hour_to_execute_action()	167
9.33.2.14 update_register()	168
9.33.2.15 wifi_app_connect_sta()	168
9.33.2.16 wifi_app_default_wifi_init()	168
9.33.2.17 wifi_app_event_handler()	169
9.33.2.18 wifi_app_event_handler_init()	169
9.33.2.19 wifi_app_get_wifi_config()	170
9.33.2.20 wifi_app_send_message()	170
9.33.2.21 wifi_app_soft_ap_config()	171
9.33.2.22 wifi_app_start()	172
9.33.2.23 wifi_app_task()	173
9.33.3 Documentación de variables	174
9.33.3.1 esp_netif_ap	174
9.33.3.2 esp_netif_sta	174
9.33.3.3 g_retry_number	174
9.33.3.4 mySemaphore	174
9.33.3.5 register_readings_from_flash	175
9.33.3.6 s_led_state	175
9.33.3.7 TAG	175
9.33.3.8 time_was_synchronized	175
9.33.3.9 wifi_app_queue_handle	175

9.33.3.10 wifi_config	176
9.34 wifi_app.c	176
9.35 Referencia del archivo wifi_app.h	186
9.35.1 Descripción detallada	189
9.35.2 Documentación de «define»	189
9.35.2.1 NUM_REGISTERS_AV	189
9.35.3 Documentación de «typedef»	189
9.35.3.1 register_saved_e	189
9.35.3.2 wifi_app_message_e	190
9.35.3.3 wifi_app_queue_message_t	190
9.35.3.4 wifi_connected_event_callback_t	190
9.35.4 Documentación de enumeraciones	190
9.35.4.1 wifi_app_message	190
9.35.5 Documentación de funciones	190
9.35.5.1 check_sta_connection_state()	190
9.35.5.2 connect_to_wifi()	191
9.35.5.3 get_state_time_was_synchronized()	191
9.35.5.4 init_obtain_time()	192
9.35.5.5 initialize_registers()	192
9.35.5.6 load_wifi_credentials()	192
9.35.5.7 obtain_time()	193
9.35.5.8 read_reg_data()	193
9.35.5.9 save_reg_data()	194
9.35.5.10 save_wifi_credentials()	194
9.35.5.11 task_compare_hour_to_execute_action()	195
9.35.5.12 update_register()	197
9.35.5.13 wifi_app_call_callback()	197
9.35.5.14 wifi_app_connect_sta()	197
9.35.5.15 wifi_app_get_rssi()	198
9.35.5.16 wifi_app_get_wifi_config()	198
9.35.5.17 wifi_app_send_message()	198
9.35.5.18 wifi_app_set_callback()	199
9.35.5.19 wifi_app_start()	199
9.35.6 Documentación de variables	200
9.35.6.1 esp_netif_ap	200
9.35.6.2 esp_netif_sta	200
9.36 wifi_app.h	201
9.37 Referencia del archivo C:/Users/Igamb/Documents/Uni/RTOS/Repositorio/RTOS_Repo_Limpio/← Proyecto_final_TITI/cuna/README.md	202

Capítulo 1

Lista de obsoletos

Global [save_reg_data](#) (uint8_t register, char *str)

Usar API moderna en [registers.c](#)

Capítulo 2

Topic Index

2.1. Topics

Here is a list of all topics with brief descriptions:

Estados de actualización OTA	11
Archivos embebidos en el firmware	12
Configuración del Access Point	14
Configuración del Sensor NTC	17

Capítulo 3

Jerarquía de directorios

3.1. Directorios

webpage	21
app.js	147
index.html	155

Capítulo 4

Índice de estructuras de datos

4.1. Estructuras de datos

Lista de estructuras con breves descripciones:

http_server_queue_message	23
http_server_queue_message_t	
Estructura de mensajes para la cola del monitor HTTP	23
pir_event_t	
Evento enviado por la cola cuando el PIR detecta cambio	24
reg_t	
Estructura de un registro programado	25
register_saved	26
register_saved_e	
Estructura de registro programado guardado en NVS	28
sensor_data_t	
Estructura con datos de sensores protegida por mutex	29
wifi_app_queue_message	30
wifi_app_queue_message_t	
Estructura de mensajes para la cola WiFi	31

Capítulo 5

Índice de archivos

5.1. Lista de archivos

Lista de todos los archivos con breves descripciones:

fan_control.c	Implementación del controlador multi-modo del ventilador	33
fan_control.h	Control automático del ventilador con múltiples modos de operación	44
fan_driver.c		51
fan_driver.h		53
http_server.c	Servidor HTTP con REST API para control del sistema	56
http_server.h	Servidor HTTP con endpoints REST para control del sistema	88
ntc_driver.c	Implementación del driver para sensor NTC con conversión Steinhart-Hart	95
ntc_driver.h	Driver para sensor de temperatura NTC (Negative Temperature Coefficient)	98
pir_driver.c	Implementación del driver PIR con soporte ISR y polling	103
pir_driver.h	Driver para sensor PIR (Passive Infrared) de detección de movimiento	108
registers.c	Implementación del sistema de programación de eventos (scheduler)	112
registers.h	API REST para gestión de registros programados con persistencia NVS	128
sensor_task.c	Implementación de tarea RTOS para lectura thread-safe de sensores	135
sensor_task.h	Tarea dedicada para lectura de sensores con acceso thread-safe mediante mutex	142
wifi_app.c	Aplicación WiFi con soporte AP/STA dual, reconexión automática y SNTP	159
wifi_app.h	Gestión completa de WiFi AP/STA con reconexión automática y persistencia	186
webpage/app.js		147
webpage/index.html		155

Capítulo 6

Topic Documentation

6.1. Estados de actualización OTA

defines

- #define [OTA_UPDATE_PENDING](#) 0
Actualización OTA pendiente.
- #define [OTA_UPDATE_SUCCESSFUL](#) 1
Actualización OTA exitosa.
- #define [OTA_UPDATE_FAILED](#) -1
Actualización OTA fallida.

6.1.1. Descripción detallada

6.1.2. Documentación de «define»

6.1.2.1. OTA_UPDATE_FAILED

```
#define OTA_UPDATE_FAILED -1
```

Actualización OTA fallida.

Definición en la línea [40](#) del archivo [http_server.h](#).

Referenciado por [http_server_monitor\(\)](#).

6.1.2.2. OTA_UPDATE_PENDING

```
#define OTA_UPDATE_PENDING 0
```

Actualización OTA pendiente.

Definición en la línea [38](#) del archivo [http_server.h](#).

6.1.2.3. OTA_UPDATE_SUCCESSFUL

```
#define OTA_UPDATE_SUCCESSFUL 1
```

Actualización OTA exitosa.

Definición en la línea 39 del archivo [http_server.h](#).

Referenciado por [http_server_fw_update_reset_timer\(\)](#) y [http_server_monitor\(\)](#).

6.2. Archivos embebidos en el firmware

Archivos estáticos compilados dentro del binario.

Funciones

- `const uint8_t jquery_3_3_1_min_js_start[] asm ("_binary_jquery_3_3_1_min_js_start")`
Inicio de jquery 3.3.1 en memoria.
- `const uint8_t jquery_3_3_1_min_js_end[] asm ("_binary_jquery_3_3_1_min_js_end")`
Fin de jquery 3.3.1 en memoria.
- `const uint8_t index_html_start[] asm ("_binary_index_html_start")`
Inicio de [index.html](#) en memoria.
- `const uint8_t index_html_end[] asm ("_binary_index_html_end")`
Fin de [index.html](#) en memoria.
- `const uint8_t app_css_start[] asm ("_binary_app_css_start")`
Inicio de [app.css](#) en memoria.
- `const uint8_t app_css_end[] asm ("_binary_app_css_end")`
Fin de [app.css](#) en memoria.
- `const uint8_t app_js_start[] asm ("_binary_app_js_start")`
Inicio de [app.js](#) en memoria.
- `const uint8_t app_js_end[] asm ("_binary_app_js_end")`
Fin de [app.js](#) en memoria.
- `const uint8_t favicon_ico_start[] asm ("_binary_favicon_ico_start")`
Inicio de [favicon.ico](#) en memoria.
- `const uint8_t favicon_ico_end[] asm ("_binary_favicon_ico_end")`
Fin de [favicon.ico](#) en memoria.

6.2.1. Descripción detallada

Archivos estáticos compilados dentro del binario.

6.2.2. Documentación de funciones

6.2.2.1. `asm()` [1/10]

```
const uint8_t app_css_end[] asm (
    "_binary_app_css_end" ) [extern]
```

Fin de [app.css](#) en memoria.

6.2.2.2. asm() [2/10]

```
const uint8_t app_css_start[] asm (
    "_binary_app_css_start" ) [extern]
```

Inicio de app.css en memoria.

6.2.2.3. asm() [3/10]

```
const uint8_t app_js_end[] asm (
    "_binary_app_js_end" ) [extern]
```

Fin de [app.js](#) en memoria.

6.2.2.4. asm() [4/10]

```
const uint8_t app_js_start[] asm (
    "_binary_app_js_start" ) [extern]
```

Inicio de [app.js](#) en memoria.

6.2.2.5. asm() [5/10]

```
const uint8_t favicon_ico_end[] asm (
    "_binary_favicon_ico_end" ) [extern]
```

Fin de favicon.ico en memoria.

6.2.2.6. asm() [6/10]

```
const uint8_t favicon_ico_start[] asm (
    "_binary_favicon_ico_start" ) [extern]
```

Inicio de favicon.ico en memoria.

6.2.2.7. asm() [7/10]

```
const uint8_t index_html_end[] asm (
    "_binary_index_html_end" ) [extern]
```

Fin de [index.html](#) en memoria.

6.2.2.8. asm() [8/10]

```
const uint8_t index_html_start[] asm (
    "_binary_index_html_start" ) [extern]
```

Inicio de [index.html](#) en memoria.

6.2.2.9. asm() [9/10]

```
const uint8_t jquery_3_3_1_min_js_end[] asm (
    "_binary_jquery_3_3_1_min_js_end" ) [extern]
```

Fin de jQuery 3.3.1 en memoria.

6.2.2.10. asm() [10/10]

```
const uint8_t jquery_3_3_1_min_js_start[] asm (
    "_binary_jquery_3_3_1_min_js_start" ) [extern]
```

Inicio de jQuery 3.3.1 en memoria.

6.3. Configuración del Access Point

defines

- #define `WIFI_AP_SSID` "Mi_esp"
Nombre de la red AP.
- #define `WIFI_AP_PASSWORD` "12345678"
Contraseña del AP (mínimo 8 caracteres).
- #define `WIFI_AP_CHANNEL` 1
Canal WiFi del AP (1-13).
- #define `WIFI_AP_SSID_HIDDEN` 0
0=visible, 1=oculto
- #define `WIFI_AP_MAX_CONNECTIONS` 5
Máximo de clientes simultáneos.
- #define `WIFI_AP_BEACON_INTERVAL` 100
Intervalo beacon en ms (recomendado 100ms).
- #define `WIFI_AP_IP` "192.168.0.1"
IP del AP.
- #define `WIFI_AP_GATEWAY` "192.168.0.1"
Gateway del AP (igual que IP).
- #define `WIFI_AP_NETMASK` "255.255.255.0"
Máscara de subred.
- #define `WIFI_AP_BANDWIDTH` WIFI_BW_HT20
Ancho de banda 20 MHz.
- #define `WIFI_STA_POWER_SAVE` WIFI_PS_NONE
Sin ahorro de energía.
- #define `MAX_SSID_LENGTH` 32
Longitud máxima SSID (estándar IEEE).
- #define `MAX_PASSWORD_LENGTH` 64
Longitud máxima contraseña (estándar IEEE).
- #define `MAX_CONNECTION_RETRIES` 5
Intentos de reconexión antes de fallar.

6.3.1. Descripción detallada

6.3.2. Documentación de «define»

6.3.2.1. MAX_CONNECTION_RETRIES

```
#define MAX_CONNECTION_RETRIES 5
```

Intentos de reconexión antes de fallar.

Definición en la línea 48 del archivo [wifi_app.h](#).

6.3.2.2. MAX_PASSWORD_LENGTH

```
#define MAX_PASSWORD_LENGTH 64
```

Longitud máxima contraseña (estándar IEEE).

Definición en la línea 47 del archivo [wifi_app.h](#).

6.3.2.3. MAX_SSID_LENGTH

```
#define MAX_SSID_LENGTH 32
```

Longitud máxima SSID (estándar IEEE).

Definición en la línea 46 del archivo [wifi_app.h](#).

6.3.2.4. WIFI_AP_BANDWIDTH

```
#define WIFI_AP_BANDWIDTH WIFI_BW_HT20
```

Ancho de banda 20 MHz.

Definición en la línea 44 del archivo [wifi_app.h](#).

Referenciado por [wifi_app_soft_ap_config\(\)](#).

6.3.2.5. WIFI_AP_BEACON_INTERVAL

```
#define WIFI_AP_BEACON_INTERVAL 100
```

Intervalo beacon en ms (recomendado 100ms).

Definición en la línea 40 del archivo [wifi_app.h](#).

Referenciado por [wifi_app_soft_ap_config\(\)](#).

6.3.2.6. WIFI_AP_CHANNEL

```
#define WIFI_AP_CHANNEL 1
```

Canal WiFi del AP (1-13).

Definición en la línea 37 del archivo [wifi_app.h](#).

Referenciado por [wifi_app_soft_ap_config\(\)](#).

6.3.2.7. WIFI_AP_GATEWAY

```
#define WIFI_AP_GATEWAY "192.168.0.1"
```

Gateway del AP (igual que IP).

Definición en la línea 42 del archivo [wifi_app.h](#).

Referenciado por [wifi_app_soft_ap_config\(\)](#).

6.3.2.8. WIFI_AP_IP

```
#define WIFI_AP_IP "192.168.0.1"
```

IP del AP.

Definición en la línea 41 del archivo [wifi_app.h](#).

Referenciado por [wifi_app_soft_ap_config\(\)](#).

6.3.2.9. WIFI_AP_MAX_CONNECTIONS

```
#define WIFI_AP_MAX_CONNECTIONS 5
```

Máximo de clientes simultáneos.

Definición en la línea 39 del archivo [wifi_app.h](#).

Referenciado por [wifi_app_soft_ap_config\(\)](#).

6.3.2.10. WIFI_AP_NETMASK

```
#define WIFI_AP_NETMASK "255.255.255.0"
```

Máscara de subred.

Definición en la línea 43 del archivo [wifi_app.h](#).

Referenciado por [wifi_app_soft_ap_config\(\)](#).

6.3.2.11. WIFI_AP_PASSWORD

```
#define WIFI_AP_PASSWORD "12345678"
```

Contraseña del AP (mínimo 8 caracteres).

Definición en la línea 36 del archivo [wifi_app.h](#).

Referenciado por [wifi_app_soft_ap_config\(\)](#).

6.3.2.12. WIFI_AP_SSID

```
#define WIFI_AP_SSID "Mi_esp"
```

Nombre de la red AP.

Definición en la línea 35 del archivo [wifi_app.h](#).

Referenciado por [wifi_app_soft_ap_config\(\)](#).

6.3.2.13. WIFI_AP_SSID_HIDDEN

```
#define WIFI_AP_SSID_HIDDEN 0
```

0=visible, 1=oculto

Definición en la línea 38 del archivo [wifi_app.h](#).

Referenciado por [wifi_app_soft_ap_config\(\)](#).

6.3.2.14. WIFI_STA_POWER_SAVE

```
#define WIFI_STA_POWER_SAVE WIFI_PS_NONE
```

Sin ahorro de energía.

Definición en la línea 45 del archivo [wifi_app.h](#).

Referenciado por [wifi_app_soft_ap_config\(\)](#).

6.4. Configuración del Sensor NTC

Parámetros de hardware y calibración del termistor.

defines

- #define [CANAL_ADC_NTC](#) ADC_CHANNEL_6
Canal del ADC donde está conectado el NTC (GPIO 34).
- #define [RESISTENCIA_DIVISOR](#) 10000.0f
Resistencia fija del divisor de voltaje (10k).
- #define [RESISTENCIA_NOMINAL_NTC](#) 10000.0f
Resistencia nominal del NTC a temperatura de referencia (10k @ 25 °C).
- #define [TEMP_NOMINAL_NTC](#) 25.0f
Temperatura nominal de calibración (25 °C).
- #define [COEFICIENTE_BETA_NTC](#) 3950.0f
Coeficiente Beta del NTC (3950K).
- #define [VOLTAJE_REFERENCIA_MV](#) 3300.0f
Voltaje de referencia del ADC (3.3V = 3300mV).

6.4.1. Descripción detallada

Parámetros de hardware y calibración del termistor.

6.4.2. Documentación de «define»

6.4.2.1. CANAL_ADC_NTC

```
#define CANAL_ADC_NTC ADC_CHANNEL_6
```

Canal del ADC donde está conectado el NTC (GPIO 34).

Definición en la línea 43 del archivo [ntc_driver.c](#).

Referenciado por [inicializar_ntc\(\)](#) y [leer_temperatura_celsius\(\)](#).

6.4.2.2. COEFICIENTE_BETA_NTC

```
#define COEFICIENTE_BETA_NTC 3950.0f
```

Coeficiente Beta del NTC (3950K).

El coeficiente Beta caracteriza la variación de resistencia con temperatura. Se obtiene de la hoja de datos del fabricante. Un Beta típico para NTC 10K es 3950K ± 1 %.

La ecuación relacionada es:

$$R(T) = R0 * \exp(B * (1/T - 1/T0))$$

Definición en la línea 66 del archivo [ntc_driver.c](#).

Referenciado por [leer_temperatura_celsius\(\)](#).

6.4.2.3. RESISTENCIA_DIVISOR

```
#define RESISTENCIA_DIVISOR 10000.0f
```

Resistencia fija del divisor de voltaje (10k).

Definición en la línea 46 del archivo [ntc_driver.c](#).

Referenciado por [leer_temperatura_celsius\(\)](#).

6.4.2.4. RESISTENCIA_NOMINAL_NTC

```
#define RESISTENCIA_NOMINAL_NTC 10000.0f
```

Resistencia nominal del NTC a temperatura de referencia (10k @ 25°C).

Definición en la línea 49 del archivo [ntc_driver.c](#).

Referenciado por [leer_temperatura_celsius\(\)](#).

6.4.2.5. TEMP_NOMINAL_NTC

```
#define TEMP_NOMINAL_NTC 25.0f
```

Temperatura nominal de calibración (25°C).

Definición en la línea 52 del archivo [ntc_driver.c](#).

Referenciado por [leer_temperatura_celsius\(\)](#).

6.4.2.6. VOLTAJE_REFERENCIA_MV

```
#define VOLTAJE_REFERENCIA_MV 3300.0f
```

Voltaje de referencia del ADC (3.3V = 3300mV).

Definición en la línea 69 del archivo [ntc_driver.c](#).

Referenciado por [leer_temperatura_celsius\(\)](#).

Capítulo 7

Documentación de directorios

7.1. Referencia del directorio webpage

Archivos

- archivo [app.js](#)
- archivo [index.html](#)

Capítulo 8

Documentación de estructuras de datos

8.1. Referencia de la estructura `http_server_queue_message`

```
#include <http_server.h>
```

Campos de datos

- `http_server_message_e msgID`
ID del mensaje a procesar.

8.1.1. Descripción detallada

Definición en la línea 78 del archivo `http_server.h`.

8.1.2. Documentación de campos

8.1.2.1. `msgID`

`http_server_message_e http_server_queue_message::msgID`

ID del mensaje a procesar.

Definición en la línea 80 del archivo `http_server.h`.

La documentación de esta estructura está generada del siguiente archivo:

- `http_server.h`

8.2. Referencia de la estructura `http_server_queue_message_t`

Estructura de mensajes para la cola del monitor HTTP.

```
#include <http_server.h>
```

8.2.1. Descripción detallada

Estructura de mensajes para la cola del monitor HTTP.

La documentación de esta estructura está generada del siguiente archivo:

- [http_server.h](#)

8.3. Referencia de la estructura `pir_event_t`

Evento enviado por la cola cuando el PIR detecta cambio.

```
#include <pir_driver.h>
```

Campos de datos

- `bool motion`
true = movimiento detectado (pin HIGH), false = sin movimiento (pin LOW)

8.3.1. Descripción detallada

Evento enviado por la cola cuando el PIR detecta cambio.

Esta estructura se envía a través de la cola cuando el sensor PIR cambia de estado (flanco de subida o bajada).

Uso:

```
QueueHandle_t pir_queue = xQueueCreate(10, sizeof(pir_event_t));
pir_init(GPIO_NUM_35, pir_queue);

pir_event_t evt;
if (xQueueReceive(pir_queue, &evt, portMAX_DELAY)) {
    if (evt.motion) {
        printf("Movimiento detectado!\n");
    } else {
        printf("Fin de movimiento\n");
    }
}
```

Definición en la línea 62 del archivo [pir_driver.h](#).

8.3.2. Documentación de campos

8.3.2.1. motion

```
bool pir_event_t::motion
```

true = movimiento detectado (pin HIGH), false = sin movimiento (pin LOW)

Definición en la línea 63 del archivo [pir_driver.h](#).

La documentación de esta estructura está generada del siguiente archivo:

- [pir_driver.h](#)

8.4. Referencia de la estructura reg_t

Estructura de un registro programado.

```
#include <registers.h>
```

Campos de datos

- int [hour](#)
Hora de ejecución (0-23).
- int [minute](#)
Minuto de ejecución (0-59).
- char [days](#) [7][12]
Array de días: "Lunes", "Martes", etc.
- int [day_count](#)
Cantidad de días válidos en el array.

8.4.1. Descripción detallada

Estructura de un registro programado.

Define una acción programada con hora, minuto y días de la semana.

Definición en la línea [36](#) del archivo [registers.h](#).

8.4.2. Documentación de campos

8.4.2.1. day_count

```
int reg_t::day_count
```

Cantidad de días válidos en el array.

Definición en la línea [40](#) del archivo [registers.h](#).

Referenciado por [api_post_register\(\)](#), [load_register_struct_from_nvs\(\)](#), [register_is_active_now\(\)](#) y [save_register_to_nvs\(\)](#).

8.4.2.2. days

```
char reg_t::days[7][12]
```

Array de días: "Lunes", "Martes", etc.

Definición en la línea [39](#) del archivo [registers.h](#).

Referenciado por [api_post_register\(\)](#), [load_register_struct_from_nvs\(\)](#), [register_is_active_now\(\)](#) y [save_register_to_nvs\(\)](#).

8.4.2.3. hour

```
int reg_t::hour
```

Hora de ejecución (0-23).

Definición en la línea 37 del archivo [registers.h](#).

Referenciado por [api_post_register\(\)](#), [load_register_struct_from_nvs\(\)](#), [register_is_active_now\(\)](#) y [save_register_to_nvs\(\)](#).

8.4.2.4. minute

```
int reg_t::minute
```

Minuto de ejecución (0-59).

Definición en la línea 38 del archivo [registers.h](#).

Referenciado por [api_post_register\(\)](#), [load_register_struct_from_nvs\(\)](#), [register_is_active_now\(\)](#) y [save_register_to_nvs\(\)](#).

La documentación de esta estructura está generada del siguiente archivo:

- [registers.h](#)

8.5. Referencia de la estructura register_saved

```
#include <wifi_app.h>
```

Campos de datos

- `uint8_t hour`
Hora de ejecución (0-23).
- `uint8_t min`
Minuto de ejecución (0-59).
- `uint8_t monday`
1=activo el lunes, 0=inactivo
- `uint8_t tuesday`
1=activo el martes, 0=inactivo
- `uint8_t wednesday`
1=activo el miércoles, 0=inactivo
- `uint8_t thursday`
1=activo el jueves, 0=inactivo
- `uint8_t friday`
1=activo el viernes, 0=inactivo
- `uint8_t sunday`
1=activo el domingo, 0=inactivo
- `uint8_t saturday`
1=activo el sábado, 0=inactivo

8.5.1. Descripción detallada

Definición en la línea 91 del archivo `wifi_app.h`.

8.5.2. Documentación de campos

8.5.2.1. `friday`

```
uint8_t register_saved::friday
```

1=activo el viernes, 0=inactivo

Definición en la línea 99 del archivo `wifi_app.h`.

8.5.2.2. `hour`

```
uint8_t register_saved::hour
```

Hora de ejecución (0-23).

Definición en la línea 93 del archivo `wifi_app.h`.

8.5.2.3. `min`

```
uint8_t register_saved::min
```

Minuto de ejecución (0-59).

Definición en la línea 94 del archivo `wifi_app.h`.

8.5.2.4. `monday`

```
uint8_t register_saved::monday
```

1=activo el lunes, 0=inactivo

Definición en la línea 95 del archivo `wifi_app.h`.

8.5.2.5. `saturday`

```
uint8_t register_saved::saturday
```

1=activo el sábado, 0=inactivo

Definición en la línea 101 del archivo `wifi_app.h`.

8.5.2.6. sunday

```
uint8_t register_saved::sunday
```

1=activo el domingo, 0=inactivo

Definición en la línea 100 del archivo [wifi_app.h](#).

8.5.2.7. thursday

```
uint8_t register_saved::thursday
```

1=activo el jueves, 0=inactivo

Definición en la línea 98 del archivo [wifi_app.h](#).

8.5.2.8. tuesday

```
uint8_t register_saved::tuesday
```

1=activo el martes, 0=inactivo

Definición en la línea 96 del archivo [wifi_app.h](#).

8.5.2.9. wednesday

```
uint8_t register_saved::wednesday
```

1=activo el miércoles, 0=inactivo

Definición en la línea 97 del archivo [wifi_app.h](#).

La documentación de esta estructura está generada del siguiente archivo:

- [wifi_app.h](#)

8.6. Referencia de la estructura register_saved_e

Estructura de registro programado guardado en NVS.

```
#include <wifi_app.h>
```

8.6.1. Descripción detallada

Estructura de registro programado guardado en NVS.

Almacena hora, minuto y días de la semana para ejecución programada.

La documentación de esta estructura está generada del siguiente archivo:

- [wifi_app.h](#)

8.7. Referencia de la estructura sensor_data_t

Estructura con datos de sensores protegida por mutex.

```
#include <sensor_task.h>
```

Campos de datos

- float [temperature_celsius](#)
Temperatura en grados Celsius del sensor NTC.
- struct tm [time_info](#)
Información de tiempo del sistema (SNTP).
- bool [time_valid](#)
true si la hora está sincronizada por SNTP
- bool [temperature_valid](#)
true si la lectura de temperatura es válida

8.7.1. Descripción detallada

Estructura con datos de sensores protegida por mutex.

Contiene las últimas lecturas válidas de temperatura, hora del sistema y flags de validez para cada dato.

Definición en la línea 29 del archivo [sensor_task.h](#).

8.7.2. Documentación de campos

8.7.2.1. temperature_celsius

```
float sensor_data_t::temperature_celsius
```

Temperatura en grados Celsius del sensor NTC.

Definición en la línea 30 del archivo [sensor_task.h](#).

Referenciado por [sensor_update_task\(\)](#).

8.7.2.2. temperature_valid

```
bool sensor_data_t::temperature_valid
```

true si la lectura de temperatura es válida

Definición en la línea 33 del archivo [sensor_task.h](#).

Referenciado por [sensor_update_task\(\)](#).

8.7.2.3. `time_info`

```
struct tm sensor_data_t::time_info
```

Información de tiempo del sistema (SNTP).

Definición en la línea 31 del archivo [sensor_task.h](#).

Referenciado por [sensor_update_task\(\)](#).

8.7.2.4. `time_valid`

```
bool sensor_data_t::time_valid
```

true si la hora está sincronizada por SNTP

Definición en la línea 32 del archivo [sensor_task.h](#).

Referenciado por [sensor_update_task\(\)](#).

La documentación de esta estructura está generada del siguiente archivo:

- [sensor_task.h](#)

8.8. Referencia de la estructura `wifi_app_queue_message`

```
#include <wifi_app.h>
```

Campos de datos

- [wifi_app_message_e msgID](#)
ID del mensaje a procesar.

8.8.1. Descripción detallada

Definición en la línea 80 del archivo [wifi_app.h](#).

8.8.2. Documentación de campos

8.8.2.1. `msgID`

```
wifi_app_message_e wifi_app_queue_message::msgID
```

ID del mensaje a procesar.

Definición en la línea 82 del archivo [wifi_app.h](#).

La documentación de esta estructura está generada del siguiente archivo:

- [wifi_app.h](#)

8.9. Referencia de la estructura wifi_app_queue_message_t

Estructura de mensajes para la cola WiFi.

```
#include <wifi_app.h>
```

8.9.1. Descripción detallada

Estructura de mensajes para la cola WiFi.

La documentación de esta estructura está generada del siguiente archivo:

- [wifi_app.h](#)

Capítulo 9

Documentación de archivos

9.1. Referencia del archivo fan_control.c

Implementación del controlador multi-modo del ventilador.

```
#include "fan_control.h"
#include "fan_driver.h"
#include "esp_adc/adc_oneshot.h"
#include "ntc_driver.h"
#include "pir_driver.h"
```

Funciones

- static uint8_t [compute_auto_speed](#) (float temp_c)
- void [fan_control_init](#) (void)
Inicializa el subsistema de control del ventilador.
- void [fan_control_set_mode](#) (fan_mode_t mode)
Cambia el modo de operación del ventilador.
- fan_mode_t [fan_control_get_mode](#) (void)
Obtiene el modo de operación actual.
- void [fan_control_set_manual_speed](#) (uint8_t percent)
Establece la velocidad del slider (usado en MANUAL y REGISTERS).
- uint8_t [fan_control_get_speed](#) (void)
Obtiene la velocidad REAL del ventilador PWM.
- void [fan_control_on_register_tick](#) (bool any_match_now)
Callback desde scheduler de registros programados.
- void [fan_control_update](#) (void)
Actualiza velocidad del ventilador en modo AUTO.

Variables

- `adc_onehot_unit_handle_t s_ntc_adc_handle`
Handle del ADC para sensor NTC (compartido globalmente).
- `static fan_mode_t s_mode = FAN_MODE_MANUAL`
Modo de operación actual del ventilador.
- `static uint8_t s_manual_speed = 0`
Velocidad configurada por usuario en slider (0-100 %).
- `static uint8_t s_current_speed = 0`
Velocidad REAL actual del ventilador PWM (0-100 %).
- `static bool s_reg_active = false`
Flag que indica si hay algún registro programado activo.

9.1.1. Descripción detallada

Implementación del controlador multi-modo del ventilador.

Autor

Jair Hernan Telpis Cuaran, Luis Fernando Gamba Bedoya

Fecha

2025

Este módulo implementa la lógica de control del ventilador con 3 modos de operación:

MODO MANUAL (FAN_MODE_MANUAL):

- Usuario controla velocidad directamente desde web UI (0-100 %)
- Velocidad PWM = velocidad del slider
- No aplica lógica automática

MODO AUTOMÁTICO (FAN_MODE_AUTO):

- Velocidad basada en temperatura del NTC
- Algoritmo: $< 25^{\circ}\text{C} \rightarrow 0\%$, $[25-30^{\circ}\text{C}] \rightarrow \text{lineal}$, $> 30^{\circ}\text{C} \rightarrow 100\%$
- Requiere sensor funcional (retorna 0 % si error NTC)
- Actualizado por `fan_task` cada 1 segundo

MODO REGISTROS (FAN_MODE_REGISTERS):

- Activado por scheduler de registros programados
- Si hay registro activo: velocidad = slider
- Si no hay registro: ventilador apagado
- Actualizado cada 10s por `registers_scheduler_task`

Arquitectura de actualización:

```

[fan_task] --- (cada 1s) ---> fan_control_update() si modo AUTO
    |
    v
[compute_auto_speed()] ---> Lee NTC ---> Calcula PWM
    |
    v
fan_set_speed_percent() ---> Actualiza hardware PWM
    |
    v
[registers_scheduler] --- (cada 10s) ---> fan_control_on_register_tick()
    |
    v
Enciende/apaga según registros activos

```

Variables de estado:

- `s_mode`: Modo actual (MANUAL/AUTO/REGISTERS)
- `s_manual_speed`: Velocidad del slider (0-100 %)
- `s_current_speed`: Velocidad REAL del PWM (puede diferir del slider en AUTO)
- `s_reg_active`: Flag de registro programado activo

Ver también

[fan_control.h](#) Para la API pública
[fan_driver.h](#) Para control de hardware PWM
[registers.c](#) Para scheduler de programación

Definición en el archivo [fan_control.c](#).

9.1.2. Documentación de funciones**9.1.2.1. `compute_auto_speed()`**

```

uint8_t compute_auto_speed (
    float temp_c) [static]

```

Definición en la línea 103 del archivo [fan_control.c](#).

Referenciado por [fan_control_set_mode\(\)](#) y [fan_control_update\(\)](#).

9.1.2.2. `fan_control_get_mode()`

```

fan_mode_t fan_control_get_mode (
    void )

```

Obtiene el modo de operación actual.

Obtener modo de operación actual.

Devuelve

[fan_mode_t](#) Modo actual (MANUAL/AUTO/REGISTERS)

Nota

Thread-safe: lectura atómica de enum

Definición en la línea 244 del archivo [fan_control.c](#).

Hace referencia a [s_mode](#).

9.1.2.3. `fan_control_get_speed()`

```
uint8_t fan_control_get_speed (
    void )
```

Obtiene la velocidad REAL del ventilador PWM.

Obtener velocidad actual del PWM.

Retorna `s_current_speed`, que es el duty cycle real del PWM. Puede diferir de `s_manual_speed` en modo AUTO.

Devuelve

uint8_t Velocidad actual (0-100 %)

Nota

Thread-safe: lectura atómica de `uint8_t`

Usado por OLED display y HTTP GET `/api/dhtSensor`

Definición en la línea 292 del archivo `fan_control.c`.

Hace referencia a `s_current_speed`.

9.1.2.4. `fan_control_init()`

```
void fan_control_init (
    void )
```

Inicializa el subsistema de control del ventilador.

Inicializar sistema de control del ventilador.

Secuencia de inicialización:

1. Inicializa hardware PWM (`fan_init()`)
2. Configura modo MANUAL por defecto
3. Velocidades a 0 %
4. Apaga ventilador

Estado inicial:

- Modo: `FAN_MODE_MANUAL`
- Velocidad manual: 0 %
- Velocidad actual: 0 %
- Registro activo: false
- PWM: apagado

Atención

Llamar antes de usar cualquier otra función de `fan_control`

Nota

Thread-safe: solo llamar una vez durante inicialización

Definición en la línea 146 del archivo `fan_control.c`.

Hace referencia a `fan_init()`, `FAN_MODE_MANUAL`, `fan_off()`, `s_current_speed`, `s_manual_speed`, `s_mode` y `s_reg_active`.

Referenciado por `http_server_start()`.

9.1.2.5. fan_control_on_register_tick()

```
void fan_control_on_register_tick (
    bool any_match_now)
```

Callback desde scheduler de registros programados.

Notificar cambio en registros programados.

Llamado cada 10 segundos por registers_scheduler_task. Solo tiene efecto si el modo actual es FAN_MODE_↔ REGISTERS.

Lógica:

- Si any_match_now = true: Enciende ventilador a velocidad del slider
- Si any_match_now = false: Apaga ventilador

En otros modos (MANUAL/AUTO), esta función no hace nada.

Parámetros

in	any_match_now	true si hay al menos un registro activo ahora
----	---------------	---

Nota

Llamado desde registers_scheduler_task (diferente tarea)

Atención

No es completamente thread-safe con cambios simultáneos de modo

Ver también

[registers.c](#) Para el scheduler que llama esta función

Definición en la línea 318 del archivo [fan_control.c](#).

Hace referencia a [FAN_MODE_REGISTERS](#), [fan_off\(\)](#), [fan_set_speed_percent\(\)](#), [s_current_speed](#), [s_manual_speed](#), [s_mode](#) y [s_reg_active](#).

Referenciado por [registers_scheduler_task\(\)](#).

9.1.2.6. fan_control_set_manual_speed()

```
void fan_control_set_manual_speed (
    uint8_t percent)
```

Establece la velocidad del slider (usado en MANUAL y REGISTERS).

Establecer velocidad manual del ventilador.

Actualiza s_manual_speed y aplica inmediatamente si está en modo MANUAL.

Comportamiento por modo:

- MANUAL: Actualiza PWM inmediatamente
- AUTO: Solo guarda valor (no afecta PWM)
- REGISTERS: Solo guarda valor (se aplica cuando registro activo)

Parámetros

in	percent	Velocidad deseada (0-100 %)
----	---------	-----------------------------

Nota

Valores > 100 se limitan automáticamente a 100

Atención

No es thread-safe: llamar solo desde HTTP server task

Definición en la línea 266 del archivo [fan_control.c](#).

Hace referencia a [FAN_MODE_MANUAL](#), [fan_set_speed_percent\(\)](#), [s_current_speed](#), [s_manual_speed](#) y [s_mode](#).

Referenciado por [http_server_fan_control_handler\(\)](#).

9.1.2.7. fan_control_set_mode()

```
void fan_control_set_mode (
    fan_mode_t mode)
```

Cambia el modo de operación del ventilador.

Establecer modo de operación del ventilador.

Actualiza el modo y aplica lógica correspondiente inmediatamente:

MANUAL:

- Aplica velocidad del slider directamente
- No requiere sensores

AUTO:

- Lee temperatura NTC
- Calcula velocidad con [compute_auto_speed\(\)](#)
- Si ADC NULL o error NTC: apaga ventilador
- PIR deshabilitado en esta versión (comentado)

REGISTERS:

- Si hay registro activo: aplica velocidad del slider
- Si no hay registro: apaga ventilador

Parámetros

<code>in</code>	<code>mode</code>	Nuevo modo (FAN_MODE_MANUAL/AUTO/REGISTERS)
-----------------	-------------------	---

Atención

No es thread-safe: llamar solo desde HTTP server task

Nota

Cambio de modo aplica velocidad inmediatamente (no espera a fan_task)

Ver también

[fan_control_update\(\)](#) Para actualizaciones periódicas en modo AUTO

Definición en la línea 184 del archivo [fan_control.c](#).

Hace referencia a [compute_auto_speed\(\)](#), [FAN_MODE_AUTO](#), [FAN_MODE_MANUAL](#), [FAN_MODE_REGISTERS](#), [fan_off\(\)](#), [fan_set_speed_percent\(\)](#), [leer_temperatura_celsius\(\)](#), [pir_is_motion_active\(\)](#), [s_current_speed](#), [s_manual_speed](#), [s_mode](#), [s_ntc_adc_handle](#) y [s_reg_active](#).

Referenciado por [http_server_fan_control_handler\(\)](#).

9.1.2.8. fan_control_update()

```
void fan_control_update (
    void )
```

Actualiza velocidad del ventilador en modo AUTO.

Actualizar control (llamado cada 1s por fan_task).

Llamado cada 1 segundo por fan_task. Solo tiene efecto si el modo actual es FAN_MODE_AUTO.

Algoritmo de actualización:

1. Verifica que el modo sea AUTO (si no, retorna)
2. Verifica que el ADC esté inicializado
3. Lee temperatura del NTC
4. Valida lectura ($> -100^{\circ}\text{C}$)
5. Calcula velocidad con [compute_auto_speed\(\)](#)
6. Actualiza PWM con [fan_set_speed_percent\(\)](#)

Manejo de errores:

- Si ADC handle es NULL: apaga ventilador
- Si temperatura $< -100^{\circ}\text{C}$: apaga ventilador (error NTC)
- PIR deshabilitado: no verifica movimiento (comentado)

Nota

Llamado desde `fan_task` cada 1 segundo
Solo procesa en modo AUTO (retorna inmediatamente en otros modos)

Atención

No es thread-safe con cambios de modo simultáneos

Ver también

`fan_task` en [http_server.c](#) Para la tarea que llama esta función
`compute_auto_speed()` Para el algoritmo de cálculo

Definición en la línea [367](#) del archivo [fan_control.c](#).

Hace referencia a [compute_auto_speed\(\)](#), [FAN_MODE_AUTO](#), [fan_off\(\)](#), [fan_set_speed_percent\(\)](#), [leer_temperatura_celsius\(\)](#), [pir_is_motion_active\(\)](#), [s_current_speed](#), [s_mode](#) y [s_ntc_adc_handle](#).

Referenciado por [fan_task\(\)](#).

9.1.3. Documentación de variables

9.1.3.1. `s_current_speed`

```
uint8_t s_current_speed = 0 [static]
```

Velocidad REAL actual del ventilador PWM (0-100 %).

Puede diferir de `s_manual_speed` en modo AUTO.

- MANUAL: `s_current_speed = s_manual_speed`
- AUTO: `s_current_speed = compute_auto_speed(temperatura)`
- REGISTERS: `s_current_speed = s_manual_speed` si registro activo, 0 si no

Esta variable representa el duty cycle real del PWM.

Definición en la línea [90](#) del archivo [fan_control.c](#).

Referenciado por [fan_control_get_speed\(\)](#), [fan_control_init\(\)](#), [fan_control_on_register_tick\(\)](#), [fan_control_set_manual_speed\(\)](#), [fan_control_set_mode\(\)](#) y [fan_control_update\(\)](#).

9.1.3.2. `s_manual_speed`

```
uint8_t s_manual_speed = 0 [static]
```

Velocidad configurada por usuario en slider (0-100 %).

Definición en la línea [77](#) del archivo [fan_control.c](#).

Referenciado por [fan_control_init\(\)](#), [fan_control_on_register_tick\(\)](#), [fan_control_set_manual_speed\(\)](#) y [fan_control_set_mode\(\)](#).

9.1.3.3. s_mode

```
fan_mode_t s_mode = FAN_MODE_MANUAL [static]
```

Modo de operación actual del ventilador.

Definición en la línea 74 del archivo fan_control.c.

Referenciado por fan_control_get_mode(), fan_control_init(), fan_control_on_register_tick(), fan_control_set_manual_speed(), fan_control_set_mode() y fan_control_update().

9.1.3.4. s_ntc_adc_handle

```
adc_oneshot_unit_handle_t s_ntc_adc_handle [extern]
```

Handle del ADC para sensor NTC (compartido globalmente).

Handle externo del ADC para lectura de temperatura.

Inicializado en http_server_configure_peripherals(). Usado por endpoint /dhtSensor.json para leer temperatura. También usado por fan_control.c (extern).

Definido en http_server.c e inicializado durante el arranque. Usado por compute_auto_speed() para leer temperatura del NTC. Si es NULL, modo AUTO retorna velocidad 0 %.

Definición en la línea 128 del archivo http_server.c.

Referenciado por fan_control_set_mode(), fan_control_update(), http_server_get_dht_sensor_readings_json_handler() y sensors_init().

9.1.3.5. s_reg_active

```
bool s_reg_active = false [static]
```

Flag que indica si hay algún registro programado activo.

Actualizado por fan_control_on_register_tick() desde registers_scheduler_task. Solo tiene efecto en modo FAN ↔ MODE_REGISTERS.

Definición en la línea 99 del archivo fan_control.c.

Referenciado por fan_control_init(), fan_control_on_register_tick() y fan_control_set_mode().

9.2. fan_control.c

[Ir a la documentación de este archivo.](#)

```

00001
00053
00054 #include "fan_control.h"
00055 #include "fan_driver.h"
00056
00057
00058 #include "esp_adc/adc_oneshot.h"
00059 #include "ntc_driver.h"
00060 #include "pir_driver.h"
00061
00071 extern adc_oneshot_unit_handle_t s_ntc_adc_handle;
00072
00074 static fan_mode_t s_mode = FAN_MODE_MANUAL;
00075
00077 static uint8_t s_manual_speed = 0;
00078
00090 static uint8_t s_current_speed = 0;
00091
00099 static bool s_reg_active = false;
00100
00101
00102
00103 static uint8_t compute_auto_speed(float temp_c)
00104 {
00105     /*
00106      Reglas:
00107      - < 25°C => 0 %
00108      - 25-30°C => proporcional
00109      - > 30°C => 100 %
00110     */
00111
00112     if (temp_c < 25.0f) {
00113         return 0;
00114     }
00115
00116     if (temp_c >= 30.0f) {
00117         return 100;
00118     }
00119
00120     // Mapeo lineal 25 → 0% 30 → 100%
00121     float scale = (temp_c - 25.0f) / 5.0f; // 0 a 1
00122     return (uint8_t)(scale * 100.0f);
00123 }
00124
00146 void fan_control_init(void)
00147 {
00148     fan_init();
00149     s_mode = FAN_MODE_MANUAL;
00150     s_manual_speed = 0;
00151     s_current_speed = 0;
00152     s_reg_active = false;
00153     fan_off();
00154 }
00155
00184 void fan_control_set_mode(fan_mode_t mode)
00185 {
00186     s_mode = mode;
00187
00188     switch (s_mode) {
00189
00190     case FAN_MODE_MANUAL:
00191         // usa directamente la velocidad del slider
00192         s_current_speed = s_manual_speed;
00193         fan_set_speed_percent(s_current_speed);
00194         break;
00195
00196     case FAN_MODE_AUTO: {
00197         // Al cambiar a AUTO, aplicamos una primera vez lógica NTC+PIR
00198         if (s_ntc_adc_handle == NULL) {
00199             s_current_speed = 0;
00200             fan_off();
00201             break;
00202         }
00203
00204         float temp_c = leer_temperatura_celsius(s_ntc_adc_handle);
00205         if (temp_c < -100.0f) { // error NTC
00206             s_current_speed = 0;
00207             fan_off();
00208             break;
00209         }
00210
00211         bool motion = pir_is_motion_active();

```

```

00212         if (!motion) {
00213             s_current_speed = 0;
00214             fan_off();
00215             break;
00216         }
00217
00218         uint8_t spd = compute_auto_speed(temp_c);
00219         s_current_speed = spd;
00220         fan_set_speed_percent(s_current_speed);
00221         break;
00222     }
00223
00224     case FAN_MODE_REGISTERS:
00225         if (s_reg_active) {
00226             s_current_speed = s_manual_speed;
00227             fan_set_speed_percent(s_current_speed);
00228         } else {
00229             s_current_speed = 0;
00230             fan_off();
00231         }
00232         break;
00233     }
00234 }
00235
00244 fan_mode_t fan_control_get_mode(void)
00245 {
00246     return s_mode;
00247 }
00248
00266 void fan_control_set_manual_speed(uint8_t percent)
00267 {
00268     if (percent > 100) {
00269         percent = 100;
00270     }
00271     s_manual_speed = percent;
00272
00273     if (s_mode == FAN_MODE_MANUAL) {
00274         s_current_speed = s_manual_speed;
00275         fan_set_speed_percent(s_current_speed);
00276     }
00277 }
00278
00292 uint8_t fan_control_get_speed(void)
00293 {
00294     return s_current_speed; // Devolver velocidad REAL del PWM, no del slider
00295 }
00296
00318 void fan_control_on_register_tick(bool any_match_now)
00319 {
00320     s_reg_active = any_match_now;
00321
00322     // Solo tiene efecto en modo REGISTROS
00323     if (s_mode != FAN_MODE_REGISTERS) {
00324         return;
00325     }
00326
00327     if (s_reg_active) {
00328         // Aquí decides a qué velocidad se enciende cuando un registro coincide.
00329         // Yo uso la velocidad del slider (s_manual_speed).
00330         s_current_speed = s_manual_speed;
00331         fan_set_speed_percent(s_current_speed);
00332     } else {
00333         // Ningún registro activo → ventilador apagado
00334         s_current_speed = 0;
00335         fan_off();
00336     }
00337 }
00338
00367 void fan_control_update(void)
00368 {
00369     if (s_mode != FAN_MODE_AUTO) {
00370         return;
00371     }
00372
00373     if (s_ntc_adc_handle == NULL) {
00374         s_current_speed = 0;
00375         fan_off();
00376         return;
00377     }
00378
00379     float temp_c = leer_temperatura_celsius(s_ntc_adc_handle);
00380     if (temp_c < -100.0f) { // error NTC
00381         s_current_speed = 0;
00382         fan_off();
00383         return;
00384     }
00385 }

```

```

00386     bool motion = pir_is_motion_active();
00387     if (!motion) {
00388         s_current_speed = 0;
00389         fan_off();
00390         return;
00391     }
00392
00393     uint8_t auto_speed = compute_auto_speed(temp_c);
00394     s_current_speed = auto_speed;
00395     fan_set_speed_percent(s_current_speed);
00396 }

```

9.3. Referencia del archivo fan_control.h

Control automático del ventilador con múltiples modos de operación.

```

#include <stdint.h>
#include <stdbool.h>

```

Enumeraciones

- enum `fan_mode_t` { `FAN_MODE_MANUAL` = 0 , `FAN_MODE_AUTO` , `FAN_MODE_REGISTERS` }
- Modos de operación del ventilador.*

Funciones

- void `fan_control_init` (void)
Inicializar sistema de control del ventilador.
- void `fan_control_set_mode` (`fan_mode_t` mode)
Establecer modo de operación del ventilador.
- `fan_mode_t` `fan_control_get_mode` (void)
Obtener modo de operación actual.
- void `fan_control_set_manual_speed` (uint8_t percent)
Establecer velocidad manual del ventilador.
- uint8_t `fan_control_get_speed` (void)
Obtener velocidad actual del PWM.
- void `fan_control_update` (void)
Actualizar control (llamado cada 1s por fan_task).
- void `fan_control_on_register_tick` (bool any_match_now)
Notificar cambio en registros programados.

9.3.1. Descripción detallada

Control automático del ventilador con múltiples modos de operación.

Autor

Jair Hernan Telpis Cuaran, Luis Fernando Gamba Bedoya

Fecha

2025

Versión

1.0.0

Implementa la lógica de control del ventilador con tres modos:

- MANUAL: Control directo por slider de la interfaz web
- AUTO: Control automático basado en temperatura del sensor NTC
- REGISTERS: Control programado mediante registros horarios

Universidad Nacional de Colombia - Curso RTOS

Definición en el archivo [fan_control.h](#).

9.3.2. Documentación de enumeraciones

9.3.2.1. fan_mode_t

```
enum fan_mode_t
```

Modos de operación del ventilador.

Valores de enumeraciones

FAN_MODE_MANUAL	Control manual mediante slider web.
FAN_MODE_AUTO	Control automático por temperatura NTC.
FAN_MODE_REGISTERS	Control programado por registros horarios.

Definición en la línea 26 del archivo [fan_control.h](#).

9.3.3. Documentación de funciones

9.3.3.1. fan_control_get_mode()

```
fan_mode_t fan_control_get_mode (  
    void )
```

Obtener modo de operación actual.

Devuelve

[fan_mode_t](#) Modo actual del ventilador

Obtener modo de operación actual.

Devuelve

[fan_mode_t](#) Modo actual (MANUAL/AUTO/REGISTERS)

Nota

Thread-safe: lectura atómica de enum

Definición en la línea 244 del archivo [fan_control.c](#).

Hace referencia a [s_mode](#).

9.3.3.2. fan_control_get_speed()

```
uint8_t fan_control_get_speed (
    void )
```

Obtener velocidad actual del PWM.

Devuelve

uint8_t Velocidad en porcentaje (0-100)

Obtener velocidad actual del PWM.

Retorna s_current_speed, que es el duty cycle real del PWM. Puede diferir de s_manual_speed en modo AUTO.

Devuelve

uint8_t Velocidad actual (0-100 %)

Nota

Thread-safe: lectura atómica de uint8_t

Usado por OLED display y HTTP GET /api/dhtSensor

Definición en la línea 292 del archivo [fan_control.c](#).

Hace referencia a [s_current_speed](#).

9.3.3.3. fan_control_init()

```
void fan_control_init (
    void )
```

Inicializar sistema de control del ventilador.

Inicializar sistema de control del ventilador.

Secuencia de inicialización:

1. Inicializa hardware PWM ([fan_init\(\)](#))
2. Configura modo MANUAL por defecto
3. Velocidades a 0 %
4. Apaga ventilador

Estado inicial:

- Modo: FAN_MODE_MANUAL
- Velocidad manual: 0 %
- Velocidad actual: 0 %
- Registro activo: false
- PWM: apagado

Atención

Llamar antes de usar cualquier otra función de fan_control

Nota

Thread-safe: solo llamar una vez durante inicialización

Definición en la línea 146 del archivo fan_control.c.

Hace referencia a fan_init(), FAN_MODE_MANUAL, fan_off(), s_current_speed, s_manual_speed, s_mode y s_reg_active.

Referenciado por http_server_start().

9.3.3.4. fan_control_on_register_tick()

```
void fan_control_on_register_tick (
    bool any_match_now)
```

Notificar cambio en registros programados.

Parámetros

any_match_now	true si algún registro está activo
---------------	------------------------------------

Notificar cambio en registros programados.

Llamado cada 10 segundos por registers_scheduler_task. Solo tiene efecto si el modo actual es FAN_MODE_↔ REGISTERS.

Lógica:

- Si any_match_now = true: Enciende ventilador a velocidad del slider
- Si any_match_now = false: Apaga ventilador

En otros modos (MANUAL/AUTO), esta función no hace nada.

Parámetros

in	any_match_now	true si hay al menos un registro activo ahora
----	---------------	---

Nota

Llamado desde registers_scheduler_task (diferente tarea)

Atención

No es completamente thread-safe con cambios simultáneos de modo

Ver también

registers.c Para el scheduler que llama esta función

Definición en la línea 318 del archivo fan_control.c.

Hace referencia a FAN_MODE_REGISTERS, fan_off(), fan_set_speed_percent(), s_current_speed, s_manual_speed, s_mode y s_reg_active.

Referenciado por registers_scheduler_task().

9.3.3.5. `fan_control_set_manual_speed()`

```
void fan_control_set_manual_speed (
    uint8_t percent)
```

Establecer velocidad manual del ventilador.

Parámetros

<i>percent</i>	Velocidad en porcentaje (0-100)
----------------	---------------------------------

Establecer velocidad manual del ventilador.

Actualiza `s_manual_speed` y aplica inmediatamente si está en modo MANUAL.

Comportamiento por modo:

- MANUAL: Actualiza PWM inmediatamente
- AUTO: Solo guarda valor (no afecta PWM)
- REGISTERS: Solo guarda valor (se aplica cuando registro activo)

Parámetros

in	<i>percent</i>	Velocidad deseada (0-100 %)
----	----------------	-----------------------------

Nota

Valores > 100 se limitan automáticamente a 100

Atención

No es thread-safe: llamar solo desde HTTP server task

Definición en la línea 266 del archivo `fan_control.c`.

Hace referencia a `FAN_MODE_MANUAL`, `fan_set_speed_percent()`, `s_current_speed`, `s_manual_speed` y `s_mode`.

Referenciado por `http_server_fan_control_handler()`.

9.3.3.6. `fan_control_set_mode()`

```
void fan_control_set_mode (
    fan_mode_t mode)
```

Establecer modo de operación del ventilador.

Parámetros

<i>mode</i>	Nuevo modo (MANUAL, AUTO o REGISTERS)
-------------	---------------------------------------

Establecer modo de operación del ventilador.

Actualiza el modo y aplica lógica correspondiente inmediatamente:

MANUAL:

- Aplica velocidad del slider directamente
- No requiere sensores

AUTO:

- Lee temperatura NTC
- Calcula velocidad con [compute_auto_speed\(\)](#)
- Si ADC NULL o error NTC: apaga ventilador
- PIR deshabilitado en esta versión (comentado)

REGISTERS:

- Si hay registro activo: aplica velocidad del slider
- Si no hay registro: apaga ventilador

Parámetros

<i>in</i>	<i>mode</i>	Nuevo modo (FAN_MODE_MANUAL/AUTO/REGISTERS)
-----------	-------------	---

Atención

No es thread-safe: llamar solo desde HTTP server task

Nota

Cambio de modo aplica velocidad inmediatamente (no espera a fan_task)

Ver también

[fan_control_update\(\)](#) Para actualizaciones periódicas en modo AUTO

Definición en la línea 184 del archivo [fan_control.c](#).

Hace referencia a [compute_auto_speed\(\)](#), [FAN_MODE_AUTO](#), [FAN_MODE_MANUAL](#), [FAN_MODE_REGISTERS](#), [fan_off\(\)](#), [fan_set_speed_percent\(\)](#), [leer_temperatura_celsius\(\)](#), [pir_is_motion_active\(\)](#), [s_current_speed](#), [s_manual_speed](#), [s_mode](#), [s_ntc_adc_handle](#) y [s_reg_active](#).

Referenciado por [http_server_fan_control_handler\(\)](#).

9.3.3.7. fan_control_update()

```
void fan_control_update (
    void )
```

Actualizar control (llamado cada 1s por fan_task).

En modo AUTO ajusta velocidad según temperatura NTC

Actualizar control (llamado cada 1s por fan_task).

Llamado cada 1 segundo por fan_task. Solo tiene efecto si el modo actual es FAN_MODE_AUTO.

Algoritmo de actualización:

1. Verifica que el modo sea AUTO (si no, retorna)
2. Verifica que el ADC esté inicializado
3. Lee temperatura del NTC
4. Valida lectura ($> -100^{\circ}\text{C}$)
5. Calcula velocidad con [compute_auto_speed\(\)](#)
6. Actualiza PWM con [fan_set_speed_percent\(\)](#)

Manejo de errores:

- Si ADC handle es NULL: apaga ventilador
- Si temperatura $< -100^{\circ}\text{C}$: apaga ventilador (error NTC)
- PIR deshabilitado: no verifica movimiento (comentado)

Nota

Llamado desde fan_task cada 1 segundo

Solo procesa en modo AUTO (retorna inmediatamente en otros modos)

Atención

No es thread-safe con cambios de modo simultáneos

Ver también

[fan_task](#) en [http_server.c](#) Para la tarea que llama esta función

[compute_auto_speed\(\)](#) Para el algoritmo de cálculo

Definición en la línea [367](#) del archivo [fan_control.c](#).

Hace referencia a [compute_auto_speed\(\)](#), [FAN_MODE_AUTO](#), [fan_off\(\)](#), [fan_set_speed_percent\(\)](#), [leer_temperatura_celsius\(\)](#), [pir_is_motion_active\(\)](#), [s_current_speed](#), [s_mode](#) y [s_ntc_adc_handle](#).

Referenciado por [fan_task\(\)](#).

9.4. fan_control.h

[Ir a la documentación de este archivo.](#)

```
00001
00015
00016 #ifndef MAIN_FAN_CONTROL_H_
00017 #define MAIN_FAN_CONTROL_H_
00018
00019 #include <stdint.h>
00020 #include <stdbool.h>
00021
00026 typedef enum {
00027     FAN_MODE_MANUAL = 0,
00028     FAN_MODE_AUTO,
00029     FAN_MODE_REGISTERS
00030 } fan_mode_t;
00031
00033 void fan_control_init(void);
00034
00039 void fan_control_set_mode(fan_mode_t mode);
00040
00045 fan_mode_t fan_control_get_mode(void);
00046
00051 void fan_control_set_manual_speed(uint8_t percent);
00052
00057 uint8_t fan_control_get_speed(void);
00058
00063 void fan_control_update(void);
00064
00069 void fan_control_on_register_tick(bool any_match_now);
00070
00071 #endif // MAIN_FAN_CONTROL_H_
```

9.5. Referencia del archivo fan_driver.c

```
#include "fan_driver.h"
```

Funciones

- void [fan_init](#) (void)
- void [fan_set_speed_percent](#) (uint8_t percent)
- void [fan_off](#) (void)

Variables

- static bool [s_fan_initialized](#) = false

9.5.1. Documentación de funciones

9.5.1.1. fan_init()

```
void fan_init (
    void )
```

Definición en la línea 6 del archivo [fan_driver.c](#).

Hace referencia a [FAN_LEDC_CHANNEL](#), [FAN_LEDC_DUTY_RES](#), [FAN_LEDC_FREQ_HZ](#), [FAN_LEDC_MODE](#), [FAN_LEDC_TIMER](#), [FAN_PWM_GPIO](#) y [s_fan_initialized](#).

Referenciado por [fan_control_init\(\)](#) y [fan_set_speed_percent\(\)](#).

9.5.1.2. `fan_off()`

```
void fan_off (
    void )
```

Definición en la línea 43 del archivo `fan_driver.c`.

Hace referencia a `fan_set_speed_percent()`.

Referenciado por `fan_control_init()`, `fan_control_on_register_tick()`, `fan_control_set_mode()` y `fan_control_update()`.

9.5.1.3. `fan_set_speed_percent()`

```
void fan_set_speed_percent (
    uint8_t percent)
```

Definición en la línea 33 del archivo `fan_driver.c`.

Hace referencia a `fan_init()`, `FAN_LEDC_CHANNEL`, `FAN_LEDC_MODE` y `s_fan_initialized`.

Referenciado por `fan_control_on_register_tick()`, `fan_control_set_manual_speed()`, `fan_control_set_mode()`, `fan_control_update()` y `fan_off()`.

9.5.2. Documentación de variables

9.5.2.1. `s_fan_initialized`

```
bool s_fan_initialized = false [static]
```

Definición en la línea 4 del archivo `fan_driver.c`.

Referenciado por `fan_init()` y `fan_set_speed_percent()`.

9.6. `fan_driver.c`

[Ir a la documentación de este archivo.](#)

```
00001 // fan_driver.c
00002 #include "fan_driver.h"
00003
00004 static bool s_fan_initialized = false;
00005
00006 void fan_init(void)
00007 {
00008     if (s_fan_initialized) return;
00009
00010     ledc_timer_config_t ledc_timer = {
00011         .duty_resolution = FAN_LEDC_DUTY_RES,
00012         .freq_hz         = FAN_LEDC_FREQ_HZ,
00013         .speed_mode      = FAN_LEDC_MODE,
00014         .timer_num       = FAN_LEDC_TIMER,
00015         .clk_cfg         = LEDC_AUTO_CLK,
00016     };
00017     ledc_timer_config(&ledc_timer);
00018
00019     ledc_channel_config_t ledc_channel = {
00020         .channel = FAN_LEDC_CHANNEL,
00021         .duty    = 0,
00022         .hpoint  = 0,
00023         .gpio_num = FAN_PWM_GPIO,
```



```

00024         .intr_type = LEDC_INTR_DISABLE,
00025         .speed_mode = FAN_LEDC_MODE,
00026         .timer_sel = FAN_LEDC_TIMER,
00027     };
00028     ledc_channel_config(&ledc_channel);
00029
00030     s_fan_initialized = true;
00031 }
00032
00033 void fan_set_speed_percent(uint8_t percent)
00034 {
00035     if (!s_fan_initialized) fan_init();
00036     if (percent > 100) percent = 100;
00037
00038     uint32_t duty = (255 * percent) / 100;
00039     ledc_set_duty(FAN_LEDC_MODE, FAN_LEDC_CHANNEL, duty);
00040     ledc_update_duty(FAN_LEDC_MODE, FAN_LEDC_CHANNEL);
00041 }
00042
00043 void fan_off(void)
00044 {
00045     fan_set_speed_percent(0);
00046 }

```

9.7. Referencia del archivo fan_driver.h

```

#include <stdint.h>
#include <stdbool.h>
#include "driver/ledc.h"

```

defines

- #define FAN_PWM_GPIO 18
- #define FAN_LEDC_CHANNEL LEDC_CHANNEL_3
- #define FAN_LEDC_MODE LEDC_LOW_SPEED_MODE
- #define FAN_LEDC_TIMER LEDC_TIMER_1
- #define FAN_LEDC_DUTY_RES LEDC_TIMER_8_BIT
- #define FAN_LEDC_FREQ_HZ 25000

Funciones

- void fan_init(void)
- void fan_set_speed_percent(uint8_t percent)
- void fan_off(void)

9.7.1. Documentación de «define»

9.7.1.1. FAN_LEDC_CHANNEL

```
#define FAN_LEDC_CHANNEL LEDC_CHANNEL_3
```

Definición en la línea 11 del archivo [fan_driver.h](#).

Referenciado por [fan_init\(\)](#) y [fan_set_speed_percent\(\)](#).

9.7.1.2. FAN_LEDC_DUTY_RES

```
#define FAN_LEDC_DUTY_RES LEDC_TIMER_8_BIT
```

Definición en la línea 14 del archivo [fan_driver.h](#).

Referenciado por [fan_init\(\)](#).

9.7.1.3. FAN_LEDC_FREQ_HZ

```
#define FAN_LEDC_FREQ_HZ 25000
```

Definición en la línea 15 del archivo [fan_driver.h](#).

Referenciado por [fan_init\(\)](#).

9.7.1.4. FAN_LEDC_MODE

```
#define FAN_LEDC_MODE LEDC_LOW_SPEED_MODE
```

Definición en la línea 12 del archivo [fan_driver.h](#).

Referenciado por [fan_init\(\)](#) y [fan_set_speed_percent\(\)](#).

9.7.1.5. FAN_LEDC_TIMER

```
#define FAN_LEDC_TIMER LEDC_TIMER_1
```

Definición en la línea 13 del archivo [fan_driver.h](#).

Referenciado por [fan_init\(\)](#).

9.7.1.6. FAN_PWM_GPIO

```
#define FAN_PWM_GPIO 18
```

Definición en la línea 10 del archivo [fan_driver.h](#).

Referenciado por [fan_init\(\)](#).

9.7.2. Documentación de funciones

9.7.2.1. fan_init()

```
void fan_init (  
    void )
```

Definición en la línea 6 del archivo [fan_driver.c](#).

Hace referencia a [FAN_LEDC_CHANNEL](#), [FAN_LEDC_DUTY_RES](#), [FAN_LEDC_FREQ_HZ](#), [FAN_LEDC_MODE](#), [FAN_LEDC_TIMER](#), [FAN_PWM_GPIO](#) y [s_fan_initialized](#).

Referenciado por [fan_control_init\(\)](#) y [fan_set_speed_percent\(\)](#).

9.7.2.2. fan_off()

```
void fan_off (
    void )
```

Definición en la línea 43 del archivo [fan_driver.c](#).

Hace referencia a [fan_set_speed_percent\(\)](#).

Referenciado por [fan_control_init\(\)](#), [fan_control_on_register_tick\(\)](#), [fan_control_set_mode\(\)](#) y [fan_control_update\(\)](#).

9.7.2.3. fan_set_speed_percent()

```
void fan_set_speed_percent (
    uint8_t percent)
```

Definición en la línea 33 del archivo [fan_driver.c](#).

Hace referencia a [fan_init\(\)](#), [FAN_LEDC_CHANNEL](#), [FAN_LEDC_MODE](#) y [s_fan_initialized](#).

Referenciado por [fan_control_on_register_tick\(\)](#), [fan_control_set_manual_speed\(\)](#), [fan_control_set_mode\(\)](#), [fan_control_update\(\)](#) y [fan_off\(\)](#).

9.8. fan_driver.h

[Ir a la documentación de este archivo.](#)

```
00001 // fan_driver.h
00002 #ifndef MAIN_FAN_DRIVER_H_
00003 #define MAIN_FAN_DRIVER_H_
00004
00005 #include <stdint.h>
00006 #include <stdbool.h>
00007 #include "driver/ledc.h"
00008
00009 // GPIO y canal para el ventilador
00010 #define FAN_PWM_GPIO      18          // cambiado a pin seguro (antes 21, conflicto con RGB_BLUE)
00011 #define FAN_LEDC_CHANNEL  LEDC_CHANNEL_3
00012 #define FAN_LEDC_MODE      LEDC_LOW_SPEED_MODE
00013 #define FAN_LEDC_TIMER     LEDC_TIMER_1
00014 #define FAN_LEDC_DUTY_RES  LEDC_TIMER_8_BIT // 0-255
00015 #define FAN_LEDC_FREQ_HZ   25000         // ~25 kHz
00016
00017 void fan_init(void);
00018 void fan_set_speed_percent(uint8_t percent); // 0-100
00019 void fan_off(void);
00020
00021 #endif
```

9.9. Referencia del archivo http_server.c

Servidor HTTP con REST API para control del sistema.

```
#include "esp_http_server.h"
#include "esp_log.h"
#include "esp_ota_ops.h"
#include "esp_timer.h"
#include "sys/param.h"
#include <stdlib.h>
#include "http_server.h"
#include "tasks_common.h"
#include "wifi_app.h"
#include "cJSON.h"
#include "driver/gpio.h"
#include <stdio.h>
#include <stdint.h>
#include <stddef.h>
#include <string.h>
#include <time.h>
#include "esp_wifi.h"
#include "esp_system.h"
#include "nvs_flash.h"
#include "esp_event.h"
#include "esp_netif.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "freertos/queue.h"
#include "lwip/sockets.h"
#include "lwip/dns.h"
#include "lwip/netdb.h"
#include "esp_sntp.h"
#include "ntc_driver.h"
#include "pir_driver.h"
#include "esp_adc/adc_oneshot.h"
#include "fan_control.h"
#include "registers.h"
#include "sensor_task.h"
```

Funciones

- static esp_err_t [http_server_read_register_handler](#) (httpd_req_t *req)
- static esp_err_t [http_server_register_erase_handler](#) (httpd_req_t *req)
Handler POST /regerase.json - Borrar registro (legacy).
- const uint8_t jquery_3_3_1_min_js_start[] [asm](#) ("_binary_jquery_3_3_1_min_js_start")
Inicio de jquery 3.3.1 en memoria.
- const uint8_t jquery_3_3_1_min_js_end[] [asm](#) ("_binary_jquery_3_3_1_min_js_end")
Fin de jquery 3.3.1 en memoria.
- const uint8_t index_html_start[] [asm](#) ("_binary_index_html_start")
Inicio de [index.html](#) en memoria.
- const uint8_t index_html_end[] [asm](#) ("_binary_index_html_end")
Fin de [index.html](#) en memoria.
- const uint8_t app_css_start[] [asm](#) ("_binary_app_css_start")

- Inicio de app.css en memoria.*
- `const uint8_t app_css_end[] asm ("_binary_app_css_end")`
- Fin de app.css en memoria.*
- `const uint8_t app_js_start[] asm ("_binary_app_js_start")`
- Inicio de app.js en memoria.*
- `const uint8_t app_js_end[] asm ("_binary_app_js_end")`
- Fin de app.js en memoria.*
- `const uint8_t favicon_ico_start[] asm ("_binary_favicon_ico_start")`
- Inicio de favicon.ico en memoria.*
- `const uint8_t favicon_ico_end[] asm ("_binary_favicon_ico_end")`
- Fin de favicon.ico en memoria.*
- `void toggle_led (void)`
- Alterna el estado del LED de depuración.*
- `static esp_err_t http_server_fan_control_handler (httpd_req_t *req)`
- Handler POST /fanControl.json - Control del ventilador.*
- `static esp_err_t http_server_time_json_handler (httpd_req_t *req)`
- Handler GET /time.json - Obtiene hora del sistema.*
- `static esp_err_t http_server_toggle_led_handler (httpd_req_t *req)`
- `static void log_error_if_nonzero (const char *message, int error_code)`
- `static void sensors_init (void)`
- Inicializa periféricos de sensores.*
- `static void http_server_monitor (void *parameter)`
- Tarea monitor de eventos WiFi.*
- `static void http_server_fw_update_reset_timer (void)`
- `static esp_err_t http_server_jquery_handler (httpd_req_t *req)`
- Handler GET /jquery-3.3.1.min.js.*
- `static esp_err_t http_server_index_html_handler (httpd_req_t *req)`
- Handler GET / (index.html).*
- `static esp_err_t http_server_app_css_handler (httpd_req_t *req)`
- Handler GET /app.css.*
- `static esp_err_t http_server_app_js_handler (httpd_req_t *req)`
- Handler GET /app.js.*
- `static esp_err_t http_server_favicon_ico_handler (httpd_req_t *req)`
- Handler GET /favicon.ico.*
- `esp_err_t http_server_OTA_update_handler (httpd_req_t *req)`
- Handler POST /OTAupdate - Actualización de firmware OTA.*
- `esp_err_t http_server_OTA_status_handler (httpd_req_t *req)`
- `static esp_err_t http_server_wifi_connect_json_handler (httpd_req_t *req)`
- Handler POST /wifiConnect.json - Conectar a red WiFi.*
- `static esp_err_t http_server_wifi_connect_status_json_handler (httpd_req_t *req)`
- Handler GET /wifiConnectStatus - Estado de conexión WiFi.*
- `static esp_err_t http_server_get_dht_sensor_readings_json_handler (httpd_req_t *req)`
- Handler GET /dhtSensor.json - Lectura de sensores.*
- `static httpd_handle_t http_server_configure (void)`
- Configura e inicia el servidor HTTP con todos los endpoints.*
- `static void fan_task (void *arg)`
- Tarea RTOS que actualiza el ventilador en modo AUTO.*
- `void http_server_start (void)`
- Inicia el servidor HTTP y sus tareas asociadas.*
- `void http_server_stop (void)`
- Detiene el servidor HTTP y limpia recursos.*

- BaseType_t [http_server_monitor_send_message](#) ([http_server_message_e](#) msgID)
Enviar mensaje a la cola del monitor HTTP.
- void [http_server_fw_update_reset_callback](#) (void *arg)
Callback de timer para reinicio tras OTA exitoso.

Variables

- static const char [TAG](#) [] = "http_server"
Tag para logging ESP-IDF.
- adc_oneshot_unit_handle_t [s_ntc_adc_handle](#) = NULL
Handle del ADC para sensor NTC (compartido globalmente).
- static int [g_wifi_connect_status](#) = NONE
Estado de conexión WiFi (NONE, CONNECTING, SUCCESS, FAIL).
- static int [g_fw_update_status](#) = 0
Estado de actualización OTA (PENDING, SUCCESSFUL, FAILED).
- static httpd_handle_t [http_server_handle](#) = NULL
Handle del servidor HTTP.
- static TaskHandle_t [task_http_server_monitor](#) = NULL
Handle de tarea monitora de eventos HTTP/WiFi.
- static QueueHandle_t [http_server_monitor_queue_handle](#)
Cola de mensajes para http_server_monitor.
- const esp_timer_create_args_t [fw_update_reset_args](#)
Configuración del timer para reinicio post-OTA.
- esp_timer_handle_t [fw_update_reset](#)
Handle del timer de reinicio OTA.
- uint8_t [s_led_state](#) = 0
Estado del LED de depuración.

9.9.1. Descripción detallada

Servidor HTTP con REST API para control del sistema.

Autor

Jair Hernan Telpis Cuaran, Luis Fernando Gamba Bedoya

Fecha

2025

Implementa servidor HTTP completo con endpoints REST para:

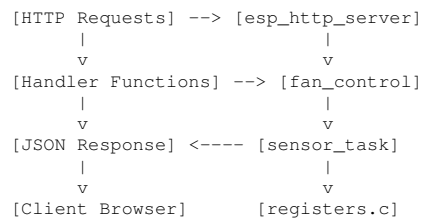
- Control de ventilador (manual/auto/registros)
- Lectura de sensores (temperatura, PIR)
- Gestión de registros programados (CRUD)
- Configuración WiFi
- Actualización OTA de firmware

- Servicio de archivos estáticos (HTML, CSS, JS)

Endpoints principales:

- GET `/dhtSensor.json` → Sensores (temp, PIR)
- POST `/fanControl.json` → Control ventilador
- GET `/api/registers` → Lista todos los registros
- POST `/api/register` → Crear/actualizar registro
- DELETE `/api/register/{id}` → Eliminar registro
- POST `/wifiConnect.json` → Conectar WiFi
- POST `/OTAupdate` → Actualización firmware

Arquitectura:



Tareas RTOS:

- `http_server_monitor`: Monitorea eventos de WiFi y OTA
- `fan_task`: Actualiza ventilador cada 1s en modo AUTO

Archivos embebidos:

- [index.html](#): Página principal
- [app.js](#): Lógica JavaScript
- `app.css`: Estilos
- `jquery-3.3.1.min.js`: Biblioteca jQuery
- `favicon.ico`: Icono del sitio

Ver también

[http_server.h](#) Para la API pública

[registers.c](#) Para gestión de programación

[fan_control.c](#) Para control del ventilador

Definición en el archivo [http_server.c](#).

9.9.2. Documentación de funciones

9.9.2.1. fan_task()

```
void fan_task (
    void * arg) [static]
```

Tarea RTOS que actualiza el ventilador en modo AUTO.

Llama [fan_control_update\(\)](#) cada 1 segundo. Solo tiene efecto si el modo es FAN_MODE_AUTO.

Parámetros tarea:

- Stack: 2048 bytes
- Prioridad: configMAX_PRIORITIES - 4 (media-baja)
- Periodo: 1000ms

Parámetros

in	arg	Parámetro de tarea (no usado)
----	-----	-------------------------------

Nota

Tarea infinita creada en [http_server_start\(\)](#)

Ver también

[fan_control_update\(\)](#) Función llamada cada segundo

Definición en la línea 1345 del archivo [http_server.c](#).

Hace referencia a [fan_control_update\(\)](#).

Referenciado por [http_server_start\(\)](#).

9.9.2.2. http_server_app_css_handler()

```
esp_err_t http_server_app_css_handler (
    httpd_req_t * req) [static]
```

Handler GET /app.css.

Sirve estilos CSS embebidos desde flash. Content-Type: text/css

Parámetros

in	req	Request HTTP
----	-----	--------------

Devuelve

esp_err_t Siempre ESP_OK

Definición en la línea 568 del archivo [http_server.c](#).

Hace referencia a [TAG](#).

Referenciado por [http_server_configure\(\)](#).

9.9.2.3. http_server_app_js_handler()

```
esp_err_t http_server_app_js_handler (
    httpd_req_t * req) [static]
```

Handler GET /app.js.

Sirve JavaScript de aplicación embebido desde flash. Content-Type: application/javascript

Parámetros

in	req	Request HTTP
----	-----	--------------

Devuelve

esp_err_t Siempre ESP_OK

Definición en la línea 589 del archivo [http_server.c](#).

Hace referencia a [TAG](#).

Referenciado por [http_server_configure\(\)](#).

9.9.2.4. http_server_configure()

```
httpd_handle_t http_server_configure (
    void ) [static]
```

Configura e inicia el servidor HTTP con todos los endpoints.

Crea servidor HTTP y registra todos los handlers:

Archivos estáticos:

- GET / → [index.html](#)
- GET /jquery-3.3.1.min.js
- GET /app.css
- GET /app.js
- GET /favicon.ico

API REST:

- POST /fanControl.json
- GET /dhtSensor.json
- POST /wifiConnect.json
- GET /wifiConnectStatus
- GET /api/registers ([registers.c](#))

- POST /api/register ([registers.c](#))
- DELETE /api/register/* ([registers.c](#))

OTA:

- POST /OTAupdate
- GET /OTAsatus

Configuración servidor:

- Puerto: 80
- Stack: 4096 bytes por conexión
- Conexiones simultáneas: CONFIG_LWIP_MAX_SOCKETS

Devuelve

httpd_handle_t Handle del servidor o NULL si error

Nota

Registra endpoints de [registers.c](#) automáticamente

Definición en la línea 1086 del archivo [http_server.c](#).

Hace referencia a [http_server_app_css_handler\(\)](#), [http_server_app_js_handler\(\)](#), [http_server_fan_control_handler\(\)](#), [http_server_favicon_ico_handler\(\)](#), [http_server_get_dht_sensor_readings_json_handler\(\)](#), [http_server_handle](#), [http_server_index_html_handler\(\)](#), [http_server_jquery_handler\(\)](#), [http_server_monitor\(\)](#), [http_server_monitor_queue_handle](#), [http_server_OTA_status_handler\(\)](#), [http_server_OTA_update_handler\(\)](#), [http_server_read_register_handler\(\)](#), [http_server_register_erase_handler\(\)](#), [http_server_time_json_handler\(\)](#), [http_server_toogle_led_handler\(\)](#), [http_server_wifi_connect_json_handler\(\)](#), [http_server_wifi_connect_status_json_handler\(\)](#), [register_registers_endpoints\(\)](#), TAG y [task_http_server_monitor](#).

Referenciado por [http_server_start\(\)](#).

9.9.2.5. http_server_fan_control_handler()

```
esp_err_t http_server_fan_control_handler (
    httpd_req_t * req) [static]
```

Handler POST /fanControl.json - Control del ventilador.

Endpoint para controlar el ventilador desde la interfaz web.

Request JSON:

```
{
  "mode": "manual" | "auto" | "registers",
  "speed": 0-100 // Solo para modo manual
}
```

Proceso:

1. Parsea JSON del body
2. Extrae modo y velocidad
3. Llama [fan_control_set_mode\(\)](#)
4. Llama [fan_control_set_manual_speed\(\)](#) si aplica
5. Retorna "OK"

Parámetros

<code>in</code>	<code>req</code>	Request HTTP
-----------------	------------------	--------------

Devuelve

`esp_err_t`

Valores devueltos

<code>ESP_OK</code>	Comando aplicado exitosamente
<code>ESP_FAIL</code>	Error en parsing o memoria

Definición en la línea 233 del archivo `http_server.c`.

Hace referencia a `fan_control_set_manual_speed()`, `fan_control_set_mode()`, `FAN_MODE_AUTO`, `FAN_MODE_MANUAL`, `FAN_MODE_REGISTERS` y `TAG`.

Referenciado por `http_server_configure()`.

9.9.2.6. `http_server_favicon_ico_handler()`

```
esp_err_t http_server_favicon_ico_handler (  
    httpd_req_t * req) [static]
```

Handler GET /favicon.ico.

Sirve favicon embebido desde flash. Content-Type: image/x-icon

Parámetros

<code>in</code>	<code>req</code>	Request HTTP
-----------------	------------------	--------------

Devuelve

`esp_err_t` Siempre `ESP_OK`

Definición en la línea 610 del archivo `http_server.c`.

Hace referencia a `TAG`.

Referenciado por `http_server_configure()`.

9.9.2.7. `http_server_fw_update_reset_callback()`

```
void http_server_fw_update_reset_callback (  
    void * arg)
```

Callback de timer para reinicio tras OTA exitoso.

Se ejecuta 8 segundos después de una actualización OTA exitosa, dando tiempo a que el navegador reciba la respuesta HTTP antes de reiniciar.

Parámetros

<i>arg</i>	Argumento del timer (sin usar)
------------	--------------------------------

Nota

Llama a `esp_restart()` para aplicar nuevo firmware

Definición en la línea 1430 del archivo [http_server.c](#).

Hace referencia a [TAG](#).

9.9.2.8. http_server_fw_update_reset_timer()

```
void http_server_fw_update_reset_timer (
    void ) [static]
```

< Actualización OTA exitosa

Definición en la línea 451 del archivo [http_server.c](#).

Hace referencia a [fw_update_reset](#), [fw_update_reset_args](#), [g_fw_update_status](#), [OTA_UPDATE_SUCCESSFUL](#) y [TAG](#).

Referenciado por [http_server_monitor\(\)](#).

9.9.2.9. http_server_get_dht_sensor_readings_json_handler()

```
esp_err_t http_server_get_dht_sensor_readings_json_handler (
    httpd_req_t * req) [static]
```

Handler GET /dhtSensor.json - Lectura de sensores.

Endpoint para leer temperatura NTC y sensor PIR.

Response JSON:

```
{"temp": 25.5, "pir": 1} // Temp válida, movimiento detectado
{"temp": null, "pir": 0} // Error NTC, sin movimiento
```

Valores:

- temp: Temperatura en °C o null si error (< -100°C)
- pir: 1 = movimiento, 0 = sin movimiento

Parámetros

in	<i>req</i>	Request HTTP
----	------------	--------------

Devuelve

`esp_err_t` ESP_OK siempre

Nota

Llamado cada 1-2 segundos desde [app.js](#)

Definición en la línea 935 del archivo [http_server.c](#).

Hace referencia a [leer_temperatura_celsius\(\)](#), [pir_is_motion_active\(\)](#), [s_ntc_adc_handle](#) y [TAG](#).

Referenciado por [http_server_configure\(\)](#).

9.9.2.10. http_server_index_html_handler()

```
esp_err_t http_server_index_html_handler (  
    httpd_req_t * req) [static]
```

Handler GET / ([index.html](#)).

Sirve página principal HTML embebida desde flash. Content-Type: text/html

Parámetros

in	req	Request HTTP
----	-----	--------------

Devuelve

esp_err_t Siempre ESP_OK

Definición en la línea [547](#) del archivo [http_server.c](#).

Hace referencia a [TAG](#).

Referenciado por [http_server_configure\(\)](#).

9.9.2.11. http_server_jquery_handler()

```
esp_err_t http_server_jquery_handler (  
    httpd_req_t * req) [static]
```

Handler GET /jquery-3.3.1.min.js.

Sirve jQuery embebido desde flash. Content-Type: application/javascript

Parámetros

in	req	Request HTTP
----	-----	--------------

Devuelve

esp_err_t Siempre ESP_OK

Definición en la línea [526](#) del archivo [http_server.c](#).

Hace referencia a [TAG](#).

Referenciado por [http_server_configure\(\)](#).

9.9.2.12. `http_server_monitor()`

```
void http_server_monitor (
    void * parameter) [static]
```

Tarea monitor de eventos WiFi.

Recibe mensajes de `wifi_app` vía `http_server_monitor_queue_handle`.

Mensajes:

- `HTTP_MSG_WIFI_CONNECT_INIT`: Intento de conexión
- `HTTP_MSG_WIFI_CONNECT_SUCCESS`: Conexión exitosa
- `HTTP_MSG_WIFI_CONNECT_FAIL`: Fallo en conexión

Funcionalidad:

- Actualiza `g_wifi_connect_status`
- Envía mensajes LED RGB para indicar estado

Parámetros

in	<i>parameter</i>	Parámetro de tarea (no usado)
----	------------------	-------------------------------

Nota

Tarea infinita, stack 3072 bytes, prioridad 4

< Actualización OTA exitosa

< Actualización OTA fallida

Definición en la línea 468 del archivo [http_server.c](#).

Hace referencia a [g_fw_update_status](#), [g_wifi_connect_status](#), [HTTP_MSG_OTA_UPDATE_FAILED](#), [HTTP_MSG_OTA_UPDATE_SUCCESS](#), [HTTP_MSG_WIFI_CONNECT_FAIL](#), [HTTP_MSG_WIFI_CONNECT_INIT](#), [HTTP_MSG_WIFI_CONNECT_SUCCESS](#), [http_server_fw_update_reset_timer\(\)](#), [http_server_monitor_queue_handle](#), [HTTP_WIFI_STATUS_CONNECT_FAILED](#), [HTTP_WIFI_STATUS_CONNECT_SUCCESS](#), [HTTP_WIFI_STATUS_CONNECTING](#), [OTA_UPDATE_FAILED](#), [OTA_UPDATE_SUCCESSFUL](#) y [TAG](#).

Referenciado por [http_server_configure\(\)](#).

9.9.2.13. `http_server_monitor_send_message()`

```
BaseType_t http_server_monitor_send_message (
    http_server_message_e msgID)
```

Enviar mensaje a la cola del monitor HTTP.

Permite comunicación thread-safe entre módulos del sistema y el servidor HTTP. Los mensajes se procesan en orden FIFO por la tarea `http_server_monitor`.

Parámetros

<i>msgID</i>	ID del mensaje desde http_server_message_e
--------------	--

Devuelve

pdTRUE si el mensaje se envió correctamente
 pdFALSE si la cola está llena (espera indefinida activada)

Nota

Esta función usa `portMAX_DELAY`, bloqueará hasta que haya espacio en la cola

Definición en la línea 1423 del archivo [http_server.c](#).

Hace referencia a [http_server_monitor_queue_handle](#).

Referenciado por [http_server_OTA_update_handler\(\)](#) y [wifi_app_task\(\)](#).

9.9.2.14. http_server_OTA_status_handler()

```
esp_err_t http_server_OTA_status_handler (
    httpd_req_t * req)
```

Definición en la línea 743 del archivo [http_server.c](#).

Hace referencia a [g_fw_update_status](#) y [TAG](#).

Referenciado por [http_server_configure\(\)](#).

9.9.2.15. http_server_OTA_update_handler()

```
esp_err_t http_server_OTA_update_handler (
    httpd_req_t * req)
```

Handler POST /OTAupdate - Actualización de firmware OTA.

Recibe archivo binario (.bin) y actualiza firmware del ESP32.

Proceso:

1. Inicia partición OTA con `esp_ota_begin()`
2. Recibe datos en chunks de 1024 bytes
3. Escribe cada chunk con `esp_ota_write()`
4. Finaliza con `esp_ota_end()`
5. Marca partición como booteable
6. Programa reinicio en 8 segundos

Request:

- Content-Type: multipart/form-data
- Body: Archivo .bin del firmware

Estados:

- `g_fw_update_status = OTA_UPDATE_SUCCESSFUL`: Éxito
- `g_fw_update_status = OTA_UPDATE_FAILED`: Error

Parámetros

<i>in</i>	<i>req</i>	Request HTTP
-----------	------------	--------------

Devuelve

`esp_err_t`

Valores devueltos

<code>ESP_OK</code>	Actualización exitosa
<code>ESP_FAIL</code>	Error en proceso

Atención

El ESP32 se reinicia 8s después de éxito

Nota

Usa LED RGB para indicar progreso

Definición en la línea 655 del archivo [http_server.c](#).

Hace referencia a [HTTP_MSG_OTA_UPDATE_FAILED](#), [HTTP_MSG_OTA_UPDATE_SUCCESSFUL](#), [http_server_monitor_send_me](#) y [TAG](#).

Referenciado por [http_server_configure\(\)](#).

9.9.2.16. `http_server_read_register_handler()`

```
esp_err_t http_server_read_register_handler (
    httpd_req_t * req) [static]
```

Definición en la línea 1256 del archivo [http_server.c](#).

Hace referencia a [load_register_from_nvs\(\)](#) y [TAG](#).

Referenciado por [http_server_configure\(\)](#).

9.9.2.17. `http_server_register_erase_handler()`

```
esp_err_t http_server_register_erase_handler (
    httpd_req_t * req) [static]
```

Handler POST /regerase.json - Borrar registro (legacy).

Endpoint legacy para borrar registros. Usa JSON diferente a API REST.

Request JSON:

```
{"selectedNumber": "3"} // String, no número
```

Proceso:

1. Parsea JSON
2. Extrae selectedNumber (como string)
3. Convierte a entero
4. Valida rango 1-10
5. Llama [delete_register_from_nvs\(\)](#)

Parámetros

<code>in</code>	<code>req</code>	Request HTTP
-----------------	------------------	--------------

Devuelve

`esp_err_t`

Valores devueltos

<code>ESP_OK</code>	Registro eliminado
<code>ESP_FAIL</code>	Error en parsing o ID inválido

Nota

Prefer usar DELETE `/api/register/{id}` (API moderna)

Definición en la línea 991 del archivo `http_server.c`.

Hace referencia a `delete_register_from_nvs()` y TAG.

Referenciado por `http_server_configure()`.

9.9.2.18. `http_server_start()`

```
void http_server_start (
    void )
```

Inicia el servidor HTTP y sus tareas asociadas.

Iniciar servidor HTTP y tareas relacionadas.

Secuencia de inicialización completa:

1. Configura periféricos (ADC, NTC, PIR, sensor_task)
2. Inicializa control del ventilador
3. Crea cola de mensajes `http_server_monitor`
4. Crea tarea `http_server_monitor`
5. Configura y arranca servidor HTTP
6. Crea `fan_task` para modo AUTO

Nota

Llamar después de que WiFi obtenga IP

Atención

Debe llamarse solo una vez

Ver también

`http_server_configure()` Configuración del servidor

`fan_task()` Tarea de actualización automática

Definición en la línea 1375 del archivo `http_server.c`.

Hace referencia a `fan_control_init()`, `fan_task()`, `http_server_configure()`, `http_server_handle`, `sensors_init()` y TAG.

Referenciado por `wifi_app_task()`.

9.9.2.19. `http_server_stop()`

```
void http_server_stop (
    void )
```

Detiene el servidor HTTP y limpia recursos.

Detener servidor HTTP y liberar recursos.

Secuencia de parada:

1. Detiene servidor HTTP con `httpd_stop()`
2. Elimina tarea `http_server_monitor`
3. Libera cola `http_server_monitor_queue_handle`

Nota

Solo ejecuta si `http_server_handle != NULL`

Atención

No detiene `fan_task` ni `sensor_task`

Definición en la línea 1404 del archivo [http_server.c](#).

Hace referencia a [http_server_handle](#), [TAG](#) y [task_http_server_monitor](#).

9.9.2.20. `http_server_time_json_handler()`

```
esp_err_t http_server_time_json_handler (
    httpd_req_t * req) [static]
```

Handler GET /time.json - Obtiene hora del sistema.

Retorna hora actual en formato JSON si SNTP está sincronizado.

Response JSON:

```
{
  "hour": 14,
  "minute": 30,
  "second": 45,
  "day": 15,
  "month": 3,
  "year": 2024
}
```

Estado: Solo retorna si `get_state_time_was_synchronized() == true`

Parámetros

in	<i>req</i>	Request HTTP
----	------------	--------------

Devuelve

`esp_err_t`

Valores devueltos

<code>ESP_OK</code>	Hora enviada
<code>ESP_FAIL</code>	SNTP no sincronizado

Definición en la línea 329 del archivo `http_server.c`.

Hace referencia a `TAG`.

Referenciado por `http_server_configure()`.

9.9.2.21. `http_server_toggle_led_handler()`

```
esp_err_t http_server_toggle_led_handler (
    httpd_req_t * req) [static]
```

Definición en la línea 359 del archivo `http_server.c`.

Hace referencia a `TAG` y `toggle_led()`.

Referenciado por `http_server_configure()`.

9.9.2.22. `http_server_wifi_connect_json_handler()`

```
esp_err_t http_server_wifi_connect_json_handler (
    httpd_req_t * req) [static]
```

Handler POST `/wifiConnect.json` - Conectar a red WiFi.

Endpoint para configurar credenciales WiFi y conectar.

Request JSON:

```
{
  "selectedSSID": "MiRed",
  "pwd": "password123"
}
```

Proceso:

1. Parsea JSON del body
2. Extrae SSID y password
3. Guarda en NVS con `save_wifi_credentials()`
4. Envía mensaje a `wifi_app` para conectar
5. Retorna "OK"

Parámetros

<code>in</code>	<code>req</code>	Request HTTP
-----------------	------------------	--------------

Devuelve

`esp_err_t`

Valores devueltos

<i>ESP_OK</i>	Credenciales guardadas, conexión iniciada
<i>ESP_FAIL</i>	Error en parsing o memoria

Nota

El estado de conexión se consulta con GET /wifiConnectStatus

Definición en la línea 791 del archivo [http_server.c](#).

Hace referencia a [connect_to_wifi\(\)](#), [save_wifi_credentials\(\)](#), [TAG](#), [wifi_app_get_wifi_config\(\)](#) y [wifi_config](#).

Referenciado por [http_server_configure\(\)](#).

9.9.2.23. http_server_wifi_connect_status_json_handler()

```
esp_err_t http_server_wifi_connect_status_json_handler (
    httpd_req_t * req) [static]
```

Handler GET /wifiConnectStatus - Estado de conexión WiFi.

Retorna el estado actual de la conexión WiFi.

Response JSON:

```
{"wifi_connect_status": 0} // NONE
{"wifi_connect_status": 1} // CONNECTING
{"wifi_connect_status": 2} // SUCCESS
{"wifi_connect_status": 3} // FAIL
```

Parámetros

in	req	Request HTTP
----	-----	--------------

Devuelve

esp_err_t ESP_OK siempre

Definición en la línea 898 del archivo [http_server.c](#).

Hace referencia a [g_wifi_connect_status](#) y [TAG](#).

Referenciado por [http_server_configure\(\)](#).

9.9.2.24. log_error_if_nonzero()

```
void log_error_if_nonzero (
    const char * message,
    int error_code) [static]
```

Definición en la línea 374 del archivo [http_server.c](#).

Hace referencia a [TAG](#).

9.9.2.25. `sensors_init()`

```
void sensors_init (
    void ) [static]
```

Inicializa periféricos de sensores.

Configura:

- ADC1 para NTC (oneshot mode)
- NTC driver con ADC handle
- PIR driver (GPIO 35)
- `sensor_task` (tarea RTOS de lectura)

Nota

Llamada desde [http_server_start\(\)](#)

Ver también

[ntc_init\(\)](#) Inicializa driver NTC

[pir_init\(\)](#) Inicializa driver PIR

[sensor_task_init\(\)](#) Crea tarea de sensores

Definición en la línea [402](#) del archivo [http_server.c](#).

Hace referencia a [inicializar_ntc\(\)](#), [pir_init\(\)](#), [s_ntc_adc_handle](#), [sensor_task_init\(\)](#) y [TAG](#).

Referenciado por [http_server_start\(\)](#).

9.9.2.26. `toogle_led()`

```
void toogle_led (
    void )
```

Alterna el estado del LED de depuración.

Alternar LED de estado.

Toggle simple del GPIO definido como `BLINK_GPIO`. Usado para debug visual del sistema. < GPIO del LED de estado

Definición en la línea [201](#) del archivo [http_server.c](#).

Hace referencia a [BLINK_GPIO](#) y [s_led_state](#).

Referenciado por [compare_hour_day_structs\(\)](#) y [http_server_toogle_led_handler\(\)](#).

9.9.3. Documentación de variables

9.9.3.1. fw_update_reset

```
esp_timer_handle_t fw_update_reset
```

Handle del timer de reinicio OTA.

Definición en la línea 160 del archivo [http_server.c](#).

Referenciado por [http_server_fw_update_reset_timer\(\)](#).

9.9.3.2. fw_update_reset_args

```
const esp_timer_create_args_t fw_update_reset_args
```

Valor inicial:

```
= {  
    .callback = &http_server_fw_update_reset_callback,  
    .arg = NULL,  
    .dispatch_method = ESP_TIMER_TASK,  
    .name = "fw_update_reset"  
}
```

Configuración del timer para reinicio post-OTA.

Timer de 8 segundos que reinicia el ESP32 después de actualizar firmware. Callback: [http_server_fw_update_reset_callback\(\)](#)

Definición en la línea 152 del archivo [http_server.c](#).

Referenciado por [http_server_fw_update_reset_timer\(\)](#).

9.9.3.3. g_fw_update_status

```
int g_fw_update_status = 0 [static]
```

Estado de actualización OTA (PENDING, SUCCESSFUL, FAILED).

Actualización OTA pendiente

Definición en la línea 134 del archivo [http_server.c](#).

Referenciado por [http_server_fw_update_reset_timer\(\)](#), [http_server_monitor\(\)](#) y [http_server_OTA_status_handler\(\)](#).

9.9.3.4. g_wifi_connect_status

```
int g_wifi_connect_status = NONE [static]
```

Estado de conexión WiFi (NONE, CONNECTING, SUCCESS, FAIL).

Definición en la línea 131 del archivo [http_server.c](#).

Referenciado por [http_server_monitor\(\)](#) y [http_server_wifi_connect_status_json_handler\(\)](#).

9.9.3.5. `http_server_handle`

```
httpd_handle_t http_server_handle = NULL [static]
```

Handle del servidor HTTP.

Definición en la línea 137 del archivo `http_server.c`.

Referenciado por `http_server_configure()`, `http_server_start()` y `http_server_stop()`.

9.9.3.6. `http_server_monitor_queue_handle`

```
QueueHandle_t http_server_monitor_queue_handle [static]
```

Cola de mensajes para `http_server_monitor`.

Definición en la línea 143 del archivo `http_server.c`.

Referenciado por `http_server_configure()`, `http_server_monitor()` y `http_server_monitor_send_message()`.

9.9.3.7. `s_led_state`

```
uint8_t s_led_state = 0
```

Estado del LED de depuración.

Estado del LED RGB (externo).

Definición en la línea 192 del archivo `http_server.c`.

Referenciado por `toggle_led()`.

9.9.3.8. `s_ntc_adc_handle`

```
adc_oneshot_unit_handle_t s_ntc_adc_handle = NULL
```

Handle del ADC para sensor NTC (compartido globalmente).

Handle externo del ADC para lectura de temperatura.

Inicializado en `http_server_configure_peripherals()`. Usado por endpoint `/dhtSensor.json` para leer temperatura. También usado por `fan_control.c` (extern).

Definido en `http_server.c` e inicializado durante el arranque. Usado por `compute_auto_speed()` para leer temperatura del NTC. Si es NULL, modo AUTO retorna velocidad 0 %.

Definición en la línea 128 del archivo `http_server.c`.

Referenciado por `fan_control_set_mode()`, `fan_control_update()`, `http_server_get_dht_sensor_readings_json_handler()` y `sensors_init()`.

9.9.3.9. TAG

```
const char TAG[] = "http_server" [static]
```

Tag para logging ESP-IDF.

Definición en la línea 117 del archivo [http_server.c](#).

Referenciado por [api_delete_register\(\)](#), [api_post_register\(\)](#), [check_sta_connection_state\(\)](#), [connect_to_wifi\(\)](#), [http_server_app_css_handler\(\)](#), [http_server_app_js_handler\(\)](#), [http_server_configure\(\)](#), [http_server_fan_control_handler\(\)](#), [http_server_favicon_ico_handler\(\)](#), [http_server_fw_update_reset_callback\(\)](#), [http_server_fw_update_reset_timer\(\)](#), [http_server_get_dht_sensor_readings_json_handler\(\)](#), [http_server_index_html_handler\(\)](#), [http_server_jquery_handler\(\)](#), [http_server_monitor\(\)](#), [http_server_OTA_status_handler\(\)](#), [http_server_OTA_update_handler\(\)](#), [http_server_read_register_handler\(\)](#), [http_server_register_erase_handler\(\)](#), [http_server_start\(\)](#), [http_server_stop\(\)](#), [http_server_time_json_handler\(\)](#), [http_server_toogle_led_handler\(\)](#), [http_server_wifi_connect_json_handler\(\)](#), [http_server_wifi_connect_status_json_handler\(\)](#), [load_wifi_credentials\(\)](#), [log_error_if_nonzero\(\)](#), [obtain_time\(\)](#), [read_reg_data\(\)](#), [register_registers_endpoints\(\)](#), [registers_scheduler_task\(\)](#), [save_register_to_nvs\(\)](#), [sensor_task_get_data\(\)](#), [sensor_task_init\(\)](#), [sensor_update_task\(\)](#), [sensors_init\(\)](#), [task_compare_hour_to_execute_action\(\)](#), [update_register\(\)](#), [wifi_app_event_handler\(\)](#), [wifi_app_start\(\)](#) y [wifi_app_task\(\)](#).

9.9.3.10. task_http_server_monitor

```
TaskHandle_t task_http_server_monitor = NULL [static]
```

Handle de tarea monitorea de eventos HTTP/WiFi.

Definición en la línea 140 del archivo [http_server.c](#).

Referenciado por [http_server_configure\(\)](#) y [http_server_stop\(\)](#).

9.10. http_server.c

[Ir a la documentación de este archivo.](#)

```
00001
00054
00055 /*
00056  * http_server.c
00057  *
00058  * Created on: Oct 20, 2021
00059  * Author: kjagu
00060  */
00061
00062 #include "esp_http_server.h"
00063 #include "esp_log.h"
00064 #include "esp_ota_ops.h"
00065 #include "esp_timer.h"
00066 #include "sys/param.h"
00067 #include <stdlib.h>
00068
00069 #include "http_server.h"
00070 #include "tasks_common.h"
00071 #include "wifi_app.h"
00072 #include "cJSON.h"
00073 #include "driver/gpio.h"
00074
00075 #include <stdio.h>
00076 #include <stdint.h>
00077 #include <stddef.h>
00078 #include <string.h>
00079 #include <time.h>
00080
00081 #include "esp_wifi.h"
00082 #include "esp_system.h"
00083 #include "nvs_flash.h"
```



```

00084 #include "esp_event.h"
00085 #include "esp_netif.h"
00086
00087 #include "freertos/FreeRTOS.h"
00088 #include "freertos/task.h"
00089 #include "freertos/semphr.h"
00090 #include "freertos/queue.h"
00091
00092 #include "lwip/sockets.h"
00093 #include "lwip/dns.h"
00094 #include "lwip/netdb.h"
00095
00096 #if 0
00097 // MQTT client removed from this build (functions unused). Enable if needed.
00098 #include "mqtt_client.h"
00099 #endif
00100 #include "esp_sntp.h"
00101
00102 #include "ntc_driver.h"
00103 #include "pir_driver.h"
00104 #include "esp_adc/adc_oneshot.h"
00105
00106 #include "fan_control.h"
00107 #include "registers.h"
00108 #include "sensor_task.h"
00109
00110
00111
00112 static esp_err_t http_server_read_register_handler(httpd_req_t *req);
00113
00114 static esp_err_t http_server_register_erase_handler(httpd_req_t *req);
00115
00117 static const char TAG[] = "http_server";
00118
00128 adc_oneshot_unit_handle_t s_ntc_adc_handle = NULL;
00129
00131 static int g_wifi_connect_status = NONE;
00132
00134 static int g_fw_update_status = OTA_UPDATE_PENDING;
00135
00137 static httpd_handle_t http_server_handle = NULL;
00138
00140 static TaskHandle_t task_http_server_monitor = NULL;
00141
00143 static QueueHandle_t http_server_monitor_queue_handle;
00144
00152 const esp_timer_create_args_t fw_update_reset_args = {
00153     .callback = &http_server_fw_update_reset_callback,
00154     .arg = NULL,
00155     .dispatch_method = ESP_TIMER_TASK,
00156     .name = "fw_update_reset"
00157 };
00158
00160 esp_timer_handle_t fw_update_reset;
00161
00167
00169 extern const uint8_t jquery_3_3_1_min_js_start[] asm("_binary_jquery_3_3_1_min_js_start");
00171 extern const uint8_t jquery_3_3_1_min_js_end[] asm("_binary_jquery_3_3_1_min_js_end");
00173 extern const uint8_t index_html_start[] asm("_binary_index_html_start");
00175 extern const uint8_t index_html_end[] asm("_binary_index_html_end");
00177 extern const uint8_t app_css_start[] asm("_binary_app_css_start");
00179 extern const uint8_t app_css_end[] asm("_binary_app_css_end");
00181 extern const uint8_t app_js_start[] asm("_binary_app_js_start");
00183 extern const uint8_t app_js_end[] asm("_binary_app_js_end");
00185 extern const uint8_t favicon_ico_start[] asm("_binary_favicon_ico_start");
00187 extern const uint8_t favicon_ico_end[] asm("_binary_favicon_ico_end");
00188 // end of EmbeddedFiles
00190
00192 uint8_t s_led_state = 0;
00193
00201 void toggle_led(void)
00202 {
00203     s_led_state = !s_led_state;
00204     gpio_set_level(BLINK_GPIO, s_led_state);
00205 }
00206
00233 static esp_err_t http_server_fan_control_handler(httpd_req_t *req)
00234 {
00235     ESP_LOGI(TAG, "/fanControl.json requested");
00236
00237     int len = req->content_len;
00238     if (len <= 0) {
00239         ESP_LOGI(TAG, "fanControl: content_len <= 0");
00240         return ESP_FAIL;
00241     }
00242
00243     char *buf = malloc(len + 1);

```

```

00244     if (!buf) {
00245         ESP_LOGI(TAG, "fanControl: malloc failed");
00246         return ESP_FAIL;
00247     }
00248
00249     int ret = httpd_req_recv(req, buf, len);
00250     if (ret <= 0) {
00251         free(buf);
00252         ESP_LOGI(TAG, "fanControl: recv failed");
00253         return ESP_FAIL;
00254     }
00255     buf[len] = '\0';
00256
00257     cJSON *root = cJSON_Parse(buf);
00258     free(buf);
00259
00260     if (!root) {
00261         ESP_LOGI(TAG, "fanControl: invalid JSON");
00262         return ESP_FAIL;
00263     }
00264
00265     cJSON *mode_json = cJSON_GetObjectItem(root, "mode");
00266     cJSON *speed_json = cJSON_GetObjectItem(root, "speed");
00267
00268     if (!cJSON_IsString(mode_json) || !cJSON_IsNumber(speed_json)) {
00269         cJSON_Delete(root);
00270         ESP_LOGI(TAG, "fanControl: missing fields");
00271         return ESP_FAIL;
00272     }
00273
00274     const char *mode_str = mode_json->valuelstring;
00275     int speed = speed_json->valueint;
00276
00277     fan_mode_t mode = FAN_MODE_MANUAL;
00278     if (strcmp(mode_str, "auto") == 0) {
00279         mode = FAN_MODE_AUTO;
00280     } else if (strcmp(mode_str, "registros") == 0) {
00281         mode = FAN_MODE_REGISTERS;
00282     }
00283
00284     fan_control_set_mode(mode);
00285     fan_control_set_manual_speed((uint8_t) speed);
00286
00287     cJSON_Delete(root);
00288
00289     char resp[128];
00290     snprintf(resp, sizeof(resp),
00291              "{\n\"status\": \"ok\", \"mode\": \"%s\", \"speed\": %d\",
00292              mode_str, speed);
00293
00294     httpd_resp_set_type(req, "application/json");
00295     httpd_resp_send(req, resp, strlen(resp));
00296
00297     return ESP_OK;
00298 }
00299
00300 // -----
00301 // TIME (/time.json)
00302 // -----
00303
00329 static esp_err_t http_server_time_json_handler(httpd_req_t *req)
00330 {
00331     ESP_LOGI(TAG, "/time.json requested");
00332
00333     time_t now;
00334     struct tm timeinfo;
00335
00336     time(&now);
00337     localtime_r(&now, &timeinfo);
00338
00339     char date_str[16];
00340     char time_str[16];
00341     char timeJSON[128];
00342
00343     strftime(date_str, sizeof(date_str), "%Y-%m-%d", &timeinfo);
00344     strftime(time_str, sizeof(time_str), "%H:%M:%S", &timeinfo);
00345
00346     snprintf(timeJSON, sizeof(timeJSON),
00347              "{\n\"date\": \"%s\", \"time\": \"%s\",
00348              date_str, time_str);
00349
00350     httpd_resp_set_type(req, "application/json");
00351     httpd_resp_send(req, timeJSON, strlen(timeJSON));
00352
00353     return ESP_OK;
00354 }
00355

```

```

00356 // -----
00357 // TOGGLE LED (/toggle_led.json)
00358 // -----
00359 static esp_err_t http_server_toggle_led_handler(httpd_req_t *req)
00360 {
00361     ESP_LOGI(TAG, "/toggle_led.json requested");
00362     toggle_led();
00363     httpd_resp_set_hdr(req, "Connection", "close");
00364     httpd_resp_send(req, NULL, 0);
00365     return ESP_OK;
00366 }
00367
00371 // -----
00372 // LOG ERROR
00373 // -----
00374 static void log_error_if_nonzero(const char *message, int error_code)
00375 {
00376     if (error_code != 0) {
00377         ESP_LOGE(TAG, "Last error%s: 0x%x", message, error_code);
00378     }
00379 }
00380
00381 // MQTT support disabled in this build; re-enable if needed.
00382
00383 // -----
00384 // SENSORES: NTC + PIR
00385 // -----
00386
00402 static void sensors_init(void)
00403 {
00404     adc_oneshot_unit_init_cfg_t init_cfg = {
00405         .unit_id = ADC_UNIT_1,
00406         .ulp_mode = ADC_ULP_MODE_DISABLE,
00407     };
00408     ESP_ERROR_CHECK(adc_oneshot_new_unit(&init_cfg, &s_ntc_adc_handle));
00409     inicializar_ntc(s_ntc_adc_handle);
00410
00411     // Inicializar sensor_task para lectura de sensores
00412     esp_err_t err = sensor_task_init(s_ntc_adc_handle);
00413     if (err != ESP_OK) {
00414         ESP_LOGE(TAG, "Failed to initialize sensor_task:%d", err);
00415     }
00416
00417     // Inicializar sensor PIR en GPIO 35
00418     pir_init(GPIO_NUM_35, NULL);
00419     ESP_LOGI(TAG, "PIR sensor initialized on GPIO 35");
00420 }
00421
00423 // -----
00424 // MONITOR TASK
00425 // -----
00426
00446 static void http_server_monitor(void *parameter);
00447
00448 // -----
00449 // FW UPDATE TIMER
00450 // -----
00451 static void http_server_fw_update_reset_timer(void)
00452 {
00453     if (g_fw_update_status == OTA_UPDATE_SUCCESSFUL)
00454     {
00455         ESP_LOGI(TAG, "http_server_fw_update_reset_timer: FW updated successful, starting reset timer");
00456         ESP_ERROR_CHECK(esp_timer_create(&fw_update_reset_args, &fw_update_reset));
00457         ESP_ERROR_CHECK(esp_timer_start_once(fw_update_reset, 8000000));
00458     }
00459     else
00460     {
00461         ESP_LOGI(TAG, "http_server_fw_update_reset_timer: FW update unsuccessful");
00462     }
00463 }
00464
00465 // -----
00466 // MONITOR TASK
00467 // -----
00468 static void http_server_monitor(void *parameter)
00469 {
00470     http_server_queue_message_t msg;
00471
00472     for (;;)
00473     {
00474         if (xQueueReceive(http_server_monitor_queue_handle, &msg, portMAX_DELAY))
00475         {

```

```

00476         switch (msg.msgID)
00477         {
00478             case HTTP_MSG_WIFI_CONNECT_INIT:
00479                 ESP_LOGI(TAG, "HTTP_MSG_WIFI_CONNECT_INIT");
00480                 g_wifi_connect_status = HTTP_WIFI_STATUS_CONNECTING;
00481                 break;
00482
00483             case HTTP_MSG_WIFI_CONNECT_SUCCESS:
00484                 ESP_LOGI(TAG, "HTTP_MSG_WIFI_CONNECT_SUCCESS");
00485                 g_wifi_connect_status = HTTP_WIFI_STATUS_CONNECT_SUCCESS;
00486                 // mqtt_app_start();
00487                 break;
00488
00489             case HTTP_MSG_WIFI_CONNECT_FAIL:
00490                 ESP_LOGI(TAG, "HTTP_MSG_WIFI_CONNECT_FAIL");
00491                 g_wifi_connect_status = HTTP_WIFI_STATUS_CONNECT_FAILED;
00492                 break;
00493
00494             case HTTP_MSG_OTA_UPDATE_SUCCESSFUL:
00495                 ESP_LOGI(TAG, "HTTP_MSG_OTA_UPDATE_SUCCESSFUL");
00496                 g_fw_update_status = OTA_UPDATE_SUCCESSFUL;
00497                 http_server_fw_update_reset_timer();
00498                 break;
00499
00500             case HTTP_MSG_OTA_UPDATE_FAILED:
00501                 ESP_LOGI(TAG, "HTTP_MSG_OTA_UPDATE_FAILED");
00502                 g_fw_update_status = OTA_UPDATE_FAILED;
00503                 break;
00504
00505             default:
00506                 break;
00507         }
00508     }
00509 }
00510 }
00511
00512 // -----
00513 // STATIC FILE HANDLERS
00514 // -----
00515
00526 static esp_err_t http_server_jquery_handler(httpd_req_t *req)
00527 {
00528     ESP_LOGI(TAG, "jquery requested");
00529
00530     httpd_resp_set_type(req, "application/javascript");
00531     httpd_resp_send(req,
00532                     (const char *)jquery_3_3_1_min_js_start,
00533                     jquery_3_3_1_min_js_end - jquery_3_3_1_min_js_start);
00534     return ESP_OK;
00535 }
00536
00547 static esp_err_t http_server_index_html_handler(httpd_req_t *req)
00548 {
00549     ESP_LOGI(TAG, "index.html requested");
00550
00551     httpd_resp_set_type(req, "text/html");
00552     httpd_resp_send(req,
00553                     (const char *)index_html_start,
00554                     index_html_end - index_html_start);
00555     return ESP_OK;
00556 }
00557
00568 static esp_err_t http_server_app_css_handler(httpd_req_t *req)
00569 {
00570     ESP_LOGI(TAG, "app.css requested");
00571
00572     httpd_resp_set_type(req, "text/css");
00573     httpd_resp_send(req,
00574                     (const char *)app_css_start,
00575                     app_css_end - app_css_start);
00576     return ESP_OK;
00577 }
00578
00589 static esp_err_t http_server_app_js_handler(httpd_req_t *req)
00590 {
00591     ESP_LOGI(TAG, "app.js requested");
00592
00593     httpd_resp_set_type(req, "application/javascript");
00594     httpd_resp_send(req,
00595                     (const char *)app_js_start,
00596                     app_js_end - app_js_start);
00597     return ESP_OK;
00598 }
00599
00610 static esp_err_t http_server_favicon_ico_handler(httpd_req_t *req)
00611 {
00612     ESP_LOGI(TAG, "favicon.ico requested");

```

```

00613
00614     httpd_resp_set_type(req, "image/x-icon");
00615     httpd_resp_send(req,
00616                     (const char *)favicon_ico_start,
00617                     favicon_ico_end - favicon_ico_start);
00618     return ESP_OK;
00619 }
00620
00621 // -----
00622 // OTA HANDLERS
00623 // -----
00624
00655 esp_err_t http_server_OTA_update_handler(httpd_req_t *req)
00656 {
00657     esp_ota_handle_t ota_handle;
00658     char ota_buff[1024];
00659     int content_length = req->content_len;
00660     int content_received = 0;
00661     int rcv_len;
00662     bool is_req_body_started = false;
00663     bool flash_successful = false;
00664
00665     const esp_partition_t *update_partition = esp_ota_get_next_update_partition(NULL);
00666
00667     do
00668     {
00669         if ((rcv_len = httpd_req_recv(req, ota_buff, MIN(content_length, sizeof(ota_buff)))) < 0)
00670         {
00671             if (rcv_len == HTTPD SOCK_ERR_TIMEOUT)
00672             {
00673                 ESP_LOGI(TAG, "http_server_OTA_update_handler: Socket Timeout");
00674                 continue;
00675             }
00676             ESP_LOGI(TAG, "http_server_OTA_update_handler: OTA other Error%d", rcv_len);
00677             return ESP_FAIL;
00678         }
00679         printf("http_server_OTA_update_handler: OTA RX:%d of %d\r",
00680                content_received, content_length);
00681
00682         if (!is_req_body_started)
00683         {
00684             is_req_body_started = true;
00685
00686             char *body_start_p = strstr(ota_buff, "\r\n\r\n") + 4;
00687             int body_part_len = rcv_len - (body_start_p - ota_buff);
00688
00689             printf("http_server_OTA_update_handler: OTA file size:%d\r\n", content_length);
00690
00691             esp_err_t err = esp_ota_begin(update_partition, OTA_SIZE_UNKNOWN, &ota_handle);
00692             if (err != ESP_OK)
00693             {
00694                 printf("http_server_OTA_update_handler: Error with OTA begin, cancelling\r\n");
00695                 return ESP_FAIL;
00696             }
00697             else
00698             {
00699                 printf("http_server_OTA_update_handler: Writing to partition subtype%d at 0x%lx\r\n",
00700                        update_partition->subtype, update_partition->address);
00701             }
00702
00703             esp_ota_write(ota_handle, body_start_p, body_part_len);
00704             content_received += body_part_len;
00705         }
00706         else
00707         {
00708             esp_ota_write(ota_handle, ota_buff, rcv_len);
00709             content_received += rcv_len;
00710         }
00711     } while (rcv_len > 0 && content_received < content_length);
00712
00713     if (esp_ota_end(ota_handle) == ESP_OK)
00714     {
00715         if (esp_ota_set_boot_partition(update_partition) == ESP_OK)
00716         {
00717             const esp_partition_t *boot_partition = esp_ota_get_boot_partition();
00718             ESP_LOGI(TAG,
00719                     "http_server_OTA_update_handler: Next boot partition subtype%d at 0x%lx",
00720                     boot_partition->subtype, boot_partition->address);
00721             flash_successful = true;
00722         }
00723         else
00724         {
00725             ESP_LOGI(TAG, "http_server_OTA_update_handler: FLASHED ERROR!!!");
00726         }
00727     }
00728 }
00729 else

```

```

00730     {
00731         ESP_LOGI(TAG, "http_server_OTA_update_handler: esp_ota_end ERROR!!!");
00732     }
00733
00734     if (flash_successful) {
00735         http_server_monitor_send_message(HTTP_MSG_OTA_UPDATE_SUCCESSFUL);
00736     } else {
00737         http_server_monitor_send_message(HTTP_MSG_OTA_UPDATE_FAILED);
00738     }
00739
00740     return ESP_OK;
00741 }
00742
00743 esp_err_t http_server_OTA_status_handler(httpd_req_t *req)
00744 {
00745     char otaJSON[100];
00746
00747     ESP_LOGI(TAG, "OTAstatus requested");
00748
00749     sprintf(otaJSON,
00750         "{\"ota_update_status\":%d,\"compile_time\": \"%s\", \"compile_date\": \"%s\"}\",",
00751         g_fw_update_status, __TIME__, __DATE__);
00752
00753     httpd_resp_set_type(req, "application/json");
00754     httpd_resp_send(req, otaJSON, strlen(otaJSON));
00755
00756     return ESP_OK;
00757 }
00758
00759 // -----
00760 // WiFi CONNECT (/wifiConnect.json)
00761 // -----
00762
00791 static esp_err_t http_server_wifi_connect_json_handler(httpd_req_t *req)
00792 {
00793     size_t header_len;
00794     char *header_value;
00795     char *ssid_str = NULL;
00796     char *pass_str = NULL;
00797     int content_length;
00798
00799     ESP_LOGI(TAG, "/wifiConnect.json requested");
00800
00801     header_len = httpd_req_get_hdr_value_len(req, "Content-Length");
00802     if (header_len <= 0) {
00803         ESP_LOGI(TAG, "Content-Length header is missing or invalid");
00804         return ESP_FAIL;
00805     }
00806
00807     header_value = (char *)malloc(header_len + 1);
00808     if (httpd_req_get_hdr_value_str(req, "Content-Length", header_value, header_len + 1) != ESP_OK) {
00809         free(header_value);
00810         ESP_LOGI(TAG, "Failed to get Content-Length header value");
00811         return ESP_FAIL;
00812     }
00813
00814     content_length = atoi(header_value);
00815     free(header_value);
00816
00817     if (content_length <= 0) {
00818         ESP_LOGI(TAG, "Invalid Content-Length value");
00819         return ESP_FAIL;
00820     }
00821
00822     char *data_buffer = (char *)malloc(content_length + 1);
00823     if (!data_buffer) {
00824         ESP_LOGI(TAG, "malloc failed");
00825         return ESP_FAIL;
00826     }
00827
00828     if (httpd_req_recv(req, data_buffer, content_length) <= 0) {
00829         free(data_buffer);
00830         ESP_LOGI(TAG, "Failed to receive request body");
00831         return ESP_FAIL;
00832     }
00833
00834     data_buffer[content_length] = '\0';
00835
00836     cJSON *root = cJSON_Parse(data_buffer);
00837     free(data_buffer);
00838
00839     if (!root) {
00840         ESP_LOGI(TAG, "Invalid JSON data");
00841         return ESP_FAIL;
00842     }
00843
00844     cJSON *ssid_json = cJSON_GetObjectItem(root, "selectedSSID");

```

```

00845     cJSON *pwd_json = cJSON_GetObjectItem(root, "pwd");
00846
00847     if (!ssid_json || !pwd_json || !cJSON_IsString(ssid_json) || !cJSON_IsString(pwd_json)) {
00848         cJSON_Delete(root);
00849         ESP_LOGI(TAG, "Missing or invalid JSON data fields");
00850         return ESP_FAIL;
00851     }
00852
00853     ssid_str = strdup(ssid_json->valuelstring);
00854     pass_str = strdup(pwd_json->valuelstring);
00855
00856     cJSON_Delete(root);
00857
00858     ESP_LOGI(TAG, "Received SSID:%s", ssid_str);
00859     ESP_LOGI(TAG, "Received Password:%s", pass_str);
00860
00861     wifi_config_t *wifi_config = wifi_app_get_wifi_config();
00862     memset(wifi_config, 0x00, sizeof(wifi_config_t));
00863     memcpy(wifi_config->sta.ssid, ssid_str, strlen(ssid_str));
00864     memcpy(wifi_config->sta.password, pass_str, strlen(pass_str));
00865
00866     save_wifi_credentials(ssid_str, pass_str);
00867     esp_wifi_disconnect();
00868     connect_to_wifi();
00869
00870     free(ssid_str);
00871     free(pass_str);
00872
00873     return ESP_OK;
00874 }
00875
00876 // -----
00877 // WiFi STATUS (/wifiConnectStatus)
00878 // -----
00879 // WIFI CONNECT STATUS (/wifiConnectStatus)
00880 // -----
00898 static esp_err_t http_server_wifi_connect_status_json_handler(httpd_req_t *req)
00899 {
00900     ESP_LOGI(TAG, "/wifiConnectStatus requested");
00901
00902     char statusJSON[100];
00903     sprintf(statusJSON, "{\"wifi_connect_status\": %d}", g_wifi_connect_status);
00904
00905     httpd_resp_set_type(req, "application/json");
00906     httpd_resp_send(req, statusJSON, strlen(statusJSON));
00907
00908     return ESP_OK;
00909 }
00910
00911 // -----
00912 // DHT/NTC + PIR (/dhtSensor.json)
00913 // -----
00935 static esp_err_t http_server_get_dht_sensor_readings_json_handler(httpd_req_t *req)
00936 {
00937     ESP_LOGI(TAG, "dhtSensor.json requested (NTC + PIR)");
00938
00939     float temp_c = leer_temperatura_celsius(s_ntc_adc_handle);
00940     bool motion = pir_is_motion_active();
00941
00942     char json[128];
00943
00944     if (temp_c < -100.0f) {
00945         snprintf(json, sizeof(json),
00946             "{\"temp\": null, \"pir\": %d",
00947             motion ? 1 : 0);
00948     } else {
00949         snprintf(json, sizeof(json),
00950             "{\"temp\": %.2f, \"pir\": %d",
00951             temp_c,
00952             motion ? 1 : 0);
00953     }
00954
00955     httpd_resp_set_type(req, "application/json");
00956     httpd_resp_send(req, json, strlen(json));
00957
00958     return ESP_OK;
00959 }
00960
00961 // -----
00962 // /regerase.json (compatibilidad botón "Borrar registro")
00963 // Body JSON: { "selectedNumber": "3" }
00964 // -----
00991 static esp_err_t http_server_register_erase_handler(httpd_req_t *req)
00992 {
00993     ESP_LOGI(TAG, "/regerase.json requested");
00994

```

```

00995     int len = req->content_len;
00996     if (len <= 0 || len > 128) {
00997         httpd_resp_send_err(req, HTTPD_400_BAD_REQUEST, "invalid length");
00998         return ESP_FAIL;
00999     }
01000
01001     char *buf = malloc(len + 1);
01002     if (!buf) {
01003         httpd_resp_send_err(req, HTTPD_500_INTERNAL_SERVER_ERROR, "no mem");
01004         return ESP_FAIL;
01005     }
01006
01007     int ret = httpd_req_recv(req, buf, len);
01008     if (ret <= 0) {
01009         free(buf);
01010         httpd_resp_send_err(req, HTTPD_500_INTERNAL_SERVER_ERROR, "recv failed");
01011         return ESP_FAIL;
01012     }
01013     buf[ret] = '\0';
01014
01015     cJSON *root = cJSON_Parse(buf);
01016     free(buf);
01017     if (!root) {
01018         httpd_resp_send_err(req, HTTPD_400_BAD_REQUEST, "invalid json");
01019         return ESP_FAIL;
01020     }
01021
01022     cJSON *reg_number_json = cJSON_GetObjectItem(root, "selectedNumber");
01023     if (!cJSON_IsString(reg_number_json)) {
01024         cJSON_Delete(root);
01025         httpd_resp_send_err(req, HTTPD_400_BAD_REQUEST, "missing selectedNumber");
01026         return ESP_FAIL;
01027     }
01028
01029     int id = atoi(reg_number_json->valstring);
01030     cJSON_Delete(root);
01031
01032     if (id <= 0 || id > 10) {
01033         httpd_resp_send_err(req, HTTPD_400_BAD_REQUEST, "invalid id");
01034         return ESP_FAIL;
01035     }
01036
01037     delete_register_from_nvs(id);
01038     ESP_LOGI(TAG, "register%d deleted from NVS", id);
01039
01040     httpd_resp_set_type(req, "application/json");
01041     httpd_resp_send(req, "{\"status\":\"ok\"}", -1);
01042
01043     return ESP_OK;
01044 }
01045
01046
01047 // -----
01048 // HTTP SERVER CONFIG
01049 // -----
01050
01086 static httpd_handle_t http_server_configure(void)
01087 {
01088     httpd_config_t config = HTTPD_DEFAULT_CONFIG();
01089
01090     http_server_monitor_queue_handle =
01091         xQueueCreate(3, sizeof(http_server_queue_message_t));
01092
01093     xTaskCreatePinnedToCore(&http_server_monitor,
01094         "http_server_monitor",
01095         HTTP_SERVER_MONITOR_STACK_SIZE,
01096         NULL,
01097         HTTP_SERVER_MONITOR_PRIORITY,
01098         &task_http_server_monitor,
01099         HTTP_SERVER_MONITOR_CORE_ID);
01100
01101     config.core_id = HTTP_SERVER_TASK_CORE_ID;
01102     config.task_priority = HTTP_SERVER_TASK_PRIORITY;
01103     config.stack_size = HTTP_SERVER_TASK_STACK_SIZE;
01104     config.max_uri_handlers = 20;
01105     config.recv_wait_timeout = 10;
01106     config.send_wait_timeout = 10;
01107
01108     ESP_LOGI(TAG,
01109         "http_server_configure: Starting server on port: '%d' with priority: '%d'",
01110         config.server_port,
01111         config.task_priority);
01112
01113     if (httpd_start(&http_server_handle, &config) == ESP_OK)
01114     {
01115         ESP_LOGI(TAG, "http_server_configure: Registering URI handlers");
01116     }

```



```

01117     httpd_uri_t fan_control_uri = {
01118         .uri      = "/fanControl.json",
01119         .method    = HTTP_POST,
01120         .handler    = http_server_fan_control_handler,
01121         .user_ctx = NULL
01122     };
01123     httpd_register_uri_handler(http_server_handle, &fan_control_uri);
01124
01125     httpd_uri_t jquery_js = {
01126         .uri      = "/jquery-3.3.1.min.js",
01127         .method    = HTTP_GET,
01128         .handler    = http_server_jquery_handler,
01129         .user_ctx = NULL
01130     };
01131     httpd_register_uri_handler(http_server_handle, &jquery_js);
01132
01133     httpd_uri_t index_html = {
01134         .uri      = "/",
01135         .method    = HTTP_GET,
01136         .handler    = http_server_index_html_handler,
01137         .user_ctx = NULL
01138     };
01139     httpd_register_uri_handler(http_server_handle, &index_html);
01140
01141     httpd_uri_t app_css = {
01142         .uri      = "/app.css",
01143         .method    = HTTP_GET,
01144         .handler    = http_server_app_css_handler,
01145         .user_ctx = NULL
01146     };
01147     httpd_register_uri_handler(http_server_handle, &app_css);
01148
01149     httpd_uri_t app_js = {
01150         .uri      = "/app.js",
01151         .method    = HTTP_GET,
01152         .handler    = http_server_app_js_handler,
01153         .user_ctx = NULL
01154     };
01155     httpd_register_uri_handler(http_server_handle, &app_js);
01156
01157     httpd_uri_t favicon_ico = {
01158         .uri      = "/favicon.ico",
01159         .method    = HTTP_GET,
01160         .handler    = http_server_favicon_ico_handler,
01161         .user_ctx = NULL
01162     };
01163     httpd_register_uri_handler(http_server_handle, &favicon_ico);
01164
01165     httpd_uri_t toogle_led_uri = {
01166         .uri      = "/toogle_led.json",
01167         .method    = HTTP_POST,
01168         .handler    = http_server_toogle_led_handler,
01169         .user_ctx = NULL
01170     };
01171     httpd_register_uri_handler(http_server_handle, &toogle_led_uri);
01172
01173     httpd_uri_t wifi_connect_json = {
01174         .uri      = "/wifiConnect.json",
01175         .method    = HTTP_POST,
01176         .handler    = http_server_wifi_connect_json_handler,
01177         .user_ctx = NULL
01178     };
01179     httpd_register_uri_handler(http_server_handle, &wifi_connect_json);
01180
01181     httpd_uri_t readreg_uri = {
01182         .uri      = "/readreg.json",
01183         .method    = HTTP_GET,
01184         .handler    = http_server_read_register_handler,
01185         .user_ctx = NULL
01186     };
01187     httpd_register_uri_handler(http_server_handle, &readreg_uri);
01188     ESP_LOGI(TAG, "Registered /readreg.json handler");
01189
01190     // Borrar registro (compatibilidad)
01191     httpd_uri_t register_erase_uri = {
01192         .uri      = "/regerase.json",
01193         .method    = HTTP_POST,
01194         .handler    = http_server_register_erase_handler,
01195         .user_ctx = NULL
01196     };
01197
01198     httpd_register_uri_handler(http_server_handle, &register_erase_uri);
01199
01200     httpd_uri_t OTA_update = {
01201         .uri      = "/OTAupdate",
01202         .method    = HTTP_POST,
01203         .handler    = http_server_OTA_update_handler,

```

```

01204         .user_ctx = NULL
01205     };
01206     httpd_register_uri_handler(http_server_handle, &OTA_update);
01207
01208     httpd_uri_t OTA_status = {
01209         .uri      = "/OTAstatus",
01210         .method   = HTTP_POST,
01211         .handler  = http_server_OTA_status_handler,
01212         .user_ctx = NULL
01213     };
01214     httpd_register_uri_handler(http_server_handle, &OTA_status);
01215
01216     httpd_uri_t dht_sensor_json = {
01217         .uri      = "/dhtSensor.json",
01218         .method   = HTTP_GET,
01219         .handler  = http_server_get_dht_sensor_readings_json_handler,
01220         .user_ctx = NULL
01221     };
01222     httpd_register_uri_handler(http_server_handle, &dht_sensor_json);
01223
01224     httpd_uri_t wifi_connect_status_json = {
01225         .uri      = "/wifiConnectStatus",
01226         .method   = HTTP_POST,
01227         .handler  = http_server_wifi_connect_status_json_handler,
01228         .user_ctx = NULL
01229     };
01230     httpd_register_uri_handler(http_server_handle, &wifi_connect_status_json);
01231
01232     // Endpoints nuevos de registros (en registers.c):
01233     register_registers_endpoints(http_server_handle);
01234
01235     httpd_uri_t time_json = {
01236         .uri      = "/time.json",
01237         .method   = HTTP_GET,
01238         .handler  = http_server_time_json_handler,
01239         .user_ctx = NULL
01240     };
01241     httpd_register_uri_handler(http_server_handle, &time_json);
01242
01243
01244
01245
01246     return http_server_handle;
01247 }
01248
01249 return NULL;
01250 }
01251
01252 // -----
01253 // /readreg.json (compatibilidad con front-end viejo)
01254 // Devuelve regi..reg10 como strings "HHMM" + días ("L","M","X"... )
01255 // -----
01256 static esp_err_t http_server_read_register_handler(httpd_req_t *req)
01257 {
01258     ESP_LOGI(TAG, "/readreg.json requested -> leyendo desde NVS");
01259
01260     cJSON *root = cJSON_CreateObject();
01261     if (!root) {
01262         httpd_resp_send_err(req, HTTPD_500_INTERNAL_SERVER_ERROR, "no mem");
01263         return ESP_FAIL;
01264     }
01265
01266     for (int i = 1; i <= 10; i++) {
01267         char key[16];
01268         snprintf(key, sizeof(key), "reg%d", i);
01269
01270         char buf[256];
01271         esp_err_t err = load_register_from_nvs(i, buf, sizeof(buf));
01272         if (err != ESP_OK) {
01273             // No hay registro: manda null
01274             cJSON_AddNullToObject(root, key);
01275             continue;
01276         }
01277
01278         cJSON *reg_json = cJSON_Parse(buf);
01279         if (!reg_json) {
01280             cJSON_AddNullToObject(root, key);
01281             continue;
01282         }
01283
01284         cJSON *hour_json = cJSON_GetObjectItem(reg_json, "hour");
01285         cJSON *minute_json = cJSON_GetObjectItem(reg_json, "minute");
01286         cJSON *days_json = cJSON_GetObjectItem(reg_json, "days");
01287
01288         if (!cJSON_IsNumber(hour_json) ||
01289             !cJSON_IsNumber(minute_json) ||
01290             !cJSON_IsArray(days_json)) {

```

```

01291         cJSON_Delete(reg_json);
01292         cJSON_AddNullToObject(root, key);
01293         continue;
01294     }
01295
01296     char reg_str[32];
01297     // "HHMM"
01298     snprintf(reg_str, sizeof(reg_str), "%02d%02d",
01299             hour_json->valueint, minute_json->valueint);
01300
01301     // Añade letras de días tal como las guardamos en NVS: "L","M","X","J","V","S","D"
01302     int ndays = cJSON_GetArraySize(days_json);
01303     for (int j = 0; j < ndays && strlen(reg_str) < sizeof(reg_str) - 1; j++) {
01304         cJSON *d = cJSON_GetArrayItem(days_json, j);
01305         if (cJSON_IsString(d) && d->valuelstring && d->valuelstring[0] != '\0') {
01306             strncat(reg_str, d->valuelstring, 1); // solo la primera letra
01307         }
01308     }
01309
01310     cJSON_AddStringToObject(root, key, reg_str);
01311     cJSON_Delete(reg_json);
01312 }
01313
01314 char *out = cJSON_PrintUnformatted(root);
01315 httpd_resp_set_type(req, "application/json");
01316 httpd_resp_send(req, out, strlen(out));
01317
01318 free(out);
01319 cJSON_Delete(root);
01320
01321 return ESP_OK;
01322 }
01323
01324 // -----
01325 // FAN TASK - Actualización automática del ventilador
01326 // -----
01327
01345 static void fan_task(void *arg)
01346 {
01347     while (1) {
01348         fan_control_update();
01349         vTaskDelay(pdMS_TO_TICKS(1000)); // actualiza cada 1s
01350     }
01351 }
01352
01353 // -----
01354 // PUBLIC START/STOP
01355 // -----
01356
01375 void http_server_start(void)
01376 {
01377     if (http_server_handle == NULL)
01378     {
01379         sensors_init();
01380         http_server_handle = http_server_configure();
01381
01382         // Inicializar control del ventilador
01383         fan_control_init();
01384         ESP_LOGI(TAG, "Fan control initialized");
01385
01386         // Crear tarea para actualización automática del ventilador
01387         xTaskCreate(fan_task, "fan_task", 3072, NULL, 5, NULL);
01388         ESP_LOGI(TAG, "Fan task created");
01389     }
01390 }
01391
01404 void http_server_stop(void)
01405 {
01406     if (http_server_handle)
01407     {
01408         httpd_stop(http_server_handle);
01409         ESP_LOGI(TAG, "http_server_stop: stopping HTTP server");
01410         http_server_handle = NULL;
01411     }
01412     if (task_http_server_monitor)
01413     {
01414         vTaskDelete(task_http_server_monitor);
01415         ESP_LOGI(TAG, "http_server_stop: stopping HTTP server monitor");
01416         task_http_server_monitor = NULL;
01417     }
01418 }
01419
01420 // -----
01421 // QUEUE MESSAGE
01422 // -----
01423 BaseType_t http_server_monitor_send_message(http_server_message_e msgID)
01424 {

```

```

01425     http_server_queue_message_t msg;
01426     msg.msgID = msgID;
01427     return xQueueSend(http_server_monitor_queue_handle, &msg, portMAX_DELAY);
01428 }
01429
01430 void http_server_fw_update_reset_callback(void *arg)
01431 {
01432     ESP_LOGI(TAG, "http_server_fw_update_reset_callback: Timer timed-out, restarting device");
01433     esp_restart();
01434 }

```

9.11. Referencia del archivo http_server.h

Servidor HTTP con endpoints REST para control del sistema.

```

#include "freertos/FreeRTOS.h"
#include "esp_event.h"
#include "esp_adc/adc_oneshot.h"

```

Estructuras de datos

- struct [http_server_queue_message](#)

defines

- #define [OTA_UPDATE_PENDING](#) 0
Actualización OTA pendiente.
- #define [OTA_UPDATE_SUCCESSFUL](#) 1
Actualización OTA exitosa.
- #define [OTA_UPDATE_FAILED](#) -1
Actualización OTA fallida.
- #define [BLINK_GPIO](#) 2
GPIO del LED de estado.

typedefs

- typedef enum [http_server_wifi_connect_status](#) [http_server_wifi_connect_status_e](#)
- typedef enum [http_server_message](#) [http_server_message_e](#)
- typedef struct [http_server_queue_message](#) [http_server_queue_message_t](#)

Enumeraciones

- enum [http_server_wifi_connect_status](#) { [NONE](#) = 0, [HTTP_WIFI_STATUS_CONNECTING](#), [HTTP_WIFI_STATUS_CONNECTED](#), [HTTP_WIFI_STATUS_CONNECT_SUCCESS](#) }
- enum [http_server_message](#) { [HTTP_MSG_WIFI_CONNECT_INIT](#) = 0, [HTTP_MSG_WIFI_CONNECT_SUCCESS](#), [HTTP_MSG_WIFI_CONNECT_FAIL](#), [HTTP_MSG_OTA_UPDATE_SUCCESSFUL](#), [HTTP_MSG_OTA_UPDATE_FAILED](#) }

Funciones

- BaseType_t [http_server_monitor_send_message](#) ([http_server_message_e](#) msgID)
Enviar mensaje a la cola del monitor HTTP.
- void [http_server_start](#) (void)
Iniciar servidor HTTP y tareas relacionadas.
- void [http_server_stop](#) (void)
Detener servidor HTTP y liberar recursos.
- void [http_server_fw_update_reset_callback](#) (void *arg)
Callback de timer para reinicio tras OTA exitoso.
- void [toggle_led](#) (void)
Alternar LED de estado.

9.11.1. Descripción detallada

Servidor HTTP con endpoints REST para control del sistema.

Autor

Jair Hernan Telpis Cuaran, Luis Fernando Gamba Bedoya

Fecha

2025

Versión

1.0.0

Servidor HTTP embebido que proporciona:

- **Interfaz Web Completa:** HTML/CSS/JS para control del sistema
- **Endpoints REST:** Control de ventilador, registros, sensores
- **OTA Updates:** Actualización de firmware vía HTTP
- **Monitoreo de Estado:** Cola de mensajes para eventos WiFi y sistema
- **Archivos Embebidos:** [index.html](#), [app.js](#), app.css, jQuery, favicon

Principales Endpoints:

- GET / → Página web principal
- GET /dhtSensor.json → Leer temperatura y PIR
- POST /fanSettings.json → Configurar ventilador
- POST /regchange.json → Modificar registro programado
- GET /api/register → API de registros (ver [registers.h](#))
- POST /OTAupdate → Actualización OTA de firmware

El servidor se inicializa automáticamente cuando WiFi se conecta y mantiene una tarea monitor que procesa mensajes de estado del sistema.

Universidad Nacional de Colombia - Curso RTOS

Definición en el archivo [http_server.h](#).

9.11.2. Documentación de «define»

9.11.2.1. BLINK_GPIO

```
#define BLINK_GPIO 2
```

GPIO del LED de estado.

Definición en la línea 43 del archivo [http_server.h](#).

Referenciado por [toggle_led\(\)](#).

9.11.3. Documentación de «typedef»

9.11.3.1. http_server_message_e

```
typedef enum http_server_message http_server_message_e
```

9.11.3.2. http_server_queue_message_t

```
typedef struct http_server_queue_message http_server_queue_message_t
```

9.11.3.3. http_server_wifi_connect_status_e

```
typedef enum http_server_wifi_connect_status http_server_wifi_connect_status_e
```

9.11.4. Documentación de enumeraciones

9.11.4.1. http_server_message

```
enum http_server_message
```

Valores de enumeraciones

HTTP_MSG_WIFI_CONNECT_INIT	Iniciar conexión WiFi.
HTTP_MSG_WIFI_CONNECT_SUCCESS	WiFi conectado exitosamente.
HTTP_MSG_WIFI_CONNECT_FAIL	Error al conectar WiFi.
HTTP_MSG_OTA_UPDATE_SUCCESSFUL	Actualización OTA completada.
HTTP_MSG_OTA_UPDATE_FAILED	Actualización OTA falló

Definición en la línea 65 del archivo [http_server.h](#).

9.11.4.2. http_server_wifi_connect_status

```
enum http_server_wifi_connect_status
```

Valores de enumeraciones

NONE	Estado inicial, sin conexión.
HTTP_WIFI_STATUS_CONNECTING	Intentando conectar a WiFi.
HTTP_WIFI_STATUS_CONNECT_FAILED	Conexión WiFi falló
HTTP_WIFI_STATUS_CONNECT_SUCCESS	Conexión WiFi exitosa.

Definición en la línea 49 del archivo `http_server.h`.

9.11.5. Documentación de funciones

9.11.5.1. `http_server_fw_update_reset_callback()`

```
void http_server_fw_update_reset_callback (
    void * arg)
```

Callback de timer para reinicio tras OTA exitoso.

Se ejecuta 8 segundos después de una actualización OTA exitosa, dando tiempo a que el navegador reciba la respuesta HTTP antes de reiniciar.

Parámetros

<i>arg</i>	Argumento del timer (sin usar)
------------	--------------------------------

Nota

Llama a `esp_restart()` para aplicar nuevo firmware

Definición en la línea 1430 del archivo `http_server.c`.

Hace referencia a [TAG](#).

9.11.5.2. `http_server_monitor_send_message()`

```
BaseType_t http_server_monitor_send_message (
    http_server_message_e msgID)
```

Enviar mensaje a la cola del monitor HTTP.

Permite comunicación thread-safe entre módulos del sistema y el servidor HTTP. Los mensajes se procesan en orden FIFO por la tarea `http_server_monitor`.

Parámetros

<i>msgID</i>	ID del mensaje desde http_server_message_e
--------------	--

Devuelve

pdTRUE si el mensaje se envió correctamente
pdFALSE si la cola está llena (espera indefinida activada)

Nota

Esta función usa portMAX_DELAY, bloqueará hasta que haya espacio en la cola

Definición en la línea 1423 del archivo [http_server.c](#).

Hace referencia a [http_server_monitor_queue_handle](#).

Referenciado por [http_server_OTA_update_handler\(\)](#) y [wifi_app_task\(\)](#).

9.11.5.3. http_server_start()

```
void http_server_start (
    void )
```

Iniciar servidor HTTP y tareas relacionadas.

Crea el servidor HTTP en el puerto 80, registra todos los handlers de endpoints REST, inicializa sensores (NTC, PIR) y crea las tareas:

- `http_server_monitor`: Procesa mensajes de estado
- `fan_task`: Actualiza control del ventilador cada 1s

Nota

Solo se ejecuta si el servidor no está ya iniciado
Debe llamarse después de que WiFi esté configurado

Ver también

[http_server_stop\(\)](#) para detener el servidor

Iniciar servidor HTTP y tareas relacionadas.

Secuencia de inicialización completa:

1. Configura periféricos (ADC, NTC, PIR, sensor_task)
2. Inicializa control del ventilador
3. Crea cola de mensajes `http_server_monitor`
4. Crea tarea `http_server_monitor`
5. Configura y arranca servidor HTTP
6. Crea `fan_task` para modo AUTO

Nota

Llamar después de que WiFi obtenga IP

Atención

Debe llamarse solo una vez

Ver también

[http_server_configure\(\)](#) Configuración del servidor

[fan_task\(\)](#) Tarea de actualización automática

Definición en la línea 1375 del archivo [http_server.c](#).

Hace referencia a [fan_control_init\(\)](#), [fan_task\(\)](#), [http_server_configure\(\)](#), [http_server_handle](#), [sensors_init\(\)](#) y [TAG](#).

Referenciado por [wifi_app_task\(\)](#).

9.11.5.4. http_server_stop()

```
void http_server_stop (
    void )
```

Detener servidor HTTP y liberar recursos.

Detiene el servidor HTTP y elimina la tarea monitor asociada. Usado típicamente al cambiar configuración WiFi o antes de reiniciar.

Detener servidor HTTP y liberar recursos.

Secuencia de parada:

1. Detiene servidor HTTP con `httpd_stop()`
2. Elimina tarea `http_server_monitor`
3. Libera cola `http_server_monitor_queue_handle`

Nota

Solo ejecuta si `http_server_handle != NULL`

Atención

No detiene `fan_task` ni `sensor_task`

Definición en la línea 1404 del archivo [http_server.c](#).

Hace referencia a [http_server_handle](#), [TAG](#) y [task_http_server_monitor](#).

9.11.5.5. `toogle_led()`

```
void toogle_led (
    void )
```

Alternar LED de estado.

Cambia el estado del LED en `BLINK_GPIO`. Usado para indicadores visuales durante operaciones del sistema.

Alternar LED de estado.

Toggle simple del GPIO definido como `BLINK_GPIO`. Usado para debug visual del sistema. < GPIO del LED de estado

Definición en la línea 201 del archivo [http_server.c](#).

Hace referencia a [BLINK_GPIO](#) y [s_led_state](#).

Referenciado por [compare_hour_day_structs\(\)](#) y [http_server_toogle_led_handler\(\)](#).

9.12. `http_server.h`

[Ir a la documentación de este archivo.](#)

```
00001
00028
00029 #ifndef MAIN_HTTP_SERVER_H_
00030 #define MAIN_HTTP_SERVER_H_
00031 #include "freertos/FreeRTOS.h"
00032 #include "esp_event.h"
00033 #include "esp_adc/adc_oneshot.h"
00034
00038 #define OTA_UPDATE_PENDING      0
00039 #define OTA_UPDATE_SUCCESSFUL   1
00040 #define OTA_UPDATE_FAILED       -1
00042
00043 #define BLINK_GPIO              2
00044
00049 typedef enum http_server_wifi_connect_status
00050 {
00051     NONE = 0,
00052     HTTP_WIFI_STATUS_CONNECTING,
00053     HTTP_WIFI_STATUS_CONNECT_FAILED,
00054     HTTP_WIFI_STATUS_CONNECT_SUCCESS,
00055 } http_server_wifi_connect_status_e;
00056
00065 typedef enum http_server_message
00066 {
00067     HTTP_MSG_WIFI_CONNECT_INIT = 0,
00068     HTTP_MSG_WIFI_CONNECT_SUCCESS,
00069     HTTP_MSG_WIFI_CONNECT_FAIL,
00070     HTTP_MSG_OTA_UPDATE_SUCCESSFUL,
00071     HTTP_MSG_OTA_UPDATE_FAILED,
00072 } http_server_message_e;
00073
00078 typedef struct http_server_queue_message
00079 {
00080     http_server_message_e msgID;
00081 } http_server_queue_message_t;
00082
00095 BaseType_t http_server_monitor_send_message(http_server_message_e msgID);
00096
00110 void http_server_start(void);
00111
00118 void http_server_stop(void);
00119
00130 void http_server_fw_update_reset_callback(void *arg);
00131
00138 void toogle_led(void);
00139
00140 #endif /* MAIN_HTTP_SERVER_H_ */
```

9.13. Referencia del archivo ntc_driver.c

Implementación del driver para sensor NTC con conversión Steinhart-Hart.

```
#include "ntc_driver.h"
#include <math.h>
```

defines

- #define **CANAL_ADC_NTC** ADC_CHANNEL_6
Canal del ADC donde está conectado el NTC (GPIO 34).
- #define **RESISTENCIA_DIVISOR** 10000.0f
Resistencia fija del divisor de voltaje (10k).
- #define **RESISTENCIA_NOMINAL_NTC** 10000.0f
Resistencia nominal del NTC a temperatura de referencia (10k @ 25 °C).
- #define **TEMP_NOMINAL_NTC** 25.0f
Temperatura nominal de calibración (25 °C).
- #define **COEFICIENTE_BETA_NTC** 3950.0f
Coeficiente Beta del NTC (3950K).
- #define **VOLTAJE_REFERENCIA_MV** 3300.0f
Voltaje de referencia del ADC (3.3V = 3300mV).

Funciones

- void **inicializar_ntc** (adc_oneShot_unit_handle_t manejador_adc)
Configura el canal ADC para el termistor NTC.
- float **leer_temperatura_celsius** (adc_oneShot_unit_handle_t manejador_adc)
Realiza lectura ADC y convierte a temperatura usando Steinhart-Hart.

9.13.1. Descripción detallada

Implementación del driver para sensor NTC con conversión Steinhart-Hart.

Autor

Jair Hernan Telpis Cuaran, Luis Fernando Gamba Bedoya

Fecha

2025

Implementa la lectura del termistor NTC mediante ADC y conversión a temperatura usando la ecuación simplificada de Steinhart-Hart con coeficiente Beta.

Configuración de hardware:

- NTC 10K @ 25 °C conectado a GPIO 34

- Divisor de voltaje: $R_{div} = 10K$
- $VCC = 3.3V$
- ADC: 12 bits, atenuación 12dB

Circuito equivalente:

```
VCC (3.3V)
|
[R_div = 10K]
|
+-----> GPIO 34 (ADC1_CH6)
|
[R_ntc = f(T)]
|
GND
```

Definición en el archivo [ntc_driver.c](#).

9.13.2. Documentación de funciones

9.13.2.1. inicializar_ntc()

```
void inicializar_ntc (
    adc_oneshot_unit_handle_t manejador_adc)
```

Configura el canal ADC para el termistor NTC.

Inicializa el canal ADC para el sensor NTC.

Establece la configuración del canal ADC con:

- Atenuación 12dB: rango completo 0-3.3V
- Resolución por defecto: 12 bits (0-4095)

La atenuación de 12dB es necesaria porque el divisor de voltaje produce voltajes en todo el rango 0-3.3V dependiendo de la temperatura.

Parámetros

in	<i>manejador_adc</i>	Handle del ADC previamente inicializado
----	----------------------	---

Definición en la línea 87 del archivo [ntc_driver.c](#).

Hace referencia a [CANAL_ADC_NTC](#).

Referenciado por [sensors_init\(\)](#).

9.13.2.2. leer_temperatura_celsius()

```
float leer_temperatura_celsius (
    adc_one_shot_unit_handle_t manejador_adc)
```

Realiza lectura ADC y convierte a temperatura usando Steinhart-Hart.

Lee la temperatura actual del sensor NTC en grados Celsius.

Algoritmo de conversión:

1. **Lectura ADC:** Obtiene valor digital (0-4095)
2. **Conversión a voltaje:** $V = \text{ADC_raw} * (3300\text{mV} / 4095)$
3. **Cálculo de resistencia NTC:**

$$R_{\text{ntc}} = R_{\text{div}} * V / (V_{\text{ref}} - V)$$
4. **Ecuación Steinhart-Hart simplificada:**

$$1/T = 1/T_0 + (1/B) * \ln(R_{\text{ntc}} / R_0)$$
5. **Conversión K → °C:** $T_{\text{celsius}} = T_{\text{kelvin}} - 273.15$

Detección de errores:

- V V_{ref} : Circuito abierto (NTC desconectado)
- R_{ntc} 0: Cortocircuito o error matemático

Parámetros

in	manejador_adc	Handle del ADC configurado
----	---------------	----------------------------

Devuelve

Temperatura en °C o -999.0f en caso de error

Definición en la línea 122 del archivo `ntc_driver.c`.

Hace referencia a `CANAL_ADC_NTC`, `COEFICIENTE_BETA_NTC`, `RESISTENCIA_DIVISOR`, `RESISTENCIA_NOMINAL_NTC`, `TEMP_NOMINAL_NTC` y `VOLTAJE_REFERENCIA_MV`.

Referenciado por `fan_control_set_mode()`, `fan_control_update()`, `http_server_get_dht_sensor_readings_json_handler()` y `sensor_update_task()`.

9.14. ntc_driver.c

[Ir a la documentación de este archivo.](#)

```

00001
00030
00031 #include "ntc_driver.h"
00032
00033
00034 #include <math.h> // Librería matemática para cálculos como el logaritmo.
00035
00041
00043 #define CANAL_ADC_NTC          ADC_CHANNEL_6
00044
00046 #define RESISTENCIA_DIVISOR    10000.0f
00047
00049 #define RESISTENCIA_NOMINAL_NTC 10000.0f
00050
00052 #define TEMP_NOMINAL_NTC      25.0f
00053
00066 #define COEFICIENTE_BETA_NTC   3950.0f
00067
00069 #define VOLTAJE_REFERENCIA_MV  3300.0f
00070 // end of NTC_Config
00072
00087 void inicializar_ntc(adc_oneshot_unit_handle_t manejador_adc) {
00088     // Estructura de configuración del canal ADC
00089     adc_oneshot_chan_cfg_t config = {
00090         .atten = ADC_ATTEN_DB_12,          // Atenuación 12dB: rango 0-3.3V
00091         .bitwidth = ADC_BITWIDTH_DEFAULT,  // Resolución 12 bits (4096 niveles)
00092     };
00093     // Aplica la configuración al canal del NTC
00094     adc_oneshot_config_channel(manejador_adc, CANAL_ADC_NTC, &config);
00095 }
00096
00122 float leer_temperatura_celsius(adc_oneshot_unit_handle_t manejador_adc) {
00123     int valor_adc_raw;
00124
00125     // PASO 1: Leer valor digital del ADC (0 a 4095)
00126     adc_oneshot_read(manejador_adc, CANAL_ADC_NTC, &valor_adc_raw);
00127
00128     // PASO 2: Convertir valor digital a voltaje analógico
00129     // Fórmula: V = (ADC / ADC_max) * V_ref
00130     float voltaje_mv = (valor_adc_raw * VOLTAJE_REFERENCIA_MV) / 4095.0f;
00131
00132     // Validación: Detectar circuito abierto (NTC desconectado)
00133     // Si V < V_ref, no hay caída de voltaje en R_div → circuito abierto
00134     if (voltaje_mv >= VOLTAJE_REFERENCIA_MV) {
00135         return -999.0f; // Código de error: circuito abierto
00136     }
00137
00138     // PASO 3: Calcular resistencia del NTC usando divisor de voltaje
00139     // Del divisor de voltaje: V_out = V_in * (R_ntc / (R_div + R_ntc))
00140     // Despejando R_ntc: R_ntc = R_div * V_out / (V_in - V_out)
00141     float resistencia_ntc = RESISTENCIA_DIVISOR * voltaje_mv / (VOLTAJE_REFERENCIA_MV - voltaje_mv);
00142
00143     // Validación: Detectar cortocircuito o error de cálculo
00144     if (resistencia_ntc <= 0) {
00145         return -999.0f; // Código de error: resistencia inválida
00146     }
00147
00148     // PASO 4: Aplicar ecuación de Steinhart-Hart simplificada con Beta
00149     // Fórmula: 1/T = 1/T0 + (1/B) * ln(R/R0)
00150     // Donde: T y T0 en Kelvin, B en Kelvin, R y R0 en Ohms
00151     float steinhart = log(resistencia_ntc / RESISTENCIA_NOMINAL_NTC) / COEFICIENTE_BETA_NTC;
00152     steinhart += 1.0 / (TEMP_NOMINAL_NTC + 273.15); // Suma 1/T0 (T0 en Kelvin)
00153
00154     // PASO 5: Convertir de Kelvin a Celsius
00155     // T_celsius = T_kelvin - 273.15
00156     // Nota: steinhart contiene 1/T, por eso se invierte primero
00157     return (1.0 / steinhart) - 273.15;
00158 }

```

9.15. Referencia del archivo ntc_driver.h

Driver para sensor de temperatura NTC (Negative Temperature Coefficient).

```
#include "esp_adc/adc_oneshot.h"
```

Funciones

- void [inicializar_ntc](#) (adc_one_shot_unit_handle_t manejador_adc)
Inicializa el canal ADC para el sensor NTC.
- float [leer_temperatura_celsius](#) (adc_one_shot_unit_handle_t manejador_adc)
Lee la temperatura actual del sensor NTC en grados Celsius.

9.15.1. Descripción detallada

Driver para sensor de temperatura NTC (Negative Temperature Coefficient).

Autor

Jair Hernan Telpis Cuaran, Luis Fernando Gamba Bedoya

Fecha

2025

Versión

1.0

Este módulo implementa el driver para leer temperatura mediante un termistor NTC 10K conectado en configuración divisor de voltaje con el ADC del ESP32.

Hardware:

- Termistor NTC 10K @ 25°C (Beta = 3950K)
- Conectado a GPIO 34 (ADC1 Canal 6)
- Resistencia divisor: 10K
- Rango de medición: -40°C a 125°C

Ecuación de conversión: Se utiliza la ecuación simplificada de Steinhart-Hart:

$$1/T = 1/T_0 + (1/B) * \ln(R/R_0)$$

Donde:

- T = Temperatura absoluta en Kelvin
- T0 = Temperatura nominal (298.15K = 25°C)
- B = Coeficiente Beta (3950K)
- R = Resistencia actual del NTC
- R0 = Resistencia nominal (10k)

Manejo de errores:

- Retorna -999.0f en caso de circuito abierto o cortocircuito
- Verifica voltajes fuera de rango (>3.3V o <0V)
- Valida resistencias negativas o nulas

Definición en el archivo [ntc_driver.h](#).

9.15.2. Documentación de funciones

9.15.2.1. inicializar_ntc()

```
void inicializar_ntc (
    adc_oneshot_unit_handle_t manejador_adc)
```

Inicializa el canal ADC para el sensor NTC.

Configura el canal ADC_CHANNEL_6 (GPIO 34) con las siguientes características:

- Atenuación: 12dB (permite medir 0-3.3V)
- Resolución: 12 bits (valores 0-4095)
- Modo: Oneshot (lectura bajo demanda)

Esta función debe llamarse antes de [leer_temperatura_celsius\(\)](#). No es thread-safe - llamar solo desde una tarea o proteger con mutex.

Parámetros

in	<i>manejador_adc</i>	Handle del ADC previamente inicializado con <code>adc_oneshot_new_unit()</code>
----	----------------------	---

Atención

El `manejador_adc` debe ser válido. No verifica NULL.
Llamar solo una vez durante la inicialización del sistema.

Ver también

[leer_temperatura_celsius\(\)](#)
`adc_oneshot_config_channel()` de ESP-IDF

Inicializa el canal ADC para el sensor NTC.

Establece la configuración del canal ADC con:

- Atenuación 12dB: rango completo 0-3.3V
- Resolución por defecto: 12 bits (0-4095)

La atenuación de 12dB es necesaria porque el divisor de voltaje produce voltajes en todo el rango 0-3.3V dependiendo de la temperatura.

Parámetros

in	<i>manejador_adc</i>	Handle del ADC previamente inicializado
----	----------------------	---

Definición en la línea 87 del archivo `ntc_driver.c`.

Hace referencia a [CANAL_ADC_NTC](#).

Referenciado por [sensors_init\(\)](#).

9.15.2.2. leer_temperatura_celsius()

```
float leer_temperatura_celsius (
    adc_onehot_unit_handle_t manejador_adc)
```

Lee la temperatura actual del sensor NTC en grados Celsius.

Proceso de lectura y conversión:

1. Lee valor ADC raw (0-4095)
2. Convierte a voltaje: $V = (ADC_raw * 3300mV) / 4095$
3. Calcula resistencia NTC: $R_ntc = R_div * V / (V_ref - V)$
4. Aplica Steinhart-Hart: $1/T = 1/T_0 + (1/B) * \ln(R/R_0)$
5. Convierte Kelvin a Celsius: $T_c = T_k - 273.15$

Ecuación del divisor de voltaje:

$$V_{out} = V_{in} * (R_{ntc} / (R_{div} + R_{ntc}))$$
Precisión:

- Resolución ADC: $\sim 0.8mV$ ($3300mV / 4095$)
- Error típico: $\pm 1^\circ C$ en rango 0-50 $^\circ C$
- Error aumenta fuera del rango nominal

Parámetros

in	manejador_adc	Handle del ADC (debe estar inicializado)
----	---------------	--

Devuelve

float Temperatura en grados Celsius

Valores devueltos

-999.0f	Error de lectura (circuito abierto/cortocircuito)
-40.0	a 125.0 Rango válido de temperatura

Atención

Retorna -999.0f si:

- Voltaje $\geq 3.3V$ (circuito abierto)
- Resistencia ≤ 0 (cortocircuito o error de cálculo)

El manejador_adc debe haberse inicializado con [inicializar_ntc\(\)](#)

Nota

No es thread-safe. Si se llama desde múltiples tareas, proteger con mutex.

Tiempo de ejecución típico: ~100s

Ver también

[inicializar_ntc\(\)](#)

[sensor_task.c](#) Uso desde tarea RTOS con mutex

Lee la temperatura actual del sensor NTC en grados Celsius.

Algoritmo de conversión:

1. **Lectura ADC:** Obtiene valor digital (0-4095)
2. **Conversión a voltaje:** $V = \text{ADC_raw} * (3300\text{mV} / 4095)$
3. **Cálculo de resistencia NTC:**

$$R_{\text{ntc}} = R_{\text{div}} * V / (V_{\text{ref}} - V)$$
4. **Ecuación Steinhart-Hart simplificada:**

$$1/T = 1/T_0 + (1/B) * \ln(R_{\text{ntc}} / R_0)$$
5. **Conversión K → °C:** $T_{\text{celsius}} = T_{\text{kelvin}} - 273.15$

Detección de errores:

- V V_ref: Circuito abierto (NTC desconectado)
- R_ntc 0: Cortocircuito o error matemático

Parámetros

in	<i>manejador_adc</i>	Handle del ADC configurado
----	----------------------	----------------------------

Devuelve

Temperatura en °C o -999.0f en caso de error

Definición en la línea 122 del archivo [ntc_driver.c](#).

Hace referencia a [CANAL_ADC_NTC](#), [COEFICIENTE_BETA_NTC](#), [RESISTENCIA_DIVISOR](#), [RESISTENCIA_NOMINAL_NTC](#), [TEMP_NOMINAL_NTC](#) y [VOLTAJE_REFERENCIA_MV](#).

Referenciado por [fan_control_set_mode\(\)](#), [fan_control_update\(\)](#), [http_server_get_dht_sensor_readings_json_handler\(\)](#) y [sensor_update_task\(\)](#).

9.16. ntc_driver.h

[Ir a la documentación de este archivo.](#)

```
00001
00035
00036 #ifndef CONTROLADOR_NTC_H
00037 #define CONTROLADOR_NTC_H
00038
00039 #include "esp_adc/adc_oneshot.h" // Librería para usar el conversor Analógico-Digital (ADC).
00040
00061 void inicializar_ntc(adc_oneshot_unit_handle_t manejador_adc);
00062
00101 float leer_temperatura_celsius(adc_oneshot_unit_handle_t manejador_adc);
00102
00103 #endif // CONTROLADOR_NTC_H
```

9.17. Referencia del archivo pir_driver.c

Implementación del driver PIR con soporte ISR y polling.

```
#include "pir_driver.h"
#include "driver/gpio.h"
```

Funciones

- static void IRAM_ATTR [pir_isr_handler](#) (void *arg)
ISR (Interrupt Service Routine) del sensor PIR.
- void [pir_init](#) (gpio_num_t pir_gpio, QueueHandle_t pir_queue)
Inicializa el sensor PIR configurando GPIO e ISR.
- bool [pir_is_motion_active](#) (void)
Lee estado del sensor PIR mediante polling.

Variables

- static gpio_num_t [s_pir_gpio](#) = GPIO_NUM_NC
Pin GPIO configurado para el sensor PIR.
- static QueueHandle_t [s_pir_evt_queue](#) = NULL
Cola para eventos del PIR (NULL si se usa modo polling).

9.17.1. Descripción detallada

Implementación del driver PIR con soporte ISR y polling.

Autor

Jair Hernan Telpis Cuaran, Luis Fernando Gamba Bedoya

Fecha

2025

Implementa driver para sensor PIR HC-SR501 con dos modos de operación:

Modo ISR (con cola):

- Detecta flancos de subida y bajada automáticamente
- Envía eventos a cola FreeRTOS
- Ideal para respuesta inmediata

Modo Polling (sin cola):

- Lee estado bajo demanda con `pir_is_motion_active()`
- Más simple, usado por `sensor_task`
- Menor consumo de CPU

Variables estáticas:

- `s_pir_gpio`: Pin configurado para el sensor
- `s_pir_evt_queue`: Cola para eventos (NULL en modo polling)

Ver también

[pir_driver.h](#) Para la API pública

Definición en el archivo [pir_driver.c](#).

9.17.2. Documentación de funciones**9.17.2.1. `pir_init()`**

```
void pir_init (
    gpio_num_t pir_gpio,
    QueueHandle_t pir_queue)
```

Inicializa el sensor PIR configurando GPIO e ISR.

Inicializa el sensor PIR con ISR o polling.

Secuencia de inicialización:

1. Guarda parámetros en variables estáticas
2. Configura GPIO como entrada sin pull-up/down
3. Habilita interrupción en ambos flancos (ANYEDGE)
4. Instala servicio ISR global (si no existe)
5. Registra handler específico para este pin

Configuración GPIO:

- Modo INPUT: Lee nivel lógico del sensor
- Sin pull-up/down: Sensor tiene electrónica interna
- ANYEDGE: Detecta inicio (LOW→HIGH) y fin (HIGH→LOW)

Alternativa solo subida: Cambiar a `GPIO_INTR_POSEDGE` para solo notificar inicio de movimiento.

Parámetros

in	<i>pir_gpio</i>	Pin GPIO (ej: GPIO_NUM_35)
in	<i>pir_queue</i>	Cola FreeRTOS o NULL para modo polling

Definición en la línea 102 del archivo [pir_driver.c](#).

Hace referencia a [pir_isr_handler\(\)](#), [s_pir_evt_queue](#) y [s_pir_gpio](#).

Referenciado por [sensors_init\(\)](#).

9.17.2.2. [pir_is_motion_active\(\)](#)

```
bool pir_is_motion_active (
    void )
```

Lee estado del sensor PIR mediante polling.

Lee el estado actual del sensor PIR (polling).

Lee directamente el nivel del GPIO sin usar interrupciones.

Proceso:

1. Verifica que el GPIO esté inicializado
2. Lee nivel con [gpio_get_level\(\)](#)
3. Retorna true si nivel es HIGH (1)

Seguridad: Si el sensor no se inicializó, retorna false para evitar lecturas erróneas.

Devuelve

bool Estado actual del sensor

Valores devueltos

<i>true</i>	Movimiento detectado (GPIO HIGH)
<i>false</i>	Sin movimiento o sensor no inicializado

Definición en la línea 146 del archivo [pir_driver.c](#).

Hace referencia a [s_pir_gpio](#).

Referenciado por [fan_control_set_mode\(\)](#), [fan_control_update\(\)](#) y [http_server_get_dht_sensor_readings_json_handler\(\)](#).

9.17.2.3. `pir_isr_handler()`

```
void IRAM_ATTR pir_isr_handler (
    void * arg) [static]
```

ISR (Interrupt Service Routine) del sensor PIR.

Se ejecuta automáticamente cuando el pin del PIR cambia de estado.

Proceso:

1. Verifica que la cola esté configurada
2. Lee nivel actual del GPIO
3. Crea evento con estado del sensor
4. Envía a cola desde ISR (`xQueueSendFromISR`)
5. Yield si una tarea de mayor prioridad está esperando

Características ISR:

- Atributo `IRAM_ATTR`: Código en RAM para ejecución rápida
- Context: Interrupt context (no puede usar funciones bloqueantes)
- Tiempo: <10s típico

Parámetros

<code>in</code>	<code>arg</code>	Parámetro de usuario (no usado)
-----------------	------------------	---------------------------------

Nota

Solo se ejecuta si se configuró cola en [pir_init\(\)](#)

Atención

No llamar directamente - ejecutada por hardware

Definición en la línea 59 del archivo [pir_driver.c](#).

Hace referencia a [s_pir_evt_queue](#) y [s_pir_gpio](#).

Referenciado por [pir_init\(\)](#).

9.17.3. Documentación de variables

9.17.3.1. `s_pir_evt_queue`

```
QueueHandle_t s_pir_evt_queue = NULL [static]
```

Cola para eventos del PIR (NULL si se usa modo polling).

Definición en la línea 34 del archivo [pir_driver.c](#).

Referenciado por [pir_init\(\)](#) y [pir_isr_handler\(\)](#).

9.17.3.2. s_pir_gpio

gpio_num_t s_pir_gpio = GPIO_NUM_NC [static]

Pin GPIO configurado para el sensor PIR.

Definición en la línea 31 del archivo [pir_driver.c](#).

Referenciado por [pir_init\(\)](#), [pir_is_motion_active\(\)](#) y [pir_isr_handler\(\)](#).

9.18. pir_driver.c

[Ir a la documentación de este archivo.](#)

```

00001
00026
00027 #include "pir_driver.h"
00028 #include "driver/gpio.h"
00029
00031 static gpio_num_t s_pir_gpio = GPIO_NUM_NC;
00032
00034 static QueueHandle_t s_pir_evt_queue = NULL;
00035
00059 static void IRAM_ATTR pir_isr_handler(void *arg)
00060 {
00061     if (s_pir_evt_queue == NULL) {
00062         return;
00063     }
00064
00065     int level = gpio_get_level(s_pir_gpio);
00066
00067     pir_event_t evt = {
00068         .motion = (level == 1)
00069     };
00070
00071     BaseType_t xHigherPriorityTaskWoken = pdFALSE;
00072     xQueueSendFromISR(s_pir_evt_queue, &evt, &xHigherPriorityTaskWoken);
00073
00074     if (xHigherPriorityTaskWoken == pdTRUE) {
00075         portYIELD_FROM_ISR();
00076     }
00077 }
00078
00102 void pir_init(gpio_num_t pir_gpio, QueueHandle_t pir_queue)
00103 {
00104     s_pir_gpio = pir_gpio;
00105     s_pir_evt_queue = pir_queue;
00106
00107     // Configuración del GPIO como entrada con detección de flancos
00108     gpio_config_t io_conf = {
00109         .pin_bit_mask = 1ULL << s_pir_gpio, // Máscara de bits para el pin
00110         .mode = GPIO_MODE_INPUT, // Modo entrada
00111         .pull_up_en = GPIO_PULLUP_DISABLE, // Sin pull-up (sensor lo incluye)
00112         .pull_down_en = GPIO_PULLDOWN_DISABLE, // Sin pull-down
00113         .intr_type = GPIO_INTR_ANYEDGE // Interrupción en ambos flancos
00114         // Alternativa: GPIO_INTR_POSEDGE para solo flanco de subida
00115     };
00116
00117     gpio_config(&io_conf);
00118
00119     // Instalar servicio global de ISR (solo primera vez)
00120     // Flags=0 usa valores por defecto
00121     gpio_install_isr_service(0);
00122
00123     // Registrar ISR específica para este pin
00124     gpio_isr_handler_add(s_pir_gpio, pir_isr_handler, NULL);
00125 }
00126
00146 bool pir_is_motion_active(void)
00147 {
00148     if (s_pir_gpio == GPIO_NUM_NC) {
00149         return false;
00150     }
00151     int level = gpio_get_level(s_pir_gpio);
00152     return (level == 1);
00153 }

```

9.19. Referencia del archivo pir_driver.h

Driver para sensor PIR (Passive Infrared) de detección de movimiento.

```
#include "freertos/FreeRTOS.h"
#include "freertos/queue.h"
#include "driver/gpio.h"
#include <stdbool.h>
```

Estructuras de datos

- struct [pir_event_t](#)

Evento enviado por la cola cuando el PIR detecta cambio.

Funciones

- void [pir_init](#) (gpio_num_t pir_gpio, QueueHandle_t pir_queue)
Inicializa el sensor PIR con ISR o polling.
- bool [pir_is_motion_active](#) (void)
Lee el estado actual del sensor PIR (polling).

9.19.1. Descripción detallada

Driver para sensor PIR (Passive Infrared) de detección de movimiento.

Autor

Jair Hernan Telpis Cuaran, Luis Fernando Gamba Bedoya

Fecha

2025

Versión

1.0

Este módulo proporciona driver para sensores PIR HC-SR501 o compatibles. El sensor detecta movimiento mediante cambios en la radiación infrarroja.

Hardware:

- Sensor PIR HC-SR501 (o compatible)
- Pin OUT conectado a GPIO 35 (configurable)
- VCC: 5V, GND: GND
- Salida digital: HIGH cuando detecta movimiento

Modos de operación:

1. **Polling:** Llamar `pir_is_motion_active()` periódicamente
2. **Interrupt:** Pasar una cola a `pir_init()` para recibir eventos

Configuración del sensor HC-SR501:

- Delay time: Ajusta cuánto tiempo permanece HIGH después de detección
- Sensitivity: Ajusta el rango de detección (3-7 metros típico)
- Trigger mode: Repeatable (H) o Non-repeatable (L)

Nota

Este driver soporta tanto ISR como polling

Ver también

[sensor_task.c](#) Para integración con sistema de sensores

Definición en el archivo [pir_driver.h](#).

9.19.2. Documentación de funciones

9.19.2.1. `pir_init()`

```
void pir_init (
    gpio_num_t pir_gpio,
    QueueHandle_t pir_queue)
```

Inicializa el sensor PIR con ISR o polling.

Configura el GPIO como entrada y opcionalmente registra una ISR para detectar cambios automáticamente.

Configuración del GPIO:

- Modo: INPUT
- Pull-up: Deshabilitado (sensor tiene pull-up interno)
- Pull-down: Deshabilitado
- Interrupción: Ambos flancos (ANYEDGE) para detectar inicio y fin

Con cola (modo interrupt):

- Cada cambio en el pin dispara ISR
- ISR envía evento a la cola
- Tarea puede esperar en `xQueueReceive()`

Sin cola (modo polling):

- Pasar NULL como `pir_queue`
- Llamar `pir_is_motion_active()` periódicamente

Parámetros

in	<i>pir_gpio</i>	Pin GPIO donde está conectado el sensor (ej: GPIO_NUM_35)
in	<i>pir_queue</i>	Cola para eventos (puede ser NULL para modo polling)

Atención

Llamar solo una vez durante inicialización

Si se usa cola, crearla antes con `xQueueCreate()`

Nota

Instala servicio ISR global si aún no está instalado

Ver también

[pir_is_motion_active\(\)](#) Para lectura directa sin cola

Inicializa el sensor PIR con ISR o polling.

Secuencia de inicialización:

1. Guarda parámetros en variables estáticas
2. Configura GPIO como entrada sin pull-up/down
3. Habilita interrupción en ambos flancos (ANYEDGE)
4. Instala servicio ISR global (si no existe)
5. Registra handler específico para este pin

Configuración GPIO:

- Modo INPUT: Lee nivel lógico del sensor
- Sin pull-up/down: Sensor tiene electrónica interna
- ANYEDGE: Detecta inicio (LOW→HIGH) y fin (HIGH→LOW)

Alternativa solo subida: Cambiar a GPIO_INTR_POSEDGE para solo notificar inicio de movimiento.

Parámetros

in	<i>pir_gpio</i>	Pin GPIO (ej: GPIO_NUM_35)
in	<i>pir_queue</i>	Cola FreeRTOS o NULL para modo polling

Definición en la línea 102 del archivo [pir_driver.c](#).

Hace referencia a [pir_isr_handler\(\)](#), [s_pir_evt_queue](#) y [s_pir_gpio](#).

Referenciado por [sensors_init\(\)](#).

9.19.2.2. pir_is_motion_active()

```
bool pir_is_motion_active (
    void )
```

Lee el estado actual del sensor PIR (polling).

Lee directamente el nivel lógico del GPIO configurado. Útil para modo polling sin necesidad de ISR/cola.

Interpretación:

- true (HIGH): Sensor detecta movimiento ahora
- false (LOW): No hay movimiento detectado

Uso recomendado: Llamar desde tarea periódica (cada 1-2 segundos) en modo polling, o desde sensor_task para centralización thread-safe.

Devuelve

bool Estado del sensor

Valores devueltos

<i>true</i>	Movimiento detectado (pin en HIGH)
<i>false</i>	Sin movimiento (pin en LOW)
<i>false</i>	Si el sensor no ha sido inicializado

Nota

Thread-safe: solo lectura de GPIO

Tiempo de ejecución: <1s

Ver también

[pir_init\(\)](#) Debe llamarse antes de usar esta función
[sensor_task.c](#) Integración con sistema de sensores

Lee el estado actual del sensor PIR (polling).

Lee directamente el nivel del GPIO sin usar interrupciones.

Proceso:

1. Verifica que el GPIO esté inicializado
2. Lee nivel con gpio_get_level()
3. Retorna true si nivel es HIGH (1)

Seguridad: Si el sensor no se inicializó, retorna false para evitar lecturas erróneas.

Devuelve

bool Estado actual del sensor

Valores devueltos

<i>true</i>	Movimiento detectado (GPIO HIGH)
<i>false</i>	Sin movimiento o sensor no inicializado

Definición en la línea 146 del archivo `pir_driver.c`.

Hace referencia a `s_pir_gpio`.

Referenciado por `fan_control_set_mode()`, `fan_control_update()` y `http_server_get_dht_sensor_readings_json_handler()`.

9.20. `pir_driver.h`

[Ir a la documentación de este archivo.](#)

```

00001
00030
00031 #ifndef PIR_DRIVER_H
00032 #define PIR_DRIVER_H
00033
00034 #include "freertos/FreeRTOS.h"
00035 #include "freertos/queue.h"
00036 #include "driver/gpio.h"
00037 #include <stdbool.h>
00038
00062 typedef struct {
00063     bool motion;
00064 } pir_event_t;
00065
00098 void pir_init(gpio_num_t pir_gpio, QueueHandle_t pir_queue);
00099
00126 bool pir_is_motion_active(void);
00127
00128 #endif // PIR_DRIVER_H

```

9.21. Referencia del archivo `registers.c`

Implementación del sistema de programación de eventos (scheduler).

```

#include <stdio.h>
#include <string.h>
#include <time.h>
#include "sdkconfig.h"
#include "registers.h"
#include "esp_http_server.h"
#include "esp_log.h"
#include "nvs_flash.h"
#include "nvs.h"
#include "cJSON.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "fan_control.h"

```

Funciones

- void `init_nvs` ()
Inicializa el subsistema NVS (Non-Volatile Storage).
- `esp_err_t save_register_to_nvs` (int id, `reg_t` *reg)
Guarda un registro en NVS como JSON.
- `esp_err_t load_register_from_nvs` (int id, char *buffer, `size_t` buffer_len)
Carga un registro desde NVS como string JSON.
- void `delete_register_from_nvs` (int id)
Elimina un registro de NVS.
- static `esp_err_t load_register_struct_from_nvs` (int id, `reg_t` *out)
Helper: Carga registro desde NVS y parsea a estructura `reg_t`.
- static bool `register_is_active_now` (const `reg_t` *reg)
Determina si un registro debe activarse en este momento.
- `esp_err_t api_post_register` (`httpd_req_t` *req)
Handler HTTP POST para crear/actualizar registro.
- `esp_err_t api_get_registers` (`httpd_req_t` *req)
Handler HTTP GET para listar todos los registros.
- `esp_err_t api_delete_register` (`httpd_req_t` *req)
Handler HTTP DELETE para eliminar registro.
- static void `registers_scheduler_task` (void *arg)
Tarea RTOS que evalúa registros programados periódicamente.
- void `register_registers_endpoints` (`httpd_handle_t` server)
Registrar todos los endpoints REST en el servidor HTTP.

Variables

- static const char * `TAG` = "REGISTERS"
Tag para logging ESP-IDF.

9.21.1. Descripción detallada

Implementación del sistema de programación de eventos (scheduler).

Autor

Jair Hernan Telpis Cuaran, Luis Fernando Gamba Bedoya

Fecha

2025

Este módulo implementa un scheduler de eventos programados para activación automática del ventilador basado en hora y días de la semana.

Características:

- Hasta 10 registros programables
- Configuración por hora:minuto y días (L,M,X,J,V,S,D)

- Persistencia en NVS (flash) - sobrevive a reinicios
- Evaluación cada 10 segundos por tarea RTOS dedicada
- API REST completa (POST/GET/DELETE)

Arquitectura:

```

[Web UI] --POST--> /api/register --> save_register_to_nvs()
|
|                                     |
|                                     v
|                                     [NVS Flash]
|                                     ^
|                                     |
v                                     |
[registers_scheduler_task] ---cada 10s---> load_register_from_nvs()
|
|                                     |
v                                     v
register_is_active_now() <--- compara hora/día actual
|
v
fan_control_on_register_tick(any_match) --> Enciende/apaga ventilador

```

Formato JSON en NVS:

```

{
  "hour": 14,
  "minute": 30,
  "days": ["L", "X", "V"] // Lunes, Miércoles, Viernes
}

```

Mapeo de días:

- L = Lunes (tm_wday = 1)
- M = Martes (tm_wday = 2)
- X = Miércoles (tm_wday = 3)
- J = Jueves (tm_wday = 4)
- V = Viernes (tm_wday = 5)
- S = Sábado (tm_wday = 6)
- D = Domingo (tm_wday = 0)

Ver también

[registers.h](#) Para la API pública y estructura `reg_t`

[fan_control.c](#) Para activación del ventilador

Definición en el archivo [registers.c](#).

9.21.2. Documentación de funciones

9.21.2.1. api_delete_register()

```

esp_err_t api_delete_register (
    httpd_req_t * req)

```

Handler HTTP DELETE para eliminar registro.

Endpoint: DELETE /api/register?id=5

Parámetros

<i>req</i>	Request HTTP del servidor
------------	---------------------------

Devuelve

ESP_OK si se eliminó correctamente

Definición en la línea 508 del archivo registers.c.

Hace referencia a [delete_register_from_nvs\(\)](#) y TAG.

Referenciado por [register_registers_endpoints\(\)](#).

9.21.2.2. api_get_registers()

```
esp_err_t api_get_registers (
    httpd_req_t * req)
```

Handler HTTP GET para listar todos los registros.

Endpoint: GET /api/register Response JSON: [{"id":1,"hour":14,"minute":30,"days":["...]}]

Parámetros

<i>req</i>	Request HTTP del servidor
------------	---------------------------

Devuelve

ESP_OK si se procesó correctamente

Definición en la línea 451 del archivo registers.c.

Hace referencia a [load_register_from_nvs\(\)](#).

Referenciado por [register_registers_endpoints\(\)](#).

9.21.2.3. api_post_register()

```
esp_err_t api_post_register (
    httpd_req_t * req)
```

Handler HTTP POST para crear/actualizar registro.

Endpoint: POST /api/register Body JSON: {"id":1, "hour":14, "minute":30, "days":["Lunes","Viernes"]}

Parámetros

<i>req</i>	Request HTTP del servidor
------------	---------------------------

Devuelve

ESP_OK si se procesó correctamente

Definición en la línea 371 del archivo registers.c.

Hace referencia a [reg_t::day_count](#), [reg_t::days](#), [reg_t::hour](#), [reg_t::minute](#), [save_register_to_nvs\(\)](#) y TAG.

Referenciado por [register_registers_endpoints\(\)](#).

9.21.2.4. `delete_register_from_nvs()`

```
void delete_register_from_nvs (
    int id)
```

Elimina un registro de NVS.

Eliminar registro de NVS.

Borra la clave "reg_<id>" del namespace "storage".

Parámetros

in	id	ID del registro a eliminar (1-10)
----	----	-----------------------------------

Nota

No retorna error si el registro no existía (nvs_erase_key silencioso)

Atención

Requiere `nvs_commit()` para persistir (ya incluido)

Definición en la línea 229 del archivo [registers.c](#).

Referenciado por [api_delete_register\(\)](#) y [http_server_register_erase_handler\(\)](#).

9.21.2.5. `init_nvs()`

```
void init_nvs ()
```

Inicializa el subsistema NVS (Non-Volatile Storage).

Inicializar sistema NVS para almacenamiento persistente.

Inicializa la partición NVS en flash para almacenamiento persistente. Si la partición está corrupta o es versión antigua, la borra y reinicializa.

Manejo de errores:

- ESP_ERR_NVS_NO_FREE_PAGES: Partición llena → borra y reinicia
- ESP_ERR_NVS_NEW_VERSION_FOUND: Versión incompatible → borra y reinicia
- Otros errores: ESP_ERROR_CHECK causa panic

Atención

Llamar antes de cualquier operación NVS

Nota

Seguro llamar múltiples veces (NVS ignora reinicializaciones)

Definición en la línea 100 del archivo [registers.c](#).

9.21.2.6. load_register_from_nvs()

```
esp_err_t load_register_from_nvs (  
    int id,  
    char * buffer,  
    size_t buffer_len)
```

Carga un registro desde NVS como string JSON.

Cargar registro desde NVS.

Lee el string JSON almacenado en NVS para el ID especificado.

Proceso:

1. Abre namespace "storage" en modo lectura
2. Verifica tamaño requerido con `nvs_get_str(NULL)`
3. Lee string JSON en buffer proporcionado
4. Cierra handle NVS

Parámetros

in	<i>id</i>	ID del registro (1-10)
out	<i>buffer</i>	Buffer donde copiar el JSON
in	<i>buffer_len</i>	Tamaño del buffer

Devuelve

`esp_err_t`

Valores devueltos

<i>ESP_OK</i>	Lectura exitosa
<i>ESP_ERR_NVS_NOT_FOUND</i>	No existe registro con ese ID
<i>ESP_ERR_NVS_INVALID_LENGTH</i>	Buffer muy pequeño

Nota

No parsea el JSON - solo retorna string crudo

Ver también

[load_register_struct_from_nvs\(\)](#) Para obtener estructura `reg_t` directamente

Definición en la línea 192 del archivo `registers.c`.

Referenciado por [api_get_registers\(\)](#), [http_server_read_register_handler\(\)](#) y [load_register_struct_from_nvs\(\)](#).

9.21.2.7. load_register_struct_from_nvs()

```
esp_err_t load_register_struct_from_nvs (
    int id,
    reg_t * out) [static]
```

Helper: Carga registro desde NVS y parsea a estructura [reg_t](#).

Combina [load_register_from_nvs\(\)](#) + parsing JSON.

Proceso:

1. Lee string JSON de NVS
2. Parsea con cJSON_Parse()
3. Extrae campos: hour, minute, days
4. Valida tipos JSON (number, array)
5. Copia a estructura [reg_t](#)

Parámetros

in	id	ID del registro (1-10)
out	out	Puntero a reg_t donde copiar datos

Devuelve

esp_err_t

Valores devueltos

ESP_OK	Carga y parsing exitoso
ESP_ERR_NVS_NOT_FOUND	No existe registro
ESP_FAIL	JSON inválido o campos faltantes

Nota

Función estática (solo visible en este módulo)
Limita días a máximo 7

Definición en la línea 265 del archivo [registers.c](#).

Hace referencia a [reg_t::day_count](#), [reg_t::days](#), [reg_t::hour](#), [load_register_from_nvs\(\)](#) y [reg_t::minute](#).

Referenciado por [registers_scheduler_task\(\)](#).

9.21.2.8. register_is_active_now()

```
bool register_is_active_now (
    const reg_t * reg) [static]
```

Determina si un registro debe activarse en este momento.

Compara hora actual del sistema con el registro programado.

Algoritmo:

1. Obtiene hora del sistema (time() + localtime_r())
2. Compara hora:minuto exacto
3. Compara día de la semana (tm_wday)
4. Retorna true si hay coincidencia

Mapeo de días:

- tm_wday: 0=Domingo, 1=Lunes, 2=Martes, 3=Miércoles, 4=Jueves, 5=Viernes, 6=Sábado
- Registro: "L","M","X","J","V","S","D"

Ventana de activación: Solo activo durante el minuto exacto (14:30:00 a 14:30:59). Scheduler debe llamar esta función con frecuencia 60s.

Parámetros

in	reg	Puntero a registro a evaluar
----	-----	------------------------------

Devuelve

bool

Valores devueltos

true	Registro coincide con hora/día actual
false	No coincide o día no incluido

Nota

Función estática (solo visible en este módulo)

Atención

Requiere que el sistema tenga hora sincronizada (SNTP)

Definición en la línea 338 del archivo registers.c.

Hace referencia a [reg_t::day_count](#), [reg_t::days](#), [reg_t::hour](#) y [reg_t::minute](#).

Referenciado por [registers_scheduler_task\(\)](#).

9.21.2.9. register_registers_endpoints()

```
void register_registers_endpoints (  
    httpd_handle_t server)
```

Registrar todos los endpoints REST en el servidor HTTP.

Debe llamarse durante la configuración del servidor HTTP para habilitar los endpoints /api/register.

Parámetros

<i>server</i>	Handle del servidor HTTP configurado
---------------	--------------------------------------

Definición en la línea 592 del archivo [registers.c](#).

Hace referencia a [api_delete_register\(\)](#), [api_get_registers\(\)](#), [api_post_register\(\)](#), [registers_scheduler_task\(\)](#) y TAG.

Referenciado por [http_server_configure\(\)](#).

9.21.2.10. registers_scheduler_task()

```
void registers_scheduler_task (
    void * arg) [static]
```

Tarea RTOS que evalúa registros programados periódicamente.

Scheduler que evalúa los 10 registros cada 10 segundos.

Ciclo de ejecución:

1. Itera sobre IDs 1-10
2. Carga cada registro desde NVS
3. Evalúa con [register_is_active_now\(\)](#)
4. Si alguno coincide: any_match_now = true
5. Llama fan_control_on_register_tick(any_match_now)
6. Espera 10 segundos (vTaskDelay)

Optimización: Break al primer match (no evalúa registros restantes).

Parámetros de tarea:

- Nombre: "registers_scheduler"
- Stack: 4096 bytes
- Prioridad: 5 (media-alta)
- Periodo: 10000ms

Parámetros

<i>in</i>	<i>arg</i>	Parámetro de tarea (no usado)
-----------	------------	-------------------------------

Nota

Tarea infinita (nunca termina)

Atención

Periodo de 10s significa resolución mínima de activación de 10s

Ver también

[fan_control_on_register_tick\(\)](#) Para activación del ventilador

Definición en la línea 568 del archivo [registers.c](#).

Hace referencia a [fan_control_on_register_tick\(\)](#), [load_register_struct_from_nvs\(\)](#), [register_is_active_now\(\)](#) y TAG.

Referenciado por [register_registers_endpoints\(\)](#).

9.21.2.11. save_register_to_nvs()

```
esp_err_t save_register_to_nvs (
    int id,
    reg_t * reg)
```

Guarda un registro en NVS como JSON.

Guardar registro en NVS con formato JSON.

Serializa la estructura `reg_t` a JSON y la guarda en NVS con clave "reg_<id>".

Proceso:

1. Crea objeto JSON con hour, minute, days
2. Serializa a string con cJSON_Print()
3. Guarda en NVS con nvs_set_str()
4. Commit para escribir a flash

Ejemplo de JSON guardado:

```
{"hour":14,"minute":30,"days":["L","X","V"]}
```

Parámetros

in	<i>id</i>	ID del registro (1-10)
in	<i>reg</i>	Puntero a estructura <code>reg_t</code> a guardar

Devuelve

`esp_err_t`

Valores devueltos

<i>ESP_OK</i>	Guardado exitoso
<i>ESP_FAIL</i>	Error de NVS

Atención

Sobrescribe registro existente con mismo ID

Nota

Máximo 10 registros soportados (IDs 1-10)

Definición en la línea 137 del archivo `registers.c`.

Hace referencia a `reg_t::day_count`, `reg_t::days`, `reg_t::hour`, `reg_t::minute` y `TAG`.

Referenciado por `api_post_register()`.

9.21.3. Documentación de variables

9.21.3.1. TAG

```
const char* TAG = "REGISTERS" [static]
```

Tag para logging ESP-IDF.

Definición en la línea 83 del archivo [registers.c](#).

9.22. registers.c

[Ir a la documentación de este archivo.](#)

```
00001
00056
00057 #include <stdio.h>
00058 #include <string.h>
00059 #include <time.h>
00060
00061 #include "sdkconfig.h"
00062 #include "registers.h"
00063
00064 #include "esp_http_server.h"
00065 #include "esp_log.h"
00066 #include "nvs_flash.h"
00067 #include "nvs.h"
00068 #include "cJSON.h"
00069
00070 #include "freertos/FreeRTOS.h"
00071 #include "freertos/task.h"
00072
00073 #include "fan_control.h"
00074
00075 // Fix for VS Code IntelliSense
00076 #if defined(__INTELLISENSE__)
00077 #ifndef CONFIG_LOG_MAXIMUM_LEVEL
00078 #define CONFIG_LOG_MAXIMUM_LEVEL 5
00079 #endif
00080 #endif
00081
00083 static const char *TAG = "REGISTERS";
00084
00100 void init_nvs()
00101 {
00102     esp_err_t ret = nvs_flash_init();
00103     if (ret == ESP_ERR_NVS_NO_FREE_PAGES || ret == ESP_ERR_NVS_NEW_VERSION_FOUND) {
00104         ESP_ERROR_CHECK(nvs_flash_erase());
00105         ret = nvs_flash_init();
00106     }
00107     ESP_ERROR_CHECK(ret);
00108 }
00109
00137 esp_err_t save_register_to_nvs(int id, reg_t *reg)
00138 {
00139     nvs_handle_t nvs;
00140     char key[16];
00141     sprintf(key, "reg_%d", id);
00142
00143     cJSON *root = cJSON_CreateObject();
00144     cJSON_AddNumberToObject(root, "hour", reg->hour);
00145     cJSON_AddNumberToObject(root, "minute", reg->minute);
00146
00147     cJSON *days = cJSON_CreateArray();
00148     for (int i = 0; i < reg->day_count; i++) {
00149         cJSON_AddItemToArray(days, cJSON_CreateString(reg->days[i]));
00150     }
00151     cJSON_AddItemToObject(root, "days", days);
00152
00153     char *json_str = cJSON_Print(root);
00154
00155     ESP_LOGI(TAG, "save_register_to_nvs: id=%d json=%s", id, json_str);
00156
00157     ESP_ERROR_CHECK(nvs_open("storage", NVS_READWRITE, &nvs));
00158     esp_err_t err = nvs_set_str(nvs, key, json_str);
00159     nvs_commit(nvs);
```

```

00160     nvs_close(nvs);
00161
00162     free(json_str);
00163     cJSON_Delete(root);
00164
00165     return err;
00166 }
00167
00192 esp_err_t load_register_from_nvs(int id, char *buffer, size_t buffer_len)
00193 {
00194     nvs_handle_t nvs;
00195     char key[16];
00196     sprintf(key, "reg_%d", id);
00197
00198     ESP_ERROR_CHECK(nvs_open("storage", NVS_READWRITE, &nvs));
00199
00200     size_t required_size = 0;
00201     esp_err_t err = nvs_get_str(nvs, key, NULL, &required_size);
00202
00203     if (err == ESP_ERR_NVS_NOT_FOUND) {
00204         nvs_close(nvs);
00205         return ESP_ERR_NOT_FOUND;
00206     }
00207
00208     if (required_size > buffer_len) {
00209         nvs_close(nvs);
00210         return ESP_ERR_NVS_INVALID_LENGTH;
00211     }
00212
00213     err = nvs_get_str(nvs, key, buffer, &required_size);
00214     nvs_close(nvs);
00215     return err;
00216 }
00217
00229 void delete_register_from_nvs(int id)
00230 {
00231     nvs_handle_t nvs;
00232     char key[16];
00233     sprintf(key, "reg_%d", id);
00234
00235     ESP_ERROR_CHECK(nvs_open("storage", NVS_READWRITE, &nvs));
00236     nvs_erase_key(nvs, key);
00237     nvs_commit(nvs);
00238     nvs_close(nvs);
00239 }
00240
00265 static esp_err_t load_register_struct_from_nvs(int id, reg_t *out)
00266 {
00267     char buf[512];
00268     esp_err_t err = load_register_from_nvs(id, buf, sizeof(buf));
00269     if (err != ESP_OK) {
00270         return err;
00271     }
00272
00273     cJSON *root = cJSON_Parse(buf);
00274     if (!root) {
00275         return ESP_FAIL;
00276     }
00277
00278     cJSON *hour_json = cJSON_GetObjectItem(root, "hour");
00279     cJSON *min_json = cJSON_GetObjectItem(root, "minute");
00280     cJSON *days_json = cJSON_GetObjectItem(root, "days");
00281
00282     if (!cJSON_IsNumber(hour_json) || !cJSON_IsNumber(min_json) || !cJSON_IsArray(days_json)) {
00283         cJSON_Delete(root);
00284         return ESP_FAIL;
00285     }
00286
00287     memset(out, 0, sizeof(*out));
00288     out->hour = hour_json->valueint;
00289     out->minute = min_json->valueint;
00290     out->day_count = cJSON_GetArraySize(days_json);
00291     if (out->day_count > 7) {
00292         out->day_count = 7;
00293     }
00294
00295     for (int i = 0; i < out->day_count; i++) {
00296         cJSON *item = cJSON_GetArrayItem(days_json, i);
00297         if (cJSON_IsString(item) && item->valuestring) {
00298             strncpy(out->days[i], item->valuestring, sizeof(out->days[i]) - 1);
00299             out->days[i][sizeof(out->days[i]) - 1] = '\0';
00300         } else {
00301             out->days[i][0] = '\0';
00302         }
00303     }
00304
00305     cJSON_Delete(root);

```



```

00306     return ESP_OK;
00307 }
00308
00338 static bool register_is_active_now(const reg_t *reg)
00339 {
00340     time_t now;
00341     struct tm timeinfo;
00342     time(&now);
00343     localtime_r(&now, &timeinfo);
00344
00345     int cur_hour = timeinfo.tm_hour;
00346     int cur_min = timeinfo.tm_min;
00347     int cur_wday = timeinfo.tm_wday; // 0=domingo, 1=lunes, ... 6=sábado
00348
00349     if (reg->hour != cur_hour || reg->minute != cur_min) {
00350         return false;
00351     }
00352
00353     for (int i = 0; i < reg->day_count; i++) {
00354         const char *d = reg->days[i];
00355
00356         if ((cur_wday == 1 && strcmp(d, "L") == 0) || // lunes
00357             (cur_wday == 2 && strcmp(d, "M") == 0) || // martes
00358             (cur_wday == 3 && strcmp(d, "X") == 0) || // miércoles
00359             (cur_wday == 4 && strcmp(d, "J") == 0) || // jueves
00360             (cur_wday == 5 && strcmp(d, "V") == 0) || // viernes
00361             (cur_wday == 6 && strcmp(d, "S") == 0) || // sábado
00362             (cur_wday == 0 && strcmp(d, "D") == 0)) { // domingo
00363             return true;
00364         }
00365     }
00366     return false;
00367 }
00368 }
00369
00370 // ----- HTTP: POST /api/register -----
00371 esp_err_t api_post_register(httpd_req_t *req)
00372 {
00373     int len = req->content_len;
00374     if (len <= 0 || len > 4096) {
00375         httpd_resp_send_err(req, HTTPD_400_BAD_REQUEST, "invalid length");
00376         return ESP_FAIL;
00377     }
00378
00379     char *buf = malloc(len + 1);
00380     if (!buf) {
00381         httpd_resp_send_err(req, HTTPD_500_INTERNAL_SERVER_ERROR, "no mem");
00382         return ESP_FAIL;
00383     }
00384
00385     int ret = httpd_req_recv(req, buf, len);
00386     if (ret <= 0) {
00387         free(buf);
00388         httpd_resp_send_err(req, HTTPD_500_INTERNAL_SERVER_ERROR, "recv failed");
00389         return ESP_FAIL;
00390     }
00391     buf[ret] = '\0';
00392
00393     cJSON *root = cJSON_Parse(buf);
00394     free(buf);
00395
00396     if (!root) {
00397         httpd_resp_send_err(req, HTTPD_400_BAD_REQUEST, "invalid json");
00398         return ESP_FAIL;
00399     }
00400
00401     cJSON *id_json = cJSON_GetObjectItem(root, "register");
00402     cJSON *hour_json = cJSON_GetObjectItem(root, "hour");
00403     cJSON *min_json = cJSON_GetObjectItem(root, "minute");
00404     cJSON *days = cJSON_GetObjectItem(root, "days");
00405
00406     if (!cJSON_IsNumber(id_json) || !cJSON_IsNumber(hour_json) ||
00407         !cJSON_IsNumber(min_json) || !cJSON_IsArray(days)) {
00408         cJSON_Delete(root);
00409         httpd_resp_send_err(req, HTTPD_400_BAD_REQUEST, "missing fields");
00410         return ESP_FAIL;
00411     }
00412
00413     int id = id_json->valueint;
00414     int hour = hour_json->valueint;
00415     int minute = min_json->valueint;
00416
00417     int day_count = cJSON_GetArraySize(days);
00418     if (day_count > 7) day_count = 7;
00419
00420     reg_t reg = {0};
00421     reg.hour = hour;

```

```

00422     reg.minute    = minute;
00423     reg.day_count = day_count;
00424
00425     for (int i = 0; i < day_count; i++) {
00426         cJSON *item = cJSON_GetArrayItem(days, i);
00427         if (cJSON_IsString(item) && item->valuelstring) {
00428             strncpy(reg.days[i], item->valuelstring, sizeof(reg.days[i]) - 1);
00429             reg.days[i][sizeof(reg.days[i]) - 1] = '\\0';
00430         } else {
00431             reg.days[i][0] = '\\0';
00432         }
00433     }
00434
00435     esp_err_t err = save_register_to_nvs(id, &reg);
00436     cJSON_Delete(root);
00437
00438     if (err != ESP_OK) {
00439         httpd_resp_send_err(req, HTTPD_500_INTERNAL_SERVER_ERROR, "nvs error");
00440         return ESP_FAIL;
00441     }
00442
00443     ESP_LOGI(TAG, "api_post_register: saved id=%d hour=%d minute=%d days=%d",
00444              id, hour, minute, day_count);
00445
00446     httpd_resp_sendstr(req, "OK");
00447     return ESP_OK;
00448 }
00449
00450 // ----- HTTP: GET /api/registers -----
00451 esp_err_t api_get_registers(httpd_req_t *req)
00452 {
00453     cJSON *root = cJSON_CreateObject();
00454     char smallbuf[256];
00455
00456     for (int i = 1; i <= 10; i++) {
00457         char key[8];
00458         snprintf(key, sizeof(key), "%d", i);
00459
00460         esp_err_t r = load_register_from_nvs(i, smallbuf, sizeof(smallbuf));
00461         if (r == ESP_OK) {
00462             cJSON *reg_json = cJSON_Parse(smallbuf);
00463             if (reg_json) {
00464                 cJSON_AddItemToObject(root, key, reg_json);
00465                 continue;
00466             }
00467         } else if (r == ESP_ERR_NVS_INVALID_LENGTH) {
00468             nvs_handle_t nvs;
00469             char nkey[16];
00470             snprintf(nkey, sizeof(nkey), "reg_%d", i);
00471             if (nvs_open("storage", NVS_READWRITE, &nvs) == ESP_OK) {
00472                 size_t required = 0;
00473                 if (nvs_get_str(nvs, nkey, NULL, &required) == ESP_OK && required > 0) {
00474                     char *dyn = malloc(required);
00475                     if (dyn) {
00476                         if (nvs_get_str(nvs, nkey, dyn, &required) == ESP_OK) {
00477                             cJSON *reg_json = cJSON_Parse(dyn);
00478                             if (reg_json) {
00479                                 cJSON_AddItemToObject(root, key, reg_json);
00480                             }
00481                         }
00482                         free(dyn);
00483                     }
00484                 }
00485                 nvs_close(nvs);
00486                 continue;
00487             }
00488         }
00489
00490         cJSON_AddNullToObject(root, key);
00491     }
00492
00493     char *out = cJSON_PrintUnformatted(root);
00494
00495     httpd_resp_set_type(req, "application/json");
00496     httpd_resp_send(req, out, strlen(out));
00497
00498     free(out);
00499     cJSON_Delete(root);
00500     return ESP_OK;
00501 }
00502 }
00503
00504
00505 // Removed redundant local wrapper: http_server.c provides its own handler.
00506
00507 // ----- HTTP: DELETE /api/register/{id} -----
00508 esp_err_t api_delete_register(httpd_req_t *req)

```

```

00509 {
00510     const char *uri = req->uri; // e.g. "/api/register/3"
00511     int id = 0;
00512
00513     if (uri) {
00514         const char *last = strrchr(uri, '/');
00515         if (last && *(last + 1) != '\0') {
00516             id = atoi(last + 1);
00517         }
00518     }
00519
00520     if (id <= 0) {
00521         char query[16] = {0};
00522         if (httpd_req_get_url_query_str(req, query, sizeof(query)) == ESP_OK) {
00523             id = atoi(query);
00524         }
00525     }
00526
00527     if (id <= 0) {
00528         httpd_resp_send_err(req, HTTPD_400_BAD_REQUEST, "invalid id");
00529         return ESP_FAIL;
00530     }
00531
00532     delete_register_from_nvs(id);
00533     ESP_LOGI(TAG, "api_delete_register: deleted id=%d", id);
00534
00535     httpd_resp_sendstr(req, "Deleted");
00536     return ESP_OK;
00537 }
00538
00568 static void registers_scheduler_task(void *arg)
00569 {
00570     ESP_LOGI(TAG, "registers_scheduler_task: started");
00571
00572     while (1) {
00573         bool any_match_now = false;
00574
00575         for (int i = 1; i <= 10; i++) {
00576             reg_t reg;
00577             if (load_register_struct_from_nvs(i, &reg) == ESP_OK) {
00578                 if (register_is_active_now(&reg)) {
00579                     any_match_now = true;
00580                     break;
00581                 }
00582             }
00583         }
00584
00585         fan_control_on_register_tick(any_match_now);
00586
00587         vTaskDelay(pdMS_TO_TICKS(10000)); // cada 10 s
00588     }
00589 }
00590
00591 // ----- REGISTRO DE ENDPOINTS -----
00592 void register_registers_endpoints(httpd_handle_t server)
00593 {
00594     ESP_LOGI(TAG, "register_registers_endpoints: registering /api/register endpoints");
00595
00596     httpd_uri_t post_register_uri = {
00597         .uri = "/api/register",
00598         .method = HTTP_POST,
00599         .handler = api_post_register
00600     };
00601
00602     httpd_uri_t get_registers_uri = {
00603         .uri = "/api/registers",
00604         .method = HTTP_GET,
00605         .handler = api_get_registers
00606     };
00607
00608     httpd_uri_t delete_register_uri = {
00609         .uri = "/api/register/*",
00610         .method = HTTP_DELETE,
00611         .handler = api_delete_register
00612     };
00613
00614     httpd_register_uri_handler(server, &post_register_uri);
00615     httpd_register_uri_handler(server, &get_registers_uri);
00616     httpd_register_uri_handler(server, &delete_register_uri);
00617
00618     static bool s_scheduler_started = false;
00619     if (!s_scheduler_started) {
00620         if (xTaskCreate(
00621             registers_scheduler_task,
00622             "registers_scheduler",
00623             4096,
00624             NULL,

```

```

00625         5,
00626         NULL
00627     ) == pdPASS)
00628 {
00629     s_scheduler_started = true;
00630     ESP_LOGI(TAG, "registers_scheduler_task created");
00631 } else {
00632     ESP_LOGE(TAG, "Failed to create registers_scheduler_task");
00633 }
00634 }
00635 }

```

9.23. Referencia del archivo registers.h

API REST para gestión de registros programados con persistencia NVS.

```

#include "esp_err.h"
#include "esp_http_server.h"

```

Estructuras de datos

- struct [reg_t](#)
Estructura de un registro programado.

Funciones

- void [init_nvs](#) ()
Inicializar sistema NVS para almacenamiento persistente.
- esp_err_t [save_register_to_nvs](#) (int id, [reg_t](#) *reg)
Guardar registro en NVS con formato JSON.
- esp_err_t [load_register_from_nvs](#) (int id, char *buffer, size_t buffer_len)
Cargar registro desde NVS.
- void [delete_register_from_nvs](#) (int id)
Eliminar registro de NVS.
- esp_err_t [api_post_register](#) (httpd_req_t *req)
Handler HTTP POST para crear/actualizar registro.
- esp_err_t [api_get_registers](#) (httpd_req_t *req)
Handler HTTP GET para listar todos los registros.
- esp_err_t [api_delete_register](#) (httpd_req_t *req)
Handler HTTP DELETE para eliminar registro.
- void [register_registers_endpoints](#) (httpd_handle_t server)
Registrar todos los endpoints REST en el servidor HTTP.

9.23.1. Descripción detallada

API REST para gestión de registros programados con persistencia NVS.

Autor

Jair Hernan Telpis Cuaran, Luis Fernando Gamba Bedoya

Fecha

2025

Versión

1.0.0

Sistema completo de registros programados que permite:

- Crear, leer y eliminar registros mediante endpoints HTTP REST
- Almacenar registros en NVS (Non-Volatile Storage) con formato JSON
- Comparar hora actual con registros para activar automáticamente el ventilador
- Sincronización mediante SNTP para precisión horaria

Cada registro define una hora, minuto y días de la semana para ejecutar una acción (ej: encender ventilador). Los registros se almacenan en NVS con claves "reg01" hasta "reg10" y persisten tras reinicios.

Endpoints HTTP:

- POST /api/register - Crear/actualizar registro
- GET /api/register - Listar todos los registros
- DELETE /api/register?id=X - Eliminar registro

Universidad Nacional de Colombia - Curso RTOS

Definición en el archivo [registers.h](#).

9.23.2. Documentación de funciones

9.23.2.1. api_delete_register()

```
esp_err_t api_delete_register (  
    httpd_req_t * req)
```

Handler HTTP DELETE para eliminar registro.

Endpoint: DELETE /api/register?id=5

Parámetros

<i>req</i>	Request HTTP del servidor
------------	---------------------------

Devuelve

ESP_OK si se eliminó correctamente

Definición en la línea 508 del archivo [registers.c](#).

Hace referencia a [delete_register_from_nvs\(\)](#) y [TAG](#).

Referenciado por [register_registers_endpoints\(\)](#).

9.23.2.2. api_get_registers()

```
esp_err_t api_get_registers (
    httpd_req_t * req)
```

Handler HTTP GET para listar todos los registros.

Endpoint: GET /api/register Response JSON: [{"id":1,"hour":14,"minute":30,"days":[...]}]

Parámetros

<i>req</i>	Request HTTP del servidor
------------	---------------------------

Devuelve

ESP_OK si se procesó correctamente

Definición en la línea 451 del archivo [registers.c](#).

Hace referencia a [load_register_from_nvs\(\)](#).

Referenciado por [register_registers_endpoints\(\)](#).

9.23.2.3. api_post_register()

```
esp_err_t api_post_register (
    httpd_req_t * req)
```

Handler HTTP POST para crear/actualizar registro.

Endpoint: POST /api/register Body JSON: {"id":1, "hour":14, "minute":30, "days":["Lunes","Viernes"]}

Parámetros

<i>req</i>	Request HTTP del servidor
------------	---------------------------

Devuelve

ESP_OK si se procesó correctamente

Definición en la línea 371 del archivo [registers.c](#).

Hace referencia a [reg_t::day_count](#), [reg_t::days](#), [reg_t::hour](#), [reg_t::minute](#), [save_register_to_nvs\(\)](#) y [TAG](#).

Referenciado por [register_registers_endpoints\(\)](#).

9.23.2.4. delete_register_from_nvs()

```
void delete_register_from_nvs (
    int id)
```

Eliminar registro de NVS.

Borra permanentemente el registro con el ID especificado de la memoria no volátil.

Parámetros

<i>id</i>	Identificador del registro a eliminar (1-10)
-----------	--

Eliminar registro de NVS.

Borra la clave "reg_<id>" del namespace "storage".

Parámetros

in	<i>id</i>	ID del registro a eliminar (1-10)
----	-----------	-----------------------------------

Nota

No retorna error si el registro no existía (nvs_erase_key silencioso)

Atención

Requiere nvs_commit() para persistir (ya incluido)

Definición en la línea 229 del archivo [registers.c](#).

Referenciado por [api_delete_register\(\)](#) y [http_server_register_erase_handler\(\)](#).

9.23.2.5. init_nvs()

```
void init_nvs ()
```

Inicializar sistema NVS para almacenamiento persistente.

Debe llamarse una vez durante el arranque del sistema antes de usar cualquier función de registros.

Nota

Inicializa la partición NVS requerida para guardar registros

Inicializar sistema NVS para almacenamiento persistente.

Inicializa la partición NVS en flash para almacenamiento persistente. Si la partición está corrupta o es versión antigua, la borra y reinicializa.

Manejo de errores:

- ESP_ERR_NVS_NO_FREE_PAGES: Partición llena → borra y reinicia
- ESP_ERR_NVS_NEW_VERSION_FOUND: Versión incompatible → borra y reinicia
- Otros errores: ESP_ERROR_CHECK causa panic

Atención

Llamar antes de cualquier operación NVS

Nota

Seguro llamar múltiples veces (NVS ignora reinicializaciones)

Definición en la línea 100 del archivo [registers.c](#).

9.23.2.6. load_register_from_nvs()

```
esp_err_t load_register_from_nvs (  
    int id,  
    char * buffer,  
    size_t buffer_len)
```

Cargar registro desde NVS.

Lee el registro con el ID especificado desde NVS y lo retorna como string JSON en el buffer proporcionado.

Parámetros

	<i>id</i>	Identificador del registro (1-10)
out	<i>buffer</i>	Buffer donde se escribirá el JSON
	<i>buffer_len</i>	Tamaño del buffer

Devuelve

ESP_OK si el registro existe y se cargó
ESP_ERR_NOT_FOUND si no existe registro con ese ID
ESP_ERR_INVALID_SIZE si el buffer es muy pequeño

Cargar registro desde NVS.

Lee el string JSON almacenado en NVS para el ID especificado.

Proceso:

1. Abre namespace "storage" en modo lectura
2. Verifica tamaño requerido con `nvs_get_str(NULL)`
3. Lee string JSON en buffer proporcionado
4. Cierra handle NVS

Parámetros

in	<i>id</i>	ID del registro (1-10)
out	<i>buffer</i>	Buffer donde copiar el JSON
in	<i>buffer_len</i>	Tamaño del buffer

Devuelve

esp_err_t

Valores devueltos

<i>ESP_OK</i>	Lectura exitosa
<i>ESP_ERR_NVS_NOT_FOUND</i>	No existe registro con ese ID
<i>ESP_ERR_NVS_INVALID_LENGTH</i>	Buffer muy pequeño

Nota

No parsea el JSON - solo retorna string crudo

Ver también

[load_register_struct_from_nvs\(\)](#) Para obtener estructura [reg_t](#) directamente

Definición en la línea 192 del archivo [registers.c](#).

Referenciado por [api_get_registers\(\)](#), [http_server_read_register_handler\(\)](#) y [load_register_struct_from_nvs\(\)](#).

9.23.2.7. register_registers_endpoints()

```
void register_registers_endpoints (
    httpd_handle_t server)
```

Registrar todos los endpoints REST en el servidor HTTP.

Debe llamarse durante la configuración del servidor HTTP para habilitar los endpoints /api/register.

Parámetros

<i>server</i>	Handle del servidor HTTP configurado
---------------	--------------------------------------

Definición en la línea 592 del archivo [registers.c](#).

Hace referencia a [api_delete_register\(\)](#), [api_get_registers\(\)](#), [api_post_register\(\)](#), [registers_scheduler_task\(\)](#) y TAG.

Referenciado por [http_server_configure\(\)](#).

9.23.2.8. save_register_to_nvs()

```
esp_err_t save_register_to_nvs (
    int id,
    reg_t * reg)
```

Guardar registro en NVS con formato JSON.

Serializa el registro a JSON y lo almacena en NVS con clave "regXX". Si ya existe un registro con ese ID, lo sobrescribe.

Parámetros

<i>id</i>	Identificador del registro (1-10)
<i>reg</i>	Puntero a estructura reg_t con los datos del registro

Devuelve

ESP_OK si se guardó correctamente

ESP_ERR_INVALID_ARG si id está fuera de rango o reg es NULL

ESP_FAIL si hubo error al escribir en NVS

Guardar registro en NVS con formato JSON.

Serializa la estructura [reg_t](#) a JSON y la guarda en NVS con clave "reg_<id>".

Proceso:

1. Crea objeto JSON con hour, minute, days
2. Serializa a string con cJSON_Print()
3. Guarda en NVS con nvs_set_str()
4. Commit para escribir a flash

Ejemplo de JSON guardado:

```
{ "hour":14, "minute":30, "days":["L", "X", "V"] }
```

Parámetros

in	<i>id</i>	ID del registro (1-10)
in	<i>reg</i>	Puntero a estructura reg_t a guardar

Devuelve

esp_err_t

Valores devueltos

<i>ESP_OK</i>	Guardado exitoso
<i>ESP_FAIL</i>	Error de NVS

Atención

Sobrescribe registro existente con mismo ID

Nota

Máximo 10 registros soportados (IDs 1-10)

Definición en la línea 137 del archivo [registers.c](#).

Hace referencia a [reg_t::day_count](#), [reg_t::days](#), [reg_t::hour](#), [reg_t::minute](#) y TAG.

Referenciado por [api_post_register\(\)](#).

9.24. registers.h

[Ir a la documentación de este archivo.](#)

```

00001
00025
00026 #pragma once
00027 #include "esp_err.h"
00028 #include "esp_http_server.h"
00029
00036 typedef struct {
00037     int hour;
00038     int minute;
00039     char days[7][12];
00040     int day_count;
00041 } reg_t;
00042
00051 void init_nvs();
00052
00065 esp_err_t save_register_to_nvs(int id, reg_t *reg);
00066
00080 esp_err_t load_register_from_nvs(int id, char *buffer, size_t buffer_len);
00081
00090 void delete_register_from_nvs(int id);
00091
00101 esp_err_t api_post_register(httpd_req_t *req);
00102
00112 esp_err_t api_get_registers(httpd_req_t *req);
00113
00122 esp_err_t api_delete_register(httpd_req_t *req);
00123
00132 void register_registers_endpoints(httpd_handle_t server);

```

9.25. Referencia del archivo sensor_task.c

Implementación de tarea RTOS para lectura thread-safe de sensores.

```

#include "sensor_task.h"
#include "ntc_driver.h"
#include "esp_log.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include <time.h>
#include <sys/time.h>

```

Funciones

- static void [sensor_update_task](#) (void *arg)
Tarea RTOS que actualiza sensores periódicamente.
- esp_err_t [sensor_task_init](#) (adc_oneshot_unit_handle_t handle)
Inicializa el subsistema de sensores y crea la tarea RTOS.
- esp_err_t [sensor_task_get_data](#) (sensor_data_t *data)
Obtiene copia thread-safe de los datos de sensores.
- void [sensor_task_update_now](#) (void)
Solicita actualización inmediata de sensores.

Variables

- static const char * [TAG](#) = "sensor_task"
Tag para logging ESP-IDF.
- static [sensor_data_t](#) [shared_sensor_data](#) = {0}
Datos de sensores compartidos entre tareas.
- static SemaphoreHandle_t [sensor_mutex](#) = NULL
Mutex para protección de [shared_sensor_data](#).
- static adc_oneshot_unit_handle_t [adc_handle](#) = NULL
Handle del ADC usado para leer el termistor NTC.

9.25.1. Descripción detallada

Implementación de tarea RTOS para lectura thread-safe de sensores.

Autor

Jair Hernan Telpis Cuaran, Luis Fernando Gamba Bedoya

Fecha

2025

Este módulo implementa una arquitectura thread-safe para acceso a sensores en FreeRTOS. Soluciona el problema de compartir el handle del ADC entre múltiples tareas mediante:

- Tarea dedicada que lee sensores cada 2 segundos
- Mutex para proteger estructura de datos compartida
- API de acceso segura para otras tareas (HTTP server, display, etc.)

Arquitectura RTOS:

```
[sensor_update_task] --- (cada 2s) ---> [Lee NTC + Time]
      |                               |
      v                               v
xSemaphoreTake()                    Actualiza datos
      |                               |
      v                               v
shared_sensor_data <---- Mutex ----> xSemaphoreGive()
      ^
      |
sensor_task_get_data() <--- Llamado desde otras tareas
```

Ventajas sobre acceso directo:

- No hay race conditions en el ADC handle
- Lectura centralizada cada 2s reduce overhead
- Todas las tareas obtienen datos consistentes
- Cumple con las reglas de RTOS para recursos compartidos

Ver también

[sensor_task.h](#) Para la API pública

[ntc_driver.h](#) Para lectura de temperatura

Definición en el archivo [sensor_task.c](#).

9.25.2. Documentación de funciones

9.25.2.1. sensor_task_get_data()

```
esp_err_t sensor_task_get_data (
    sensor_data_t * data)
```

Obtiene copia thread-safe de los datos de sensores.

Obtener datos de sensores de forma thread-safe.

Implementa lectura segura desde cualquier contexto (tarea o ISR si se usa FromISR).

- Adquiere mutex con timeout de 100ms
- Copia estructura completa (atomic memcpy)
- Libera mutex inmediatamente

Uso desde múltiples tareas: Esta función es completamente thread-safe. Puede ser llamada simultáneamente desde HTTP server, display task, keypad task, etc. El mutex garantiza que nunca se lean datos parcialmente actualizados.

Ejemplo de uso:

```
sensor_data_t data;
if (sensor_task_get_data(&data) == ESP_OK) {
    if (data.temperature_valid) {
        printf("Temp: %.1f C\n", data.temperature_celsius);
    }
    if (data.time_valid) {
        printf("Hora: %02d: %02d\n", data.time_info.tm_hour, data.time_info.tm_min);
    }
}
```

Parámetros

out	data	Puntero a estructura donde se copiarán los datos
-----	------	--

Devuelve

esp_err_t

Valores devueltos

ESP_OK	Datos copiados exitosamente
ESP_ERR_INVALID_ARG	data es NULL
ESP_ERR_INVALID_STATE	sensor_task_init() no ha sido llamado
ESP_ERR_TIMEOUT	No se pudo adquirir mutex en 100ms

Nota

Si retorna error, data se rellena con ceros (memset)

Atención

No llamar desde ISR - usar versión FromISR si existe

Definición en la línea 244 del archivo [sensor_task.c](#).

Hace referencia a [sensor_mutex](#), [shared_sensor_data](#) y [TAG](#).

9.25.2.2. `sensor_task_init()`

```
esp_err_t sensor_task_init (
    adc_one_shot_unit_handle_t handle)
```

Inicializa el subsistema de sensores y crea la tarea RTOS.

Inicializar tarea de sensores.

Secuencia de inicialización:

1. Guarda handle del ADC en variable estática
2. Crea mutex para protección de datos compartidos
3. Crea tarea `sensor_update_task` con parámetros predefinidos

Recursos creados:

- 1 Mutex binario (`sensor_mutex`)
- 1 Tarea RTOS (3072 bytes de stack)

Parámetros

in	<i>handle</i>	Handle del ADC previamente configurado con <code>ntc_driver</code>
----	---------------	--

Devuelve

`esp_err_t`

Valores devueltos

<i>ESP_OK</i>	Inicialización exitosa
<i>ESP_ERR_NO_MEM</i>	No hay memoria para crear mutex
<i>ESP_FAIL</i>	Error al crear tarea RTOS

Atención

Llamar solo una vez durante inicialización del sistema

El handle del ADC debe estar inicializado con [inicializar_ntc\(\)](#)

Ver también

[ntc_driver.h](#) Para inicialización del ADC

Definición en la línea 175 del archivo [sensor_task.c](#).

Hace referencia a [adc_handle](#), [sensor_mutex](#), [sensor_update_task\(\)](#) y [TAG](#).

Referenciado por [sensors_init\(\)](#).

9.25.2.3. sensor_task_update_now()

```
void sensor_task_update_now (
    void )
```

Solicita actualización inmediata de sensores.

Forzar actualización inmediata de sensores.

Nota de implementación: Actualmente no implementado (stub). La tarea actualiza automáticamente cada 2s.

Implementación futura: Podría usar xTaskNotify() para despertar sensor_update_task inmediatamente:

```
static TaskHandle_t sensor_task_handle = NULL;

// En sensor_task_init:
xTaskCreate(..., &sensor_task_handle);

// En sensor_update_task:
ulTaskNotifyTake(pdTRUE, pdMS_TO_TICKS(2000)); // Wait 2s o notificación

// En sensor_task_update_now:
if (sensor_task_handle) {
    xTaskNotifyGive(sensor_task_handle);
}
```

Nota

Por ahora, esperar hasta 2 segundos para nueva lectura automática

Definición en la línea 291 del archivo [sensor_task.c](#).

9.25.2.4. sensor_update_task()

```
void sensor_update_task (
    void * arg) [static]
```

Tarea RTOS que actualiza sensores periódicamente.

Ciclo de ejecución:

1. Lee temperatura del NTC (si adc_handle válido)
2. Obtiene hora actual del sistema (time() + localtime_r())
3. Valida datos (temperatura > -100°C, año > 2000)
4. Actualiza shared_sensor_data con protección de mutex
5. Espera 2 segundos (vTaskDelay)

Parámetros de la tarea:

- Nombre: "sensor_task"
- Stack: 3072 bytes
- Prioridad: configMAX_PRIORITIES - 3 (prioridad media)
- Periodo: 2000ms

Manejo de errores:

- Si ADC handle es NULL: temperature_valid = false
- Si temperatura < -100°C: temperatura inválida (error NTC)
- Si tm_year < 100: hora aún no sincronizada con SNTP
- Si mutex no disponible en 100ms: skip update (sin crash)

Parámetros

<code>in</code>	<code>arg</code>	Parámetro de tarea (no usado)
-----------------	------------------	-------------------------------

Nota

Esta tarea nunca termina (bucle infinito)

Atención

No modificar periodo sin revisar consumo de otros recursos

Definición en la línea 118 del archivo [sensor_task.c](#).

Hace referencia a [adc_handle](#), [leer_temperatura_celsius\(\)](#), [sensor_mutex](#), [shared_sensor_data](#), [TAG](#), [sensor_data_t::temperature_ce](#), [sensor_data_t::temperature_valid](#), [sensor_data_t::time_info](#) y [sensor_data_t::time_valid](#).

Referenciado por [sensor_task_init\(\)](#).

9.25.3. Documentación de variables

9.25.3.1. `adc_handle`

```
adc_one_shot_unit_handle_t adc_handle = NULL [static]
```

Handle del ADC usado para leer el termistor NTC.

Inicializado por [sensor_task_init\(\)](#). Compartido solo dentro de esta tarea (no hay race conditions). Si es NULL, `temperature_valid` se marca como false.

Definición en la línea 87 del archivo [sensor_task.c](#).

Referenciado por [sensor_task_init\(\)](#) y [sensor_update_task\(\)](#).

9.25.3.2. `sensor_mutex`

```
SemaphoreHandle_t sensor_mutex = NULL [static]
```

Mutex para protección de `shared_sensor_data`.

Implementa exclusión mutua para evitar race conditions.

- Toma: `sensor_update_task` (escritura) y `sensor_task_get_data` (lectura)
- Timeout: 100ms para evitar bloqueos prolongados
- Tipo: Mutex binario de FreeRTOS

Definición en la línea 76 del archivo [sensor_task.c](#).

Referenciado por [sensor_task_get_data\(\)](#), [sensor_task_init\(\)](#) y [sensor_update_task\(\)](#).

9.25.3.3. shared_sensor_data

```
sensor_data_t shared_sensor_data = {0} [static]
```

Datos de sensores compartidos entre tareas.

Esta variable contiene la última lectura válida de:

- Temperatura del NTC (en °C)
- Hora actual del sistema (struct tm)
- Flags de validez para cada sensor

IMPORTANTE: Acceso protegido por sensor_mutex. Nunca acceder directamente - usar [sensor_task_get_data\(\)](#).

Definición en la línea 64 del archivo [sensor_task.c](#).

Referenciado por [sensor_task_get_data\(\)](#) y [sensor_update_task\(\)](#).

9.25.3.4. TAG

```
const char* TAG = "sensor_task" [static]
```

Tag para logging ESP-IDF.

Definición en la línea 49 del archivo [sensor_task.c](#).

9.26. sensor_task.c

[Ir a la documentación de este archivo.](#)

```
00001
00037
00038 // sensor_task.c
00039 #include "sensor_task.h"
00040 #include "ntc_driver.h"
00041 #include "esp_log.h"
00042 #include "freertos/FreeRTOS.h"
00043 #include "freertos/task.h"
00044 #include "freertos/semphr.h"
00045 #include <time.h>
00046 #include <sys/time.h>
00047
00049 static const char *TAG = "sensor_task";
00050
00064 static sensor_data_t shared_sensor_data = {0};
00065
00076 static SemaphoreHandle_t sensor_mutex = NULL;
00077
00087 static adc_oneshot_unit_handle_t adc_handle = NULL;
00088
00118 static void sensor_update_task(void* arg) {
00119     ESP_LOGI(TAG, "Sensor task started");
00120
00121     while (1) {
00122         sensor_data_t local_data = {0};
00123
00124         // Leer temperatura
00125         if (adc_handle) {
00126             local_data.temperature_celsius = leer_temperatura_celsius(adc_handle);
00127             local_data.temperature_valid = (local_data.temperature_celsius > -100);
00128         } else {
00129             local_data.temperature_valid = false;
00130         }
00130     }
```

```

00131
00132     // Leer hora actual
00133     time_t now;
00134     time(&now);
00135     localtime_r(&now, &local_data.time_info);
00136     local_data.time_valid = (local_data.time_info.tm_year > (2000 - 1900));
00137
00138     // Actualizar datos compartidos de forma segura
00139     if (xSemaphoreTake(sensor_mutex, pdMS_TO_TICKS(100)) == pdTRUE) {
00140         shared_sensor_data = local_data;
00141         xSemaphoreGive(sensor_mutex);
00142     }
00143
00144     // Actualizar cada 2 segundos
00145     vTaskDelay(pdMS_TO_TICKS(2000));
00146 }
00147 }
00148
00175 esp_err_t sensor_task_init(adc_oneshot_unit_handle_t handle) {
00176     adc_handle = handle;
00177
00178     // Crear mutex
00179     sensor_mutex = xSemaphoreCreateMutex();
00180     if (!sensor_mutex) {
00181         ESP_LOGE(TAG, "Failed to create mutex");
00182         return ESP_ERR_NO_MEM;
00183     }
00184
00185     // Crear tarea de sensores
00186     BaseType_t ret = xTaskCreate(
00187         sensor_update_task,
00188         "sensor_task",
00189         3072,
00190         NULL,
00191         configMAX_PRIORITIES - 3, // Prioridad media
00192         NULL
00193     );
00194
00195     if (ret != pdPASS) {
00196         ESP_LOGE(TAG, "Failed to create sensor task");
00197         vSemaphoreDelete(sensor_mutex);
00198         return ESP_FAIL;
00199     }
00200
00201     ESP_LOGI(TAG, "Sensor task initialized");
00202     return ESP_OK;
00203 }
00204
00244 esp_err_t sensor_task_get_data(sensor_data_t* data) {
00245     if (!data) return ESP_ERR_INVALID_ARG;
00246
00247     // Verificar que el mutex existe antes de usarlo
00248     if (!sensor_mutex) {
00249         ESP_LOGW(TAG, "Sensor task not initialized");
00250         memset(data, 0, sizeof(*data));
00251         return ESP_ERR_INVALID_STATE;
00252     }
00253
00254     if (xSemaphoreTake(sensor_mutex, pdMS_TO_TICKS(100)) == pdTRUE) {
00255         *data = shared_sensor_data;
00256         xSemaphoreGive(sensor_mutex);
00257         return ESP_OK;
00258     }
00259
00260     ESP_LOGW(TAG, "Failed to acquire mutex");
00261     return ESP_ERR_TIMEOUT;
00262 }
00263
00291 void sensor_task_update_now(void) {
00292     // Para implementar: notificar a la tarea para actualización inmediata
00293     // Por ahora, la tarea actualiza cada 2s automáticamente
00294 }

```

9.27. Referencia del archivo sensor_task.h

Tarea dedicada para lectura de sensores con acceso thread-safe mediante mutex.

```

#include "esp_err.h"
#include "esp_adc/adc_oneshot.h"
#include <time.h>

```

Estructuras de datos

- struct `sensor_data_t`
Estructura con datos de sensores protegida por mutex.

Funciones

- `esp_err_t sensor_task_init` (`adc_oneshot_unit_handle_t` `adc_handle`)
Inicializar tarea de sensores.
- `esp_err_t sensor_task_get_data` (`sensor_data_t` *`data`)
Obtener datos de sensores de forma thread-safe.
- void `sensor_task_update_now` (void)
Forzar actualización inmediata de sensores.

9.27.1. Descripción detallada

Tarea dedicada para lectura de sensores con acceso thread-safe mediante mutex.

Autor

Jair Hernan Telpis Cuaran, Luis Fernando Gamba Bedoya

Fecha

2025

Versión

1.0.0

Esta tarea RTOS lee periódicamente los sensores (temperatura NTC y hora del sistema) y almacena los datos en una estructura protegida por mutex, garantizando acceso concurrente seguro desde múltiples tareas (HTTP server, pantalla OLED, etc).

Universidad Nacional de Colombia - Curso RTOS

Definición en el archivo `sensor_task.h`.

9.27.2. Documentación de funciones

9.27.2.1. `sensor_task_get_data()`

```
esp_err_t sensor_task_get_data (  
    sensor_data_t * data)
```

Obtener datos de sensores de forma thread-safe.

Lee los datos de sensores desde la estructura compartida protegida por mutex. Bloquea hasta 100ms esperando el mutex.

Parámetros

out	<i>data</i>	Puntero a estructura donde se copiarán los datos
-----	-------------	--

Devuelve

ESP_OK si los datos fueron leídos correctamente
 ESP_ERR_INVALID_ARG si data es NULL
 ESP_ERR_TIMEOUT si no se pudo adquirir el mutex en 100ms

Atención

Siempre verificar el return value antes de usar los datos

Nota

Esta función es thread-safe y puede llamarse desde cualquier tarea

Obtener datos de sensores de forma thread-safe.

Implementa lectura segura desde cualquier contexto (tarea o ISR si se usa FromISR).

- Adquiere mutex con timeout de 100ms
- Copia estructura completa (atomic memcpy)
- Libera mutex inmediatamente

Uso desde múltiples tareas: Esta función es completamente thread-safe. Puede ser llamada simultáneamente desde HTTP server, display task, keypad task, etc. El mutex garantiza que nunca se lean datos parcialmente actualizados.

Ejemplo de uso:

```
sensor_data_t data;
if (sensor_task_get_data(&data) == ESP_OK) {
    if (data.temperature_valid) {
        printf("Temp: %.1f C\n", data.temperature_celsius);
    }
    if (data.time_valid) {
        printf("Hora: %02d: %02d\n", data.time_info.tm_hour, data.time_info.tm_min);
    }
}
```

Parámetros

out	<i>data</i>	Puntero a estructura donde se copiarán los datos
-----	-------------	--

Devuelve

esp_err_t

Valores devueltos

<code>ESP_OK</code>	Datos copiados exitosamente
<code>ESP_ERR_INVALID_ARG</code>	data es NULL
<code>ESP_ERR_INVALID_STATE</code>	sensor_task_init() no ha sido llamado
<code>ESP_ERR_TIMEOUT</code>	No se pudo adquirir mutex en 100ms

Nota

Si retorna error, data se rellena con ceros (memset)

Atención

No llamar desde ISR - usar versión FromISR si existe

Definición en la línea 244 del archivo [sensor_task.c](#).

Hace referencia a [sensor_mutex](#), [shared_sensor_data](#) y [TAG](#).

9.27.2.2. sensor_task_init()

```
esp_err_t sensor_task_init (
    adc_one_shot_unit_handle_t handle)
```

Inicializar tarea de sensores.

Crea una tarea FreeRTOS que lee sensores cada 2 segundos y actualiza los datos compartidos de forma thread-safe usando un mutex.

Parámetros

<code>adc_handle</code>	Handle del ADC configurado para el sensor NTC
-------------------------	---

Devuelve

ESP_OK si la inicialización fue exitosa
 ESP_ERR_NO_MEM si no hay memoria para crear el mutex o la tarea
 ESP_FAIL si falla la creación de la tarea

Nota

Esta función debe llamarse después de inicializar el ADC y antes de cualquier llamada a [sensor_task_get_data\(\)](#)

Inicializar tarea de sensores.

Secuencia de inicialización:

1. Guarda handle del ADC en variable estática
2. Crea mutex para protección de datos compartidos
3. Crea tarea `sensor_update_task` con parámetros predefinidos

Recursos creados:

- 1 Mutex binario (`sensor_mutex`)
- 1 Tarea RTOS (3072 bytes de stack)

Parámetros

<code>in</code>	<code>handle</code>	Handle del ADC previamente configurado con <code>ntc_driver</code>
-----------------	---------------------	--

Devuelve

`esp_err_t`

Valores devueltos

<code>ESP_OK</code>	Inicialización exitosa
<code>ESP_ERR_NO_MEM</code>	No hay memoria para crear mutex
<code>ESP_FAIL</code>	Error al crear tarea RTOS

Atención

Llamar solo una vez durante inicialización del sistema

El handle del ADC debe estar inicializado con [inicializar_ntc\(\)](#)

Ver también

[ntc_driver.h](#) Para inicialización del ADC

Definición en la línea 175 del archivo [sensor_task.c](#).

Hace referencia a [adc_handle](#), [sensor_mutex](#), [sensor_update_task\(\)](#) y [TAG](#).

Referenciado por [sensors_init\(\)](#).

9.27.2.3. `sensor_task_update_now()`

```
void sensor_task_update_now (
    void )
```

Forzar actualización inmediata de sensores.

Notifica a la tarea de sensores para que realice una lectura inmediata en lugar de esperar el próximo intervalo de 2 segundos.

Nota

Implementación futura - actualmente la tarea actualiza cada 2s automáticamente

Forzar actualización inmediata de sensores.

Nota de implementación: Actualmente no implementado (stub). La tarea actualiza automáticamente cada 2s.

Implementación futura: Podría usar `xTaskNotify()` para despertar `sensor_update_task` inmediatamente:

```
static TaskHandle_t sensor_task_handle = NULL;

// En sensor_task_init:
xTaskCreate(..., &sensor_task_handle);

// En sensor_update_task:
ulTaskNotifyTake(pdTRUE, pdMS_TO_TICKS(2000)); // Wait 2s o notificación

// En sensor_task_update_now:
if (sensor_task_handle) {
    xTaskNotifyGive(sensor_task_handle);
}
```

Nota

Por ahora, esperar hasta 2 segundos para nueva lectura automática

Definición en la línea 291 del archivo [sensor_task.c](#).

9.28. sensor_task.h

[Ir a la documentación de este archivo.](#)

```

00001
00014
00015 #ifndef SENSOR_TASK_H
00016 #define SENSOR_TASK_H
00017
00018 #include "esp_err.h"
00019 #include "esp_adc/adc_oneshot.h"
00020 #include <time.h>
00021
00029 typedef struct {
00030     float temperature_celsius;
00031     struct tm time_info;
00032     bool time_valid;
00033     bool temperature_valid;
00034 } sensor_data_t;
00035
00050 esp_err_t sensor_task_init(adc_oneshot_unit_handle_t adc_handle);
00051
00066 esp_err_t sensor_task_get_data(sensor_data_t* data);
00067
00076 void sensor_task_update_now(void);
00077
00078 #endif // SENSOR_TASK_H

```

9.29. Referencia del archivo webpage/app.js

9.30. app.js

[Ir a la documentación de este archivo.](#)

```

00001 /**
00002  * @file app.js
00003  * @brief Aplicación web JavaScript para control del sistema de cuna inteligente
00004  * @author Jair Hernan Telpis Cuaran, Luis Fernando Gamba Bedoya
00005  * @date 2025
00006  * @version 1.0.0
00007  *
00008  * @details Interfaz web que proporciona:
00009  * - Control de ventilador (manual/auto/registros)
00010  * - Gestión de registros programados (CRUD)
00011  * - Monitoreo de sensores (temperatura NTC, PIR)
00012  * - Configuración WiFi
00013  * - Actualización OTA de firmware
00014  * - Sincronización de fecha/hora
00015  *
00016  * **Endpoints REST consumidos:**
00017  * - GET /dhtSensor.json → Leer temperatura y PIR
00018  * - POST /fanSettings.json → Configurar ventilador
00019  * - POST /regchange.json → Modificar registro
00020  * - POST /regerase.json → Eliminar registro
00021  * - GET /readreg.json → Leer todos los registros
00022  * - POST /OTAupdate → Actualizar firmware
00023  * - POST /wifiConnect.json → Conectar a WiFi
00024  *
00025  * Universidad Nacional de Colombia - Curso RTOS
00026  */
00027
00028 /**
00029  * @var seconds
00030  * @brief Contador de segundos para el temporizador de reinicio OTA
00031  */
00032 var seconds = null;
00033
00034 /**
00035  * @var otaTimerVar
00036  * @brief Handle del temporizador de reinicio tras actualización OTA
00037  */
00038 var otaTimerVar = null;
00039
00040 /**
00041  * @var wifiConnectInterval
00042  * @brief Intervalo para verificar estado de conexión WiFi
00043  */

```

```

00044 var wifiConnectInterval = null;
00045
00046 /**
00047  * @brief Inicialización de la aplicación web
00048  * @details Se ejecuta cuando el DOM está completamente cargado. Inicializa:
00049  * - Intervalos de actualización de sensores (temperatura, PIR)
00050  * - Intervalos de sincronización de fecha/hora
00051  * - Event handlers para botones (animaciones, WiFi, ventilador, registros)
00052  * - Animaciones CSS para feedback visual
00053  */
00054 $(document).ready(function() {
00055     //getUpdateStatus();
00056     startDHTSensorInterval();
00057     startDateTimeInterval();
00058
00059     // Animación genérica para todos los botones .btn
00060     $(".btn").on("click", function() {
00061         const $btn = $(this);
00062
00063         // evita acumular la clase si se spammea el click
00064         $btn.removeClass("is-pressed");
00065
00066         // forza reflow para reiniciar la animación
00067         void this.offsetWidth;
00068
00069         $btn.addClass("is-pressed");
00070
00071         // quita la clase al terminar la animación
00072         setTimeout(function() {
00073             $btn.removeClass("is-pressed");
00074         }, 200);
00075     });
00076
00077     // Lógica específica que ya tenías
00078     $("#connect_wifi").on("click", function(){
00079         checkCredentials();
00080     });
00081 });
00082
00083 /**
00084  * @brief Obtener información del archivo seleccionado para OTA
00085  * @details Lee el archivo seleccionado en el input file y muestra
00086  * su nombre y tamaño en bytes en el elemento HTML #file_info
00087  */
00088 function getFileInfo()
00089 {
00090     var x = document.getElementById("selected_file");
00091     var file = x.files[0];
00092
00093     document.getElementById("file_info").innerHTML = "<h4>File: " + file.name + "<br>" + "Size: " +
file.size + " bytes</h4>";
00094 }
00095
00096 /**
00097  * @brief Ejecutar actualización de firmware via OTA
00098  * @details Envía el archivo .bin seleccionado al ESP32 mediante POST a /OTAupdate.
00099  * Muestra progreso de carga y maneja la respuesta del servidor.
00100  *
00101  * @note El ESP32 debe reiniciarse automáticamente tras aplicar el nuevo firmware
00102  * @warning Solo acepta archivos .bin válidos de ESP-IDF
00103  */
00104 function updateFirmware()
00105 {
00106     // Form Data
00107     var formData = new FormData();
00108     var fileSelect = document.getElementById("selected_file");
00109
00110     if (fileSelect.files && fileSelect.files.length == 1)
00111     {
00112         var file = fileSelect.files[0];
00113         formData.set("file", file, file.name);
00114         document.getElementById("ota_update_status").innerHTML = "Uploading " + file.name + ",
Firmware Update in Progress...";
00115
00116         // Http Request
00117         var request = new XMLHttpRequest();
00118
00119         request.upload.addEventListener("progress", updateProgress);
00120         request.open('POST', "/OTAupdate");
00121         request.responseType = "blob";
00122         request.send(formData);
00123     }
00124     else
00125     {
00126         window.alert('Select A File First')
00127     }
00128 }

```



```

00129
00130 /**
00131  * @brief Actualizar barra de progreso durante carga OTA
00132  * @param oEvent Evento de progreso con información de bytes transferidos
00133  * @details Callback para XMLHttpRequest.upload.addEventListener("progress")
00134  */
00135 function updateProgress(oEvent)
00136 {
00137     if (oEvent.lengthComputable)
00138     {
00139         getUpdateStatus();
00140     }
00141     else
00142     {
00143         window.alert('total size is unknown')
00144     }
00145 }
00146
00147 /**
00148  * @brief Consultar estado de actualización OTA
00149  * @details Realiza POST a /OTAsstatus y procesa la respuesta JSON:
00150  * - ota_update_status = 1: Actualización exitosa, inicia temporizador de reinicio
00151  * - ota_update_status = -1: Error en la actualización
00152  * - ota_update_status = 0: Información sobre firmware actual
00153  *
00154  * @note También actualiza #latest_firmware con fecha/hora de compilación
00155  */
00156 function getUpdateStatus()
00157 {
00158     var xhr = new XMLHttpRequest();
00159     var requestURL = "/OTAsstatus";
00160     xhr.open('POST', requestURL, false);
00161     xhr.send('ota_update_status');
00162
00163     if (xhr.readyState == 4 && xhr.status == 200)
00164     {
00165         var response = JSON.parse(xhr.responseText);
00166
00167         document.getElementById("latest_firmware").innerHTML = response.compile_date + " - " +
response.compile_time
00168
00169         // If flashing was complete it will return a 1, else -1
00170         // A return of 0 is just for information on the Latest Firmware request
00171         if (response.ota_update_status == 1)
00172         {
00173             // Set the countdown timer time
00174             seconds = 10;
00175             // Start the countdown timer
00176             otaRebootTimer();
00177         }
00178         else if (response.ota_update_status == -1)
00179         {
00180             document.getElementById("ota_update_status").innerHTML = "!!! Upload Error !!!";
00181         }
00182     }
00183 }
00184
00185 /**
00186  * @brief Temporizador de cuenta regresiva para reinicio tras OTA
00187  * @details Muestra contador de 10 segundos antes de recargar la página.
00188  * Permite que el ESP32 reinicie y aplique el nuevo firmware.
00189  */
00190 function otaRebootTimer()
00191 {
00192     document.getElementById("ota_update_status").innerHTML = "OTA Firmware Update Complete. This page
will close shortly, Rebooting in: " + seconds;
00193
00194     if (--seconds == 0)
00195     {
00196         clearTimeout(otaTimerVar);
00197         window.location.reload();
00198     }
00199     else
00200     {
00201         otaTimerVar = setTimeout(otaRebootTimer, 1000);
00202     }
00203 }
00204
00205 /**
00206  * Gets DHT22 sensor temperature and humidity values for display on the web page.
00207  */
00208
00209
00210 /**
00211  * @brief Leer valores de registros programados
00212  * @details Realiza GET a /readreg.json y actualiza elementos HTML #reg_1 a #reg_10
00213  * con los datos de cada registro. Muestra "--" si no hay datos.

```

```

00214 *
00215 * @note Usado al cargar la página para mostrar registros existentes
00216 */
00217 function getregValues()
00218 {
00219     $.getJSON('/readreg.json', function(data) {
00220         $("#reg_1").text(data["reg1"] || "--");
00221         $("#reg_2").text(data["reg2"] || "--");
00222         $("#reg_3").text(data["reg3"] || "--");
00223         $("#reg_4").text(data["reg4"] || "--");
00224         $("#reg_5").text(data["reg5"] || "--");
00225         $("#reg_6").text(data["reg6"] || "--");
00226         $("#reg_7").text(data["reg7"] || "--");
00227         $("#reg_8").text(data["reg8"] || "--");
00228         $("#reg_9").text(data["reg9"] || "--");
00229         $("#reg_10").text(data["reg10"] || "--");
00230     }).fail(function(jq, status, err) {
00231         console.error("Error leyendo registros:", status, err);
00232     });
00233 }
00234
00235
00236 /**
00237 * @brief Leer valores de sensores (temperatura NTC y PIR)
00238 * @details Realiza GET a /dhtSensor.json y actualiza:
00239 * - #temperature_reading: Temperatura en °C del sensor NTC
00240 * - #pir_status: Estado del sensor PIR (movimiento detectado / sin movimiento)
00241 *
00242 * Maneja valores nulos y errores de lectura mostrando "--" o "Sin datos"
00243 */
00244 function getDHTSensorValues()
00245 {
00246     $.getJSON('/dhtSensor.json', function(data) {
00247
00248         // ----- Temperatura NTC -----
00249         if (data["temp"] === null || data["temp"] === undefined) {
00250             $("#temperature_reading").text("--");
00251         } else {
00252             // Si viene como número, mostramos con 1 decimal
00253             var tempVal = data["temp"];
00254             if (typeof tempVal === "number") {
00255                 $("#temperature_reading").text(tempVal.toFixed(1) + " °C");
00256             } else {
00257                 $("#temperature_reading").text(tempVal + " °C");
00258             }
00259         }
00260
00261         // ----- Estado PIR -----
00262         if (data["pir"] === 1) {
00263             $("#pir_status").text("Movimiento detectado");
00264         } else {
00265             $("#pir_status").text("Sin movimiento");
00266         }
00267     }).fail(function() {
00268         $("#temperature_reading").text("--");
00269         $("#pir_status").text("Sin datos");
00270     });
00271 }
00272
00273
00274 /**
00275 * @brief Iniciar intervalo de actualización de sensores
00276 * @details Ejecuta getDHTSensorValues() cada 5 segundos para mantener
00277 * actualizada la lectura de temperatura NTC y estado del PIR
00278 */
00279 function startDHTSensorInterval()
00280 {
00281     setInterval(getDHTSensorValues, 5000);
00282 }
00283
00284 /**
00285 * @brief Obtener fecha y hora actual del ESP32
00286 * @details Realiza GET a /time.json y actualiza el elemento #current_datetime.
00287 * Utiliza la hora sincronizada por SNTP del sistema.
00288 */
00289 function getCurrentDateTime()
00290 {
00291     $.getJSON('/time.json', function(data) {
00292         if (data.date && data.time) {
00293             $("#current_datetime").text(data.date + " " + data.time);
00294         } else {
00295             $("#current_datetime").text("--");
00296         }
00297     }).fail(function() {
00298         // Si falla, no revienta la página
00299         $("#current_datetime").text("No time");
00300     });

```

```

00301 }
00302
00303 /**
00304  * @brief Iniciar intervalo de actualización de fecha/hora
00305  * @details Ejecuta getCurrentDateTime() inmediatamente y luego cada 1 segundo
00306  * para mantener sincronizado el reloj en la interfaz web
00307  */
00308 function startDateTimeInterval()
00309 {
00310     // Primera llamada inmediata
00311     getCurrentDateTime();
00312     // Luego actualiza cada segundo (si quieres cada 5 s, cambia 1000 -> 5000)
00313     setInterval(getCurrentDateTime, 1000);
00314 }
00315
00316
00317 /**
00318  * @brief Detener intervalo de verificación de conexión WiFi
00319  * @details Limpia el intervalo wifiConnectInterval para dejar de consultar
00320  * el estado de conexión una vez que se conectó o falló
00321  */
00322 function stopWifiConnectStatusInterval()
00323 {
00324     if (wifiConnectInterval != null)
00325     {
00326         clearInterval(wifiConnectInterval);
00327         wifiConnectInterval = null;
00328     }
00329 }
00330
00331 /**
00332  * @brief Consultar estado de conexión WiFi
00333  * @details Realiza POST a /wifiConnectStatus y actualiza #wifi_connect_status con:
00334  * - Estado 2: Falló la conexión (credenciales incorrectas o incompatibilidad)
00335  * - Estado 3: Conexión exitosa
00336  *
00337  * Detiene el intervalo de consulta cuando se alcanza un estado final
00338  */
00339 function getWifiConnectStatus()
00340 {
00341     var xhr = new XMLHttpRequest();
00342     var requestURL = "/wifiConnectStatus";
00343     xhr.open('POST', requestURL, false);
00344     xhr.send('wifi_connect_status');
00345
00346     if (xhr.readyState == 4 && xhr.status == 200)
00347     {
00348         var response = JSON.parse(xhr.responseText);
00349
00350         document.getElementById("wifi_connect_status").innerHTML = "Connecting...";
00351
00352         if (response.wifi_connect_status == 2)
00353         {
00354             document.getElementById("wifi_connect_status").innerHTML = "<h4 class='rd'>Failed to
Connect. Please check your AP credentials and compatibility</h4>";
00355             stopWifiConnectStatusInterval();
00356         }
00357         else if (response.wifi_connect_status == 3)
00358         {
00359             document.getElementById("wifi_connect_status").innerHTML = "<h4 class='gr'>Connection
Success!</h4>";
00360             stopWifiConnectStatusInterval();
00361         }
00362     }
00363 }
00364
00365 /**
00366  * @brief Iniciar intervalo de verificación de conexión WiFi
00367  * @details Ejecuta getWifiConnectStatus() cada 2.8 segundos durante
00368  * el proceso de conexión para actualizar el estado en tiempo real
00369  */
00370 function startWifiConnectStatusInterval()
00371 {
00372     wifiConnectInterval = setInterval(getWifiConnectStatus, 2800);
00373 }
00374
00375 /**
00376  * @brief Conectar ESP32 a red WiFi
00377  * @details Envía SSID y contraseña al endpoint /wifiConnect.json mediante POST.
00378  * El ESP32 intentará conectarse a la red especificada y guardará las credenciales
00379  * en NVS para reconexión automática tras reinicios.
00380  *
00381  * @note Las credenciales deben ser validadas previamente con checkCredentials()
00382  */
00383 function connectWifi()
00384 {
00385     // Get the SSID and password

```

```

00386     /*selectedSSID = $("#connect_ssid").val();
00387     pwd = $("#connect_pass").val();
00388
00389     $.ajax({
00390         url: '/wifiConnect.json',
00391         dataType: 'json',
00392         method: 'POST',
00393         cache: false,
00394         headers: {'my-connect-ssid': selectedSSID, 'my-connect-pwd': pwd},
00395         data: {'timestamp': Date.now()}
00396     });
00397     */
00398     selectedSSID = $("#connect_ssid").val();
00399     pwd = $("#connect_pass").val();
00400
00401     // Create an object to hold the data to be sent in the request body
00402     var requestData = {
00403         'selectedSSID': selectedSSID,
00404         'pwd': pwd,
00405         'timestamp': Date.now()
00406     };
00407
00408     // Serialize the data object to JSON
00409     var requestDataJSON = JSON.stringify(requestData);
00410
00411     $.ajax({
00412         url: '/wifiConnect.json',
00413         dataType: 'json',
00414         method: 'POST',
00415         cache: false,
00416         data: requestDataJSON, // Send the JSON data in the request body
00417         contentType: 'application/json', // Set the content type to JSON
00418         success: function(response) {
00419             // Handle the success response from the server
00420             console.log(response);
00421         },
00422         error: function(xhr, status, error) {
00423             // Handle errors
00424             console.error(xhr.responseText);
00425         }
00426     });
00427
00428     //startWifiConnectStatusInterval();
00429 }
00430 }
00431
00432 /**
00433  * @brief Validar credenciales WiFi antes de conectar
00434  * @details Verifica que SSID y contraseña no estén vacíos.
00435  * Si las credenciales son válidas, llama a connectWifi().
00436  * Si hay errores, los muestra en #wifi_connect_credentials_errors
00437  */
00438 function checkCredentials()
00439 {
00440     errorList = "";
00441     credsOk = true;
00442
00443     selectedSSID = $("#connect_ssid").val();
00444     pwd = $("#connect_pass").val();
00445
00446     if (selectedSSID == "")
00447     {
00448         errorList += "<h4 class='rd'>SSID cannot be empty!</h4>";
00449         credsOk = false;
00450     }
00451     if (pwd == "")
00452     {
00453         errorList += "<h4 class='rd'>Password cannot be empty!</h4>";
00454         credsOk = false;
00455     }
00456
00457     if (credsOk == false)
00458     {
00459         $("#wifi_connect_credentials_errors").html(errorList);
00460     }
00461     else
00462     {
00463         $("#wifi_connect_credentials_errors").html("");
00464         connectWifi();
00465     }
00466 }
00467
00468 /**
00469  * @brief Alternar visibilidad de la contraseña WiFi
00470  * @details Cambia el tipo del input #connect_pass entre 'password' y 'text'
00471  * para permitir al usuario ver la contraseña que está ingresando
00472  */

```

```

00473 function showPassword()
00474 {
00475     var x = document.getElementById("connect_pass");
00476     if (x.type === "password")
00477     {
00478         x.type = "text";
00479     }
00480     else
00481     {
00482         x.type = "password";
00483     }
00484 }
00485 /**
00486  * @brief Crear o actualizar registro programado
00487  * @details Obtiene datos del formulario (número de registro, hora, minuto, días)
00488  * y los envía al endpoint REST /api/register mediante POST.
00489  *
00490  * Los días se envían en formato abreviado: L, M, X, J, V, S, D
00491  *
00492  * @note El registro se guarda en NVS y persiste tras reinicios
00493  */
00494 function send_register()
00495 {
00496     // Get form values
00497     var selectedNumber = parseInt($("#selectNumber").val());
00498     var hours = parseInt($("#hours").val());
00499     var minutes = parseInt($("#minutes").val());
00500
00501     // Create array for selected days using new format (L, M, X, J, V, S, D)
00502     var selectedDays = [];
00503     if ($("#day_mon").prop("checked")) selectedDays.push("L"); // Lunes
00504     if ($("#day_tue").prop("checked")) selectedDays.push("M"); // Martes
00505     if ($("#day_wed").prop("checked")) selectedDays.push("X"); // Miércoles
00506     if ($("#day_thu").prop("checked")) selectedDays.push("J"); // Jueves
00507     if ($("#day_fri").prop("checked")) selectedDays.push("V"); // Viernes
00508     if ($("#day_sat").prop("checked")) selectedDays.push("S"); // Sábado
00509     if ($("#day_sun").prop("checked")) selectedDays.push("D"); // Domingo
00510
00511     // Create data object for new REST API
00512     var requestData = {
00513         'register': selectedNumber,
00514         'hour': hours,
00515         'minute': minutes,
00516         'days': selectedDays
00517     };
00518
00519     // Send to new REST API endpoint
00520     $.ajax({
00521         url: '/api/register',
00522         dataType: 'json',
00523         method: 'POST',
00524         cache: false,
00525         data: JSON.stringify(requestData),
00526         contentType: 'application/json',
00527         success: function(response) {
00528             console.log('Register saved successfully:', response);
00529             alert('Registro guardado exitosamente');
00530         },
00531         error: function(xhr, status, error) {
00532             console.error('Error saving register:', xhr.responseText);
00533             alert('Error al guardar registro');
00534         }
00535     });
00536 }
00537
00538 /**
00539  * toogle led function.
00540  */
00541 function read_reg()
00542 {
00543
00544
00545     $.ajax({
00546         url: '/readreg.json',
00547         dataType: 'json',
00548         method: 'POST',
00549         cache: false,
00550         //headers: {'my-connect-ssid': selectedSSID, 'my-connect-pwd': pwd},
00551         //data: {'timestamp': Date.now()}
00552     });
00553     // var xhr = new XMLHttpRequest();
00554     // xhr.open("POST", "/toogle_led.json");
00555     // xhr.setRequestHeader("Content-Type", "application/json");
00556     // xhr.send(JSON.stringify({data: "mi información"}));
00557 }
00558 /**
00559  * @brief Eliminar registro programado

```

```

00560 * @details Envía el número de registro seleccionado al endpoint /regerase.json
00561 * mediante POST para eliminarlo permanentemente de NVS
00562 *
00563 * @note El registro eliminado no se puede recuperar
00564 */
00565 function erase_register()
00566 {
00567     // Assuming you have selectedNumber, hours, minutes variables populated from your form
00568     selectedNumber = $("#selectNumber").val();
00569
00570
00571
00572     // Create an object to hold the data to be sent in the request body
00573     var requestData = {
00574         'selectedNumber': selectedNumber,
00575         'timestamp': Date.now()
00576     };
00577
00578     // Serialize the data object to JSON
00579     var requestDataJSON = JSON.stringify(requestData);
00580
00581     $.ajax({
00582         url: '/regerase.json',
00583         dataType: 'json',
00584         method: 'POST',
00585         cache: false,
00586         data: requestDataJSON, // Send the JSON data in the request body
00587         contentType: 'application/json', // Set the content type to JSON
00588         success: function(response) {
00589             // Handle the success response from the server
00590             console.log(response);
00591         },
00592         error: function(xhr, status, error) {
00593             // Handle errors
00594             console.error(xhr.responseText);
00595         }
00596     });
00597
00598     // Print the resulting JSON to the console (for testing)
00599     //console.log(requestDataJSON);
00600 }
00601
00602 function toogle_led()
00603 {
00604     $.ajax({
00605         url: '/toogle_led.json',
00606         dataType: 'json',
00607         method: 'POST',
00608         cache: false,
00609     });
00610
00611 }
00612
00613 function brighthness_up()
00614 {
00615     $.ajax({
00616         url: '/toogle_led.json',
00617         dataType: 'json',
00618         method: 'POST',
00619         cache: false,
00620     });
00621
00622 }
00623
00624
00625 // Actualizar el valor del slider en texto
00626 $(document).ready(function() {
00627     const $range = $("#fan_speed");
00628     const $label = $("#fan_speed_value");
00629
00630     $label.text($range.val() + "%");
00631
00632     $range.on("input change", function() {
00633         $label.text($(this).val() + "%");
00634     });
00635 });
00636
00637 /**
00638 * @brief Aplicar configuración del ventilador
00639 * @details Envía modo de operación y velocidad al endpoint /fanControl.json.
00640 *
00641 * Modos disponibles:
00642 * - "manual": Control directo por slider
00643 * - "auto": Control automático por temperatura NTC
00644 * - "registros": Control por registros programados
00645 *
00646 * @note En modo auto, la velocidad del slider se usa solo como referencia

```

```

00647  * @note En modo registros, la velocidad se aplica cuando un registro coincide
00648  */
00649  function apply_fan_control() {
00650      const mode = $("#fan_mode").val(); // "manual", "auto", "registros"
00651      const speed = parseInt($("#fan_speed").val(), 10);
00652
00653      const payload = JSON.stringify({
00654          mode: mode,
00655          speed: speed
00656      });
00657
00658      $.ajax({
00659          url: "/fanControl.json",
00660          dataType: "json",
00661          method: "POST",
00662          cache: false,
00663          data: payload,
00664          contentType: "application/json",
00665          success: function(resp) {
00666              $("#fan_status").text(resp.status || "Configuración aplicada");
00667          },
00668          error: function(xhr, status, error) {
00669              console.error(xhr.responseText);
00670              $("#fan_status").text("Error al aplicar configuración");
00671          }
00672      });
00673  }
00674
00675
00676
00677
00678
00679
00680
00681
00682
00683
00684
00685
00686
00687
00688
00689
00690
00691
00692
00693
00694
00695
00696
00697
00698

```

9.31. Referencia del archivo webpage/index.html

9.32. index.html

[Ir a la documentación de este archivo.](#)

```

00001 <!DOCTYPE html>
00002 <html lang="es">
00003 <head>
00004     <meta charset="utf-8" />
00005     <meta name="viewport" content="width=device-width, initial-scale=1.0, user-scalable=no">
00006     <meta name="apple-mobile-web-app-capable" content="yes" />
00007
00008     <script src="jquery-3.3.1.min.js"></script>
00009     <link rel="stylesheet" href="app.css">
00010     <script async src="app.js"></script>
00011
00012     <title>ESP32 Control Center</title>
00013 </head>
00014 <body>
00015
00016     <!-- Barra superior fija -->
00017     <div class="top-status">
00018         <div class="top-status__inner">
00019             <div class="top-status__device">
00020                 <span class="top-status__dot"></span>

```

```

00021         <span class="top-status__device-name">ESP32 Control Center</span>
00022     </div>
00023
00024     <div class="top-status__spacer"></div>
00025
00026     <div class="top-status__group">
00027         <span class="top-status__label">Fecha/Hora</span>
00028         <span class="top-status__value" id="current_datetime">--</span>
00029     </div>
00030
00031     <div class="top-status__group">
00032         <span class="top-status__label">Temp</span>
00033         <span class="top-status__value" id="temperature_reading">--</span>
00034     </div>
00035
00036     <div class="top-status__group">
00037         <span class="top-status__label">PIR</span>
00038         <span class="top-status__value" id="pir_status">Sin datos</span>
00039     </div>
00040 </div>
00041 </div>
00042
00043 <div class="page">
00044     <!-- Hero -->
00045     <header class="hero">
00046         <div class="hero__text">
00047             <p class="eyebrow">ESP32 CONTROL</p>
00048
00049             <div class="hero__title-row">
00050                 <h1>Control Center</h1>
00051                 <span class="hero__chip">
00052                     <span class="hero__chip-dot"></span>
00053                     Panel interactivo
00054                 </span>
00055             </div>
00056
00057             <p class="subtitle">
00058                 Administra firmware, Wi-Fi, ventilador y horarios del ESP32 en una sola vista.
00059             </p>
00060         </div>
00061
00062         <div class="hero__badge">
00063             <div class="badge__label" id="latest_firmware_label">Último firmware</div>
00064             <div class="badge__value" id="latest_firmware">--</div>
00065         </div>
00066     </header>
00067
00068     <!-- Contenido principal -->
00069     <main class="main">
00070
00071         <!-- Control ventilador -->
00072         <section class="section" id="ManualControl">
00073             <div class="section__header">
00074                 <span class="section__eyebrow">Ventilador</span>
00075                 <h2 class="section__title">Control y modo de operación</h2>
00076             </div>
00077             <div class="section__body">
00078                 <label class="field">
00079                     <span>Modo de funcionamiento</span>
00080                     <select id="fan_mode" name="fan_mode">
00081                         <option value="manual">Manual (control por barra)</option>
00082                         <option value="auto">Automático por sensores</option>
00083                         <option value="registros">Programado (registros)</option>
00084                     </select>
00085                 </label>
00086
00087                 <label class="field">
00088                     <span>Velocidad del ventilador</span>
00089                     <div class="range-row">
00090                         <input
00091                             type="range"
00092                             id="fan_speed"
00093                             name="fan_speed"
00094                             min="0"
00095                             max="100"
00096                             value="50"
00097                         >
00098                         <span class="range-value" id="fan_speed_value">50 %</span>
00099                     </div>
00100                 </label>
00101
00102                 <div class="actions">
00103                     <button class="btn primary" type="button" onclick="apply_fan_control()">
00104                         Aplicar modo y velocidad
00105                     </button>
00106                 </div>
00107

```



```

00108         <div class="status" id="fan_status"></div>
00109     </div>
00110 </section>
00111
00112 <!-- OTA -->
00113 <section class="section" id="OTA">
00114     <div class="section__header">
00115         <span class="section__eyebrow">OTA</span>
00116         <h2 class="section__title">Actualización de firmware</h2>
00117     </div>
00118     <div class="section__body">
00119         <input type="file" id="selected_file" accept=".bin" onchange="getFileInfo()"
hidden />
00120         <div class="actions">
00121             <button class="btn ghost" type="button"
onclick="document.getElementById('selected_file').click();">
                Seleccionar archivo
00122             </button>
00123             <button class="btn primary" type="button" onclick="updateFirmware()">
                Actualizar ESP32
00124             </button>
00125         </div>
00126     </div>
00127     <div class="status" id="file_info"></div>
00128     <div class="status highlight" id="ota_update_status"></div>
00129 </div>
00130 </section>
00131
00132 <!-- WiFi -->
00133 <section class="section" id="WiFiConnect">
00134     <div class="section__header">
00135         <span class="section__eyebrow">Conectividad</span>
00136         <h2 class="section__title">Conexión Wi-Fi</h2>
00137     </div>
00138     <div class="section__body">
00139         <label class="field">
00140             <span>SSID</span>
00141             <input id="connect_ssid" type="text" maxlength="32" placeholder="Nombre de la
00142 red">
00143         </label>
00144         <label class="field">
00145             <span>Password</span>
00146             <input id="connect_pass" type="password" maxlength="64"
placeholder="Contraseña Wi-Fi">
00147         </label>
00148         <label class="checkbox">
00149             <input type="checkbox" onclick="showPassword()">
00150             <span>Mostrar contraseña</span>
00151         </label>
00152         <div class="actions">
00153             <button class="btn primary" id="connect_wifi" type="button">
                Conectar
00154             </button>
00155             <button class="btn secondary" type="button" onclick="toggle_led()">
                Toggle LED
00156             </button>
00157         </div>
00158         <div class="status error" id="wifi_connect_credentials_errors"></div>
00159         <div class="status" id="wifi_connect_status"></div>
00160     </div>
00161 </section>
00162
00163 <!-- Registros (lectura + programación) -->
00164 <section class="section section--regs" id="Regs">
00165     <div class="section__header">
00166         <span class="section__eyebrow">Registros</span>
00167         <h2 class="section__title">Tiempos configurados y programación</h2>
00168     </div>
00169     <div class="section__body">
00170         <!-- Columna izquierda: lectura -->
00171         <div>
00172             <h3 class="section__title" style="font-size:14px; margin-bottom:6px;">Tiempos
00173 configurados</h3>
00174             <div class="reg-list">
00175                 <div class="reg-row"><span>Registro 1</span><span id="reg_1"
class="reg-value">---</span></div>
00176                 <div class="reg-row"><span>Registro 2</span><span id="reg_2"
class="reg-value">---</span></div>
00177                 <div class="reg-row"><span>Registro 3</span><span id="reg_3"
class="reg-value">---</span></div>
00178                 <div class="reg-row"><span>Registro 4</span><span id="reg_4"
class="reg-value">---</span></div>
00179                 <div class="reg-row"><span>Registro 5</span><span id="reg_5"
class="reg-value">---</span></div>
00180                 <div class="reg-row"><span>Registro 6</span><span id="reg_6"
class="reg-value">---</span></div>
00181                 <div class="reg-row"><span>Registro 7</span><span id="reg_7"
class="reg-value">---</span></div>

```

```

00183         <div class="reg-row"><span>Registro 8</span><span id="reg_8"
class="reg-value">--</span></div>
00184         <div class="reg-row"><span>Registro 9</span><span id="reg_9"
class="reg-value">--</span></div>
00185         <div class="reg-row"><span>Registro 10</span><span id="reg_10"
class="reg-value">--</span></div>
00186     </div>
00187 </div>
00188
00189     <!-- Columna derecha: programación -->
00190     <div>
00191         <h3 class="section_title" style="font-size:14px;
margin-bottom:6px;">Configurar registro</h3>
00192         <div class="section_body" style="gap:8px;">
00193             <label class="field">
00194                 <span>Selecciona el registro</span>
00195                 <select id="selectNumber" name="selectNumber">
00196                     <option value="1">Registro 1</option>
00197                     <option value="2">Registro 2</option>
00198                     <option value="3">Registro 3</option>
00199                     <option value="4">Registro 4</option>
00200                     <option value="5">Registro 5</option>
00201                     <option value="6">Registro 6</option>
00202                     <option value="7">Registro 7</option>
00203                     <option value="8">Registro 8</option>
00204                     <option value="9">Registro 9</option>
00205                     <option value="10">Registro 10</option>
00206                 </select>
00207             </label>
00208
00209             <div class="double">
00210                 <label class="field">
00211                     <span>Hora</span>
00212                     <input type="number" id="hours" name="hours" min="0" max="23"
value="12">
00213                 </label>
00214                 <label class="field">
00215                     <span>Minutos</span>
00216                     <input type="number" id="minutes" name="minutes" min="0" max="59"
value="30">
00217                 </label>
00218             </div>
00219
00220             <div class="days">
00221                 <span class="days_title">Días</span>
00222                 <label class="checkbox"><input type="checkbox" id="day_mon"
name="day_mon" value="Monday" checked><span>Lunes</span></label>
00223                 <label class="checkbox"><input type="checkbox" id="day_tue"
name="day_tue" value="Tuesday"><span>Martes</span></label>
00224                 <label class="checkbox"><input type="checkbox" id="day_wed"
name="day_wed" value="Wednesday"><span>Miércoles</span></label>
00225                 <label class="checkbox"><input type="checkbox" id="day_thu"
name="day_thu" value="Thursday"><span>Jueves</span></label>
00226                 <label class="checkbox"><input type="checkbox" id="day_fri"
name="day_fri" value="Friday"><span>Viernes</span></label>
00227                 <label class="checkbox"><input type="checkbox" id="day_sat"
name="day_sat" value="Saturday"><span>Sábado</span></label>
00228                 <label class="checkbox"><input type="checkbox" id="day_sun"
name="day_sun" value="Sunday"><span>Domingo</span></label>
00229             </div>
00230
00231             <div class="actions">
00232                 <button class="btn primary" type="button"
onclick="send_register()">Enviar registro</button>
00233                 <button class="btn secondary" type="button"
onclick="getregValues()">Leer registros</button>
00234                 <button class="btn ghost" type="button"
onclick="erase_register()">Borrar registro</button>
00235             </div>
00236         </div>
00237     </div>
00238 </div>
00239 </section>
00240
00241 </main>
00242 </div>
00243
00244 <!-- Nota: ajusta tu app.js para el slider y el modo, por ejemplo:
00245 $(document).ready(function(){
00246     const $range = $("#fan_speed");
00247     const $label = $("#fan_speed_value");
00248     $label.text($range.val() + "%");
00249     $range.on("input change", function(){
00250         $label.text($(this).val() + "%");
00251     });
00252 });
00253

```

```

00254     function apply_fan_control() {
00255         const mode = $("#fan_mode").val();
00256         const speed = $("#fan_speed").val();
00257         // Aquí mandas mode y speed al ESP32 por AJAX si quieres
00258         $("#fan_status").text("Modo: " + mode + " | Velocidad: " + speed + "%");
00259     }
00260     -->
00261
00262 </body>
00263 </html>

```

9.33. Referencia del archivo wifi_app.c

Aplicación WiFi con soporte AP/STA dual, reconexión automática y SNTP.

```

#include "freertos/FreeRTOS.h"
#include "freertos/event_groups.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "esp_err.h"
#include "esp_log.h"
#include "esp_wifi.h"
#include "lwip/netdb.h"
#include "nvs.h"
#include "http_server.h"
#include "rgb_led.h"
#include "tasks_common.h"
#include "wifi_app.h"
#include "esp_sntp.h"

```

Funciones

- void [init_obtain_time](#) (void)
Inicializa el estado de sincronización de tiempo.
- bool [get_state_time_was_synchronized](#) (void)
Obtiene el estado de sincronización de tiempo.
- static void [obtain_time](#) (void)
Sincroniza el reloj del sistema con servidor SNTP.
- void [save_wifi_credentials](#) (const char *ssid, const char *password)
Guarda credenciales WiFi en NVS flash.
- void [save_reg_data](#) (uint8_t register_num, char *str)
Guarda datos de registro en NVS (función legacy).
- esp_err_t [read_reg_data](#) (char *str_to_save, uint8_t register_num)
Leer datos de registro desde NVS.
- void [update_register](#) (int reg_to_update)
Actualizar registro específico.
- void [initialize_registers](#) (void)
Inicializar sistema de registros programados.
- void [load_wifi_credentials](#) (char *ssid, char *password)
Carga credenciales WiFi desde NVS.
- bool [nvs_credentials_exist](#) ()
- void [connect_to_wifi](#) (void)
Conectar a WiFi usando credenciales guardadas.

- void [check_sta_connection_state](#) (void *pvParameters)
Tarea que monitorea conexión WiFi STA y reintenta.
- static void [wifi_app_event_handler](#) (void *arg, esp_event_base_t event_base, int32_t event_id, void *event_data)
Event handler principal para eventos WiFi y IP.
- static void [wifi_app_event_handler_init](#) (void)
Initializes the WiFi application event handler for WiFi and IP events.
- static void [wifi_app_default_wifi_init](#) (void)
Initializes the TCP stack and default WiFi configuration.
- static void [wifi_app_soft_ap_config](#) (void)
Configura el Access Point (SoftAP).
- static void [wifi_app_connect_sta](#) (void)
Conecta el ESP32 en modo Station a un AP externo.
- static void [wifi_app_task](#) (void *pvParameters)
Tarea principal de la aplicación WiFi.
- BaseType_t [wifi_app_send_message](#) ([wifi_app_message_e](#) msgID)
Enviar mensaje a la cola de la tarea WiFi.
- wifi_config_t * [wifi_app_get_wifi_config](#) (void)
Obtiene puntero a configuración WiFi STA.
- bool [compare_hour_day_structs](#) (struct tm timeinfo, [register_saved_e](#) aux_reg)
- void [task_compare_hour_to_execute_action](#) (void *pvParameters)
Tarea que ejecuta registros programados basados en hora SNTP.
- void [wifi_app_start](#) (void)
Inicializa y arranca la aplicación WiFi completa.

Variables

- static const char [TAG](#) [] = "wifi_app"
Tag para logging ESP-IDF.
- SemaphoreHandle_t [mySemaphore](#)
Semáforo para sincronización entre tareas.
- [register_saved_e](#) [register_readings_from_flash](#) [10]
Array de registros guardados desde flash (legacy).
- wifi_config_t * [wifi_config](#) = NULL
Configuración WiFi actual (NULL si no inicializado).
- static int [g_retry_number](#)
Contador de reintentos de conexión WiFi.
- static QueueHandle_t [wifi_app_queue_handle](#)
Cola para mensajes de la aplicación WiFi.
- esp_netif_t * [esp_netif_sta](#) = NULL
Interfaz de red para modo Station.
- esp_netif_t * [esp_netif_ap](#) = NULL
Interfaz de red para modo Access Point.
- bool [time_was_synchronized](#)
Flag que indica si el tiempo fue sincronizado vía SNTP.
- uint8_t [s_led_state](#)
Estado del LED RGB (externo).

9.33.1. Descripción detallada

Aplicación WiFi con soporte AP/STA dual, reconexión automática y SNTP.

Autor

Jair Hernan Telpis Cuaran, Luis Fernando Gamba Bedoya

Fecha

2025

Este módulo implementa la gestión completa de WiFi para el ESP32:

Modos de operación:

1. **Access Point (AP):** Para configuración inicial
 - SSID: Configurable
 - Usuario puede conectarse para configurar credenciales
2. **Station (STA):** Conexión a red WiFi existente
 - Credenciales almacenadas en NVS
 - Reconexión automática tras desconexión
 - Máximo de reintentos configurable
3. **Dual AP+STA:** Permite configuración mientras mantiene conexión

Características:

- Sincronización de tiempo mediante SNTP (pool.ntp.org)
- Persistencia de credenciales en NVS flash
- Event handlers para WiFi (connect, disconnect, got IP)
- Cola de mensajes para comunicación entre tareas
- LED RGB indicador de estado
- Notificación al HTTP server de cambios de estado

Arquitectura RTOS:

```
[WiFi Event Handler] ---> [wifi_app_queue] ---> [wifi_app_task]
      |                               |
      v                               v
IP_EVENT_STA_GOT_IP -----> http_server_monitor_send_message()
      |                               |
      v                               v
obtain_time() -> SNTP sync          LED color update
```

Ver también

[wifi_app.h](#) Para la API pública

[http_server.c](#) Para integración con servidor web

Definición en el archivo [wifi_app.c](#).

9.33.2. Documentación de funciones

9.33.2.1. `check_sta_connection_state()`

```
void check_sta_connection_state (
    void * pvParameters)
```

Tarea que monitorea conexión WiFi STA y reintenta.

Tarea que verifica estado de conexión STA.

Cada 30 segundos verifica si está conectado a AP. Si pierde conexión, intenta reconectar automáticamente.

Funcionalidad:

- Llama `esp_wifi_sta_get_ap_info()` cada 30s
- Si falla (`ESP_ERR_WIFI_NOT_CONNECT`), intenta conectar
- Registra RSSI si está conectado

Parámetros

in	<i>pvParameters</i>	Parámetro de tarea (no usado)
----	---------------------	-------------------------------

Nota

Tarea infinita creada en [wifi_app_start\(\)](#)

Definición en la línea 611 del archivo [wifi_app.c](#).

Hace referencia a [connect_to_wifi\(\)](#), [get_state_time_was_synchronized\(\)](#), [nvs_credentials_exist\(\)](#), [obtain_time\(\)](#) y [TAG](#).

Referenciado por [wifi_app_task\(\)](#).

9.33.2.2. `compare_hour_day_structs()`

```
bool compare_hour_day_structs (
    struct tm timeinfo,
    register_saved_e aux_reg)
```

Definición en la línea 967 del archivo [wifi_app.c](#).

Hace referencia a [toogle_led\(\)](#).

Referenciado por [task_compare_hour_to_execute_action\(\)](#).

9.33.2.3. `connect_to_wifi()`

```
void connect_to_wifi (
    void )
```

Conectar a WiFi usando credenciales guardadas.

Lee credenciales de NVS e intenta conexión STA

Definición en la línea 569 del archivo `wifi_app.c`.

Hace referencia a `load_wifi_credentials()`, `mySemaphore`, `TAG`, `wifi_app_connect_sta()`, `wifi_app_get_wifi_config()` y `wifi_config`.

Referenciado por `check_sta_connection_state()` y `http_server_wifi_connect_json_handler()`.

9.33.2.4. `get_state_time_was_synchronized()`

```
bool get_state_time_was_synchronized (
    void )
```

Obtiene el estado de sincronización de tiempo.

Verificar si la hora ya fue sincronizada.

Devuelve

`bool`

Valores devueltos

<i>true</i>	Tiempo sincronizado correctamente con SNTP
<i>false</i>	Aún no sincronizado o fallo

Definición en la línea 124 del archivo `wifi_app.c`.

Hace referencia a `time_was_synchronized`.

Referenciado por `check_sta_connection_state()` y `task_compare_hour_to_execute_action()`.

9.33.2.5. `init_obtain_time()`

```
void init_obtain_time (
    void )
```

Inicializa el estado de sincronización de tiempo.

Inicializar sincronización de hora SNTP.

Marca el tiempo como NO sincronizado. Llamar antes de intentar sincronizar con SNTP.

Definición en la línea 113 del archivo `wifi_app.c`.

Hace referencia a `time_was_synchronized`.

9.33.2.6. initialize_registers()

```
void initialize_registers (
    void )
```

Inicializar sistema de registros programados.

Carga registros desde NVS al arranque < Número máximo de registros programados

Definición en la línea 431 del archivo [wifi_app.c](#).

Hace referencia a [NUM_REGISTERS_AV](#), [read_reg_data\(\)](#) y [register_readings_from_flash](#).

Referenciado por [wifi_app_task\(\)](#).

9.33.2.7. load_wifi_credentials()

```
void load_wifi_credentials (
    char * ssid,
    char * password)
```

Carga credenciales WiFi desde NVS.

Cargar credenciales WiFi desde NVS.

Lee SSID y password almacenados previamente en NVS.

Namespace NVS: "storage" Keys:

- "wifi_ssid": SSID de la red
- "wifi_password": Contraseña

Parámetros

out	<i>ssid</i>	Buffer donde se copia el SSID (mínimo 33 bytes)
out	<i>password</i>	Buffer donde se copia el password (mínimo 65 bytes)

Nota

Llama `ESP_ERROR_CHECK` - programa panic si no existen credenciales

Atención

Validar existencia con [nvs_credentials_exist\(\)](#) antes de llamar

Definición en la línea 508 del archivo [wifi_app.c](#).

Hace referencia a [TAG](#).

Referenciado por [connect_to_wifi\(\)](#).

9.33.2.8. `nvs_credentials_exist()`

```
bool nvs_credentials_exist ()
```

Definición en la línea 548 del archivo `wifi_app.c`.

Referenciado por `check_sta_connection_state()`.

9.33.2.9. `obtain_time()`

```
void obtain_time (
    void ) [static]
```

Sincroniza el reloj del sistema con servidor SNTP.

Configura y ejecuta sincronización de tiempo mediante SNTP (Simple Network Time Protocol).

Proceso:

1. Configura zona horaria EST5EDT (Eastern Time con DST)
2. Detiene SNTP si ya está corriendo
3. Configura servidor NTP (0.co.pool.ntp.org)
4. Espera hasta 20 segundos (10 reintentos x 2s) por sincronización
5. Valida que `tm_year > 2016`

Zona horaria:

- EST5EDT: Eastern Standard Time (-5h UTC)
- M3.2.0/2: DST inicia 2do domingo de marzo a las 2am
- M11.1.0: DST termina 1er domingo de noviembre

Nota

Requiere conexión WiFi activa

Atención

Bloquea hasta 20 segundos esperando sincronización

Definición en la línea 149 del archivo `wifi_app.c`.

Hace referencia a `TAG` y `time_was_synchronized`.

Referenciado por `check_sta_connection_state()`.

9.33.2.10. `read_reg_data()`

```
esp_err_t read_reg_data (
    char * str_to_save,
    uint8_t register_num)
```

Leer datos de registro desde NVS.

Parámetros

out	<i>str_to_save</i>	Buffer donde se escribirá el registro (JSON)
	<i>register_num</i>	Número de registro (1-10)

Devuelve

ESP_OK si se leyó correctamente

Definición en la línea 279 del archivo [wifi_app.c](#).

Hace referencia a [TAG](#).

Referenciado por [initialize_registers\(\)](#) y [update_register\(\)](#).

9.33.2.11. save_reg_data()

```
void save_reg_data (
    uint8_t register_num,
    char * str)
```

Guarda datos de registro en NVS (función legacy).

Guardar registro en NVS.

Función antigua para guardar registros (1-10). **DEPRECADA:** Usar [registers.c](#) (save_register_to_nvs) en su lugar.

Parámetros

in	<i>register_num</i>	Número de registro (1-10)
in	<i>str</i>	Datos del registro como string

Obsoleto Usar API moderna en [registers.c](#)

Nota

Solo para compatibilidad con código antiguo

Definición en la línea 230 del archivo [wifi_app.c](#).

9.33.2.12. save_wifi_credentials()

```
void save_wifi_credentials (
    const char * ssid,
    const char * password)
```

Guarda credenciales WiFi en NVS flash.

Guardar credenciales WiFi en NVS.

Almacena SSID y contraseña en memoria no volátil. Sobrevive a reinicios del ESP32.

Claves NVS:

- "wifi_ssid": Nombre de la red WiFi
- "wifi_password": Contraseña

Parámetros

in	<i>ssid</i>	SSID de la red WiFi (máx 32 caracteres)
in	<i>password</i>	Contraseña (máx 64 caracteres)

Atención

ESP_ERROR_CHECK causa panic si falla NVS

Nota

Namespace "storage" debe existir

Definición en la línea 208 del archivo `wifi_app.c`.

Referenciado por `http_server_wifi_connect_json_handler()`.

9.33.2.13. task_compare_hour_to_execute_action()

```
void task_compare_hour_to_execute_action (
    void * pvParameters)
```

Tarea que ejecuta registros programados basados en hora SNTP.

Tarea que compara hora actual con registros programados.

Espera a que SNTP sincronice, luego cada 60s compara hora actual con registros programados. Si coincide, ejecuta acción (ventilador).

Proceso:

1. Espera a `get_state_time_was_synchronized() == true`
2. Cada 60s obtiene hora con `time()` y `localtime_r()`
3. Itera registros y compara con `compare_hour_day_structs()`
4. Si coincide, ejecuta acción del registro

Parámetros

in	<i>pvParameters</i>	Parámetro de tarea (no usado)
----	---------------------	-------------------------------

Nota

Tarea infinita, stack 4096 bytes, prioridad 5

Ver también

`compare_hour_day_structs()` Comparación hora/día

< Número máximo de registros programados

Definición en la línea 1058 del archivo `wifi_app.c`.

Hace referencia a `compare_hour_day_structs()`, `get_state_time_was_synchronized()`, `NUM_REGISTERS_AV`, `register_readings_from_flash` y `TAG`.

Referenciado por `wifi_app_start()`.

9.33.2.14. `update_register()`

```
void update_register (
    int reg_to_update)
```

Actualizar registro específico.

Parámetros

<code>reg_to_update</code>	Número de registro a actualizar (1-10)
----------------------------	--

Definición en la línea [387](#) del archivo `wifi_app.c`.

Hace referencia a [read_reg_data\(\)](#), [register_readings_from_flash](#) y [TAG](#).

9.33.2.15. `wifi_app_connect_sta()`

```
void wifi_app_connect_sta (
    void ) [static]
```

Conecta el ESP32 en modo Station a un AP externo.

Utiliza las credenciales de [wifi_app_get_wifi_config\(\)](#). Si la conexión falla, `wifi_app_event_handler` reintenta.

Nota

Llama `esp_wifi_set_config()` y `esp_wifi_connect()`

Definición en la línea [836](#) del archivo `wifi_app.c`.

Hace referencia a [wifi_app_get_wifi_config\(\)](#).

Referenciado por [connect_to_wifi\(\)](#) y [wifi_app_task\(\)](#).

9.33.2.16. `wifi_app_default_wifi_init()`

```
void wifi_app_default_wifi_init (
    void ) [static]
```

Initializes the TCP stack and default WiFi configuration.

Definición en la línea [758](#) del archivo `wifi_app.c`.

Hace referencia a [esp_netif_ap](#) y [esp_netif_sta](#).

Referenciado por [wifi_app_task\(\)](#).

9.33.2.17. `wifi_app_event_handler()`

```
void wifi_app_event_handler (
    void * arg,
    esp_event_base_t event_base,
    int32_t event_id,
    void * event_data) [static]
```

Event handler principal para eventos WiFi y IP.

Procesa todos los eventos WiFi del ESP32:

Eventos WiFi:

- `WIFI_EVENT_AP_STACONNECTED`: Cliente conectado al AP
- `WIFI_EVENT_AP_STADISCONNECTED`: Cliente desconectado del AP
- `WIFI_EVENT_STA_START`: Station iniciado → intenta conectar
- `WIFI_EVENT_STA_CONNECTED`: Conectado a AP externo
- `WIFI_EVENT_STA_DISCONNECTED`: Desconectado → reintentar

Eventos IP:

- `IP_EVENT_STA_GOT_IP`: IP asignada → notifica HTTP server, inicia SNTP

Reconexión automática: Máximo `WIFI_RETRY_ATTEMPTS` reintentos antes de fallar.

Parámetros

in	<i>arg</i>	Parámetro de usuario (no usado)
in	<i>event_base</i>	Base del evento (<code>WIFI_EVENT</code> o <code>IP_EVENT</code>)
in	<i>event_id</i>	ID específico del evento
in	<i>event_data</i>	Datos del evento (IP, MAC, etc)

Definición en la línea 676 del archivo `wifi_app.c`.

Hace referencia a `TAG`, `WIFI_APP_MSG_STA_CONNECTED_GOT_IP` y `wifi_app_send_message()`.

Referenciado por `wifi_app_event_handler_init()`.

9.33.2.18. `wifi_app_event_handler_init()`

```
void wifi_app_event_handler_init (
    void ) [static]
```

Initializes the WiFi application event handler for WiFi and IP events.

Definición en la línea 743 del archivo `wifi_app.c`.

Hace referencia a `wifi_app_event_handler()`.

Referenciado por `wifi_app_task()`.

9.33.2.19. wifi_app_get_wifi_config()

```
wifi_config_t * wifi_app_get_wifi_config (
    void )
```

Obtiene puntero a configuración WiFi STA.

Obtener configuración actual de WiFi.

Devuelve

wifi_config_t* Puntero a wifi_config global

Nota

Contiene SSID y password cargados de NVS

Definición en la línea 962 del archivo [wifi_app.c](#).

Hace referencia a [wifi_config](#).

Referenciado por [connect_to_wifi\(\)](#), [http_server_wifi_connect_json_handler\(\)](#) y [wifi_app_connect_sta\(\)](#).

9.33.2.20. wifi_app_send_message()

```
BaseType_t wifi_app_send_message (
    wifi_app_message_e msgID)
```

Enviar mensaje a la cola de la tarea WiFi.

Parámetros

<i>msgID</i>	ID del mensaje desde wifi_app_message_e
--------------	---

Devuelve

pdTRUE si se envió correctamente, pdFALSE si la cola está llena

Nota

Thread-safe, puede llamarse desde cualquier tarea

Definición en la línea 948 del archivo [wifi_app.c](#).

Hace referencia a [wifi_app_queue_handle](#).

Referenciado por [wifi_app_event_handler\(\)](#) y [wifi_app_task\(\)](#).

9.33.2.21. wifi_app_soft_ap_config()

```
void wifi_app_soft_ap_config (
    void ) [static]
```

Configura el Access Point (SoftAP).

Establece los parámetros del AP:

Configuración AP:

- SSID: WIFI_AP_SSID (definido en [wifi_app.h](#))
- Password: WIFI_AP_PASSWORD
- Canal: WIFI_AP_CHANNEL
- SSID oculto: No
- Máx conexiones: WIFI_AP_MAX_CONNECTIONS
- Tipo auth: WPA/WPA2

IP estática:

- IP: 192.168.0.1
- Gateway: 192.168.0.1
- Netmask: 255.255.255.0

Nota

Llama esp_netif_dhcps_stop() y esp_netif_dhcps_start()

< Nombre de la red AP

< Nombre de la red AP

< Contraseña del AP (mínimo 8 caracteres)

< Canal WiFi del AP (1-13)

< 0=visible, 1=oculto

< Máximo de clientes simultáneos

< Intervalo beacon en ms (recomendado 100ms)

must call this first

< IP del AP

Assign access point's static IP, GW, and netmask

< Gateway del AP (igual que IP)

< Máscara de subred

Statically configure the network interface

Start the AP DHCP server (for connecting stations e.g. your mobile device)

Setting the mode as Access Point / Station Mode

Set our configuration

< Ancho de banda 20 MHz

Our default bandwidth 20 MHz

< Sin ahorro de energía

Power save set to "NONE"

Definición en la línea 792 del archivo [wifi_app.c](#).

Hace referencia a [esp_netif_ap](#), [WIFI_AP_BANDWIDTH](#), [WIFI_AP_BEACON_INTERVAL](#), [WIFI_AP_CHANNEL](#), [WIFI_AP_GATEWAY](#), [WIFI_AP_IP](#), [WIFI_AP_MAX_CONNECTIONS](#), [WIFI_AP_NETMASK](#), [WIFI_AP_PASSWORD](#), [WIFI_AP_SSID](#), [WIFI_AP_SSID_HIDDEN](#) y [WIFI_STA_POWER_SAVE](#).

Referenciado por [wifi_app_task\(\)](#).

9.33.2.22. [wifi_app_start\(\)](#)

```
void wifi_app_start (  
    void )
```

Inicializa y arranca la aplicación WiFi completa.

Iniciar tarea RTOS de WiFi.

Punto de entrada principal para WiFi. Secuencia:

1. Inicializa netif TCP/IP
2. Configura event handlers WiFi
3. Inicializa WiFi con config por defecto
4. Configura Access Point
5. Crea cola de mensajes
6. Crea tarea [wifi_app_task](#)
7. Crea tarea [check_sta_connection_state](#) (reconexión)

Nota

Llamar una sola vez durante inicialización del sistema

Atención

Requiere NVS inicializado previamente

Ver también

[wifi_app_task\(\)](#) Tarea principal creada

[check_sta_connection_state\(\)](#) Tarea de reconexión

Definición en la línea 1115 del archivo `wifi_app.c`.

Hace referencia a [mySemaphore](#), [TAG](#), [task_compare_hour_to_execute_action\(\)](#), [wifi_app_queue_handle](#), [wifi_app_task\(\)](#) y [wifi_config](#).

9.33.2.23. wifi_app_task()

```
void wifi_app_task (
    void * pvParameters) [static]
```

Tarea principal de la aplicación WiFi.

Tarea RTOS que procesa mensajes de la cola `wifi_app_queue_handle`.

Mensajes soportados:

- `WIFI_APP_MSG_START_HTTP_SERVER`: Inicia servidor HTTP
- `WIFI_APP_MSG_CONNECTING_FROM_HTTP_SERVER`: Intenta conexión STA
- `WIFI_APP_MSG_STA_CONNECTED_GOT_IP`: IP obtenida exitosamente

Parámetros tarea:

- Stack: `WIFI_APP_TASK_STACK_SIZE`
- Prioridad: `WIFI_APP_TASK_PRIORITY`
- Core: `WIFI_APP_TASK_CORE_ID`

Parámetros

in	<i>pvParameters</i>	Parámetro de tarea (no usado)
----	---------------------	-------------------------------

Nota

Tarea infinita que nunca termina

Definición en la línea 862 del archivo `wifi_app.c`.

Hace referencia a [check_sta_connection_state\(\)](#), [g_retry_number](#), [HTTP_MSG_WIFI_CONNECT_FAIL](#), [HTTP_MSG_WIFI_CONNECT_INIT](#), [HTTP_MSG_WIFI_CONNECT_SUCCESS](#), [http_server_monitor_send_message\(\)](#), [http_server_start\(\)](#), [initialize_registers\(\)](#), [TAG](#), [wifi_app_connect_sta\(\)](#), [WIFI_APP_CONNECT_TO_STA](#), [wifi_app_default_wifi_init\(\)](#), [wifi_app_event_handler_init\(\)](#), [WIFI_APP_MSG_CONNECTING_FROM_HTTP_SERVER](#), [WIFI_APP_MSG_STA_CONNECTED_GOT_IP](#), [WIFI_APP_MSG_STA_DISCONNECTED](#), [WIFI_APP_MSG_START_HTTP_SERVER](#), [wifi_app_queue_handle](#), [wifi_app_send_message\(\)](#) y [wifi_app_soft_ap_config\(\)](#).

Referenciado por [wifi_app_start\(\)](#).

9.33.3. Documentación de variables

9.33.3.1. esp_netif_ap

```
esp_netif_t* esp_netif_ap = NULL
```

Interfaz de red para modo Access Point.

Handle de interfaz de red Access Point.

Definición en la línea 98 del archivo [wifi_app.c](#).

Referenciado por [wifi_app_default_wifi_init\(\)](#) y [wifi_app_soft_ap_config\(\)](#).

9.33.3.2. esp_netif_sta

```
esp_netif_t* esp_netif_sta = NULL
```

Interfaz de red para modo Station.

Handle de interfaz de red Station.

Definición en la línea 95 del archivo [wifi_app.c](#).

Referenciado por [wifi_app_default_wifi_init\(\)](#).

9.33.3.3. g_retry_number

```
int g_retry_number [static]
```

Contador de reintentos de conexión WiFi.

Definición en la línea 89 del archivo [wifi_app.c](#).

Referenciado por [wifi_app_task\(\)](#).

9.33.3.4. mySemaphore

```
SemaphoreHandle_t mySemaphore
```

Semáforo para sincronización entre tareas.

Definición en la línea 73 del archivo [wifi_app.c](#).

Referenciado por [connect_to_wifi\(\)](#) y [wifi_app_start\(\)](#).

9.33.3.5. `register_readings_from_flash`

```
register_saved_e register_readings_from_flash[10]
```

Array de registros guardados desde flash (legacy).

Definición en la línea 76 del archivo `wifi_app.c`.

Referenciado por `initialize_registers()`, `task_compare_hour_to_execute_action()` y `update_register()`.

9.33.3.6. `s_led_state`

```
uint8_t s_led_state [extern]
```

Estado del LED RGB (externo).

Estado del LED RGB (externo).

Definición en la línea 192 del archivo `http_server.c`.

Referenciado por `toggle_led()`.

9.33.3.7. `TAG`

```
const char TAG[] = "wifi_app" [static]
```

Tag para logging ESP-IDF.

Definición en la línea 70 del archivo `wifi_app.c`.

9.33.3.8. `time_was_synchronized`

```
bool time_was_synchronized
```

Flag que indica si el tiempo fue sincronizado vía SNTP.

Definición en la línea 101 del archivo `wifi_app.c`.

Referenciado por `get_state_time_was_synchronized()`, `init_obtain_time()` y `obtain_time()`.

9.33.3.9. `wifi_app_queue_handle`

```
QueueHandle_t wifi_app_queue_handle [static]
```

Cola para mensajes de la aplicación WiFi.

Definición en la línea 92 del archivo `wifi_app.c`.

Referenciado por `wifi_app_send_message()`, `wifi_app_start()` y `wifi_app_task()`.

9.33.3.10. wifi_config

```
wifi_config_t* wifi_config = NULL
```

Configuración WiFi actual (NULL si no inicializado).

Almacena SSID y contraseña para modo Station. Asignado dinámicamente durante inicialización.

Definición en la línea 86 del archivo `wifi_app.c`.

Referenciado por `connect_to_wifi()`, `http_server_wifi_connect_json_handler()`, `wifi_app_get_wifi_config()` y `wifi_app_start()`.

9.34. wifi_app.c

[Ir a la documentación de este archivo.](#)

```
00001
00044
00045 /*
00046  * wifi_app.c
00047  *
00048  * Created on: Oct 17, 2021
00049  * Author: kjaqu
00050  */
00051
00052 #include "freertos/FreeRTOS.h"
00053 #include "freertos/event_groups.h"
00054 #include "freertos/task.h"
00055 #include "freertos/semphr.h"
00056
00057 #include "esp_err.h"
00058 #include "esp_log.h"
00059 #include "esp_wifi.h"
00060 #include "lwip/netdb.h"
00061 #include "nvs.h"
00062
00063 #include "http_server.h"
00064 #include "rgb_led.h"
00065 #include "tasks_common.h"
00066 #include "wifi_app.h"
00067 #include "esp_sntp.h"
00068
00070 static const char TAG [] = "wifi_app";
00071
00073 SemaphoreHandle_t mySemaphore;
00074
00076 register_saved_e register_readings_from_flash [NUM_REGISTERS_AV];
00077
00086 wifi_config_t *wifi_config = NULL;
00087
00089 static int g_retry_number;
00090
00092 static QueueHandle_t wifi_app_queue_handle;
00093
00095 esp_netif_t* esp_netif_sta = NULL;
00096
00098 esp_netif_t* esp_netif_ap = NULL;
00099
00101 bool time_was_synchronized;
00102
00104 extern uint8_t s_led_state;
00105
00113 void init_obtain_time( void ){
00114     time_was_synchronized = false;
00115 }
00116
00124 bool get_state_time_was_synchronized( void ){
00125     return time_was_synchronized;
00126 }
00127
00149 static void obtain_time(void)
00150 {
00151     setenv("TZ", "EST5EDT,M3.2.0/2,M11.1.0", 1);
00152     tzset();
00153     ESP_LOGI(TAG, "Initializing SNTP");
```

```

00154
00155 // Stop SNTP if already running to avoid assert
00156 if (esp_sntp_enabled()) {
00157     ESP_LOGI(TAG, "SNTP already running, stopping first");
00158     esp_sntp_stop();
00159 }
00160
00161 // Configurar el servidor SNTP. Aquí se utiliza "pool.ntp.org" como ejemplo. Puedes cambiarlo
según tus necesidades.
00162 esp_sntp_setoperatingmode(SNTP_OPMODE_POLL);
00163 esp_sntp_setservername(0, "0.co.pool.ntp.org");
00164 esp_sntp_init();
00165
00166 // Esperar a que se sincronice el tiempo con el servidor SNTP
00167 time_t now = 0;
00168 struct tm timeinfo = {0};
00169 int retry = 0;
00170 const int retry_count = 10;
00171
00172 while (timeinfo.tm_year < (2016 - 1900) && ++retry < retry_count)
00173 {
00174     ESP_LOGI(TAG, "Waiting for system time to be set... (%d/%d)", retry, retry_count);
00175     vTaskDelay(2000 / portTICK_PERIOD_MS);
00176     time(&now);
00177     localtime_r(&now, &timeinfo);
00178 }
00179
00180 if (retry < retry_count)
00181 {
00182     ESP_LOGI(TAG, "System time is set!");
00183     time_was_synchronized = true;
00184 }
00185 else
00186 {
00187     ESP_LOGE(TAG, "Unable to set system time. Check your SNTP configuration.");
00188 }
00189 }
00190
00208 void save_wifi_credentials(const char *ssid, const char *password) {
00209     nvs_handle_t nvs_handle;
00210     ESP_ERROR_CHECK(nvs_open("storage", NVS_READWRITE, &nvs_handle));
00211     ESP_ERROR_CHECK(nvs_set_str(nvs_handle, "wifi_ssid", ssid));
00212     ESP_ERROR_CHECK(nvs_set_str(nvs_handle, "wifi_password", password));
00213     ESP_ERROR_CHECK(nvs_commit(nvs_handle));
00214     nvs_close(nvs_handle);
00215 }
00216
00230 void save_reg_data(uint8_t register_num, char *str) {
00231     nvs_handle_t nvs_handle;
00232     ESP_ERROR_CHECK(nvs_open("storage", NVS_READWRITE, &nvs_handle));
00233
00234     if (register_num == 1){
00235         ESP_ERROR_CHECK(nvs_set_str(nvs_handle, "reg01", str));
00236     }
00237     else if (register_num == 2){
00238         ESP_ERROR_CHECK(nvs_set_str(nvs_handle, "reg02", str));
00239     }
00240     else if (register_num == 3){
00241         ESP_ERROR_CHECK(nvs_set_str(nvs_handle, "reg03", str));
00242     }
00243     else if (register_num == 4){
00244         ESP_ERROR_CHECK(nvs_set_str(nvs_handle, "reg04", str));
00245     }
00246     else if (register_num == 5){
00247         ESP_ERROR_CHECK(nvs_set_str(nvs_handle, "reg05", str));
00248     }
00249     else if (register_num == 6){
00250         ESP_ERROR_CHECK(nvs_set_str(nvs_handle, "reg06", str));
00251     }
00252     else if (register_num == 7){
00253         ESP_ERROR_CHECK(nvs_set_str(nvs_handle, "reg07", str));
00254     }
00255     else if (register_num == 8){
00256         ESP_ERROR_CHECK(nvs_set_str(nvs_handle, "reg08", str));
00257     }
00258     else if (register_num == 9){
00259         ESP_ERROR_CHECK(nvs_set_str(nvs_handle, "reg09", str));
00260     }
00261     else if (register_num == 10){
00262         ESP_ERROR_CHECK(nvs_set_str(nvs_handle, "reg10", str));
00263     }
00264
00265     esp_err_t err;
00266     err = nvs_commit(nvs_handle);
00267     if (err == ESP_OK) {
00268         printf("information saved\n");
00269     }

```

```

00270     else if (err != ESP_OK) {
00271         printf("Error al confirmar cambios\n");
00272     }
00273
00274
00275     nvs_close(nvs_handle);
00276 }
00277
00278
00279 esp_err_t read_reg_data(char *str_to_save ,uint8_t register_num){
00280
00281     nvs_handle_t nvs_handle;
00282     esp_err_t err;
00283
00284     // Abrir el NVS
00285     err = nvs_open("storage", NVS_READWRITE, &nvs_handle);
00286     if (err != ESP_OK) {
00287         // Manejar el error
00288         return ESP_FAIL;
00289     }
00290
00291     char reg_to_send[6];
00292     const char *ptr_const_char = reg_to_send;
00293     memset(&reg_to_send[0], 0x00, 6);
00294
00295     if ( register_num == 1 ){
00296         strcpy(reg_to_send, "reg01");
00297     }
00298
00299     else if ( register_num == 2 ){
00300         strcpy(reg_to_send, "reg02");
00301     }
00302
00303     else if ( register_num == 3 ){
00304         strcpy(reg_to_send, "reg03");
00305     }
00306
00307     else if ( register_num == 4 ){
00308         strcpy(reg_to_send, "reg04");
00309     }
00310
00311     else if ( register_num == 5 ){
00312         strcpy(reg_to_send, "reg05");
00313     }
00314
00315     else if ( register_num == 6 ){
00316         strcpy(reg_to_send, "reg06");
00317     }
00318
00319     else if ( register_num == 7 ){
00320         strcpy(reg_to_send, "reg07");
00321     }
00322
00323     else if ( register_num == 8 ){
00324         strcpy(reg_to_send, "reg08");
00325     }
00326
00327     else if ( register_num == 9 ){
00328         strcpy(reg_to_send, "reg09");
00329     }
00330
00331     else if ( register_num == 10 ){
00332         strcpy(reg_to_send, "reg10");
00333     }
00334
00335
00336
00337
00338
00339
00340     size_t required_size;
00341     // Get the size of wifi_ssid
00342     esp_err_t erras;
00343     erras = nvs_get_str(nvs_handle, ptr_const_char, NULL, &required_size);
00344     if (erras != ESP_OK) {
00345         printf("not found\n");
00346         return ESP_FAIL;
00347     }
00348     if (erras == ESP_OK) {
00349         printf("register found\n");
00350     }
00351     // Allocate memory for wifi_ssid
00352
00353     char *reg_buffer = malloc(required_size);
00354     if (reg_buffer == NULL) {
00355         // Handle memory allocation error
00356         ESP_LOGE(TAG, "Failed to allocate memory for wifi_ssid");

```

```

00357         nvs_close(nvs_handle);
00358         return ESP_FAIL;
00359     }
00360     // Get reg
00361     erras = nvs_get_str(nvs_handle, ptr_const_char, reg_buffer, &required_size);
00362     strncpy(str_to_save, reg_buffer, required_size);
00363     ESP_LOGI(TAG, "Valor leído para la clave '%s':%s", reg_to_send, str_to_save);
00364     free(reg_buffer);
00365
00366
00367
00368
00369 /*
00370
00371     err = nvs_get_str(nvs_handle, ptr_const_char, str_to_save, 11);
00372     if (err == ESP_OK) {
00373         // Imprimir la cadena leída
00374         printf("Valor leído:%s\n", str_to_save);
00375     } else {
00376         // Manejar el error al leer
00377         fprintf(stderr, "Error al leer el valor desde el NVS.\n");
00378     }
00379 */
00380
00381 // Cerrar el NVS
00382 nvs_close(nvs_handle);
00383 return ESP_OK;
00384
00385
00386 }
00387 void update_register(int reg_to_update){
00388     char register_information_read[12];
00389     register_information_read[11] = 0x00;
00390     char hora_min_str[3];
00391     hora_min_str[2] = 0x00;
00392     char day[2];
00393     day[1] = 0x00;
00394     register_information_read[11] = 0x00;
00395     if ( read_reg_data( &register_information_read[0], reg_to_update ) == ESP_OK ){
00396
00397         strncpy(&hora_min_str[0], &register_information_read[0], 2);
00398         register_readings_from_flash[reg_to_update-1].hour = atoi(hora_min_str);
00399
00400         strncpy(&hora_min_str[0], &register_information_read[2], 2);
00401         register_readings_from_flash[reg_to_update-1].min = atoi(hora_min_str);
00402
00403         strncpy(&day[0], &register_information_read[4], 1);
00404         register_readings_from_flash[reg_to_update-1].monday = atoi(day);
00405
00406         strncpy(&day[0], &register_information_read[5], 1);
00407         register_readings_from_flash[reg_to_update-1].tuesday = atoi(day);
00408
00409         strncpy(&day[0], &register_information_read[6], 1);
00410         register_readings_from_flash[reg_to_update-1].wednesday = atoi(day);
00411
00412         strncpy(&day[0], &register_information_read[7], 1);
00413         register_readings_from_flash[reg_to_update-1].thursday = atoi(day);
00414
00415         strncpy(&day[0], &register_information_read[8], 1);
00416         register_readings_from_flash[reg_to_update-1].friday = atoi(day);
00417
00418         strncpy(&day[0], &register_information_read[9], 1);
00419         register_readings_from_flash[reg_to_update-1].saturday = atoi(day);
00420
00421         strncpy(&day[0], &register_information_read[10], 1);
00422         register_readings_from_flash[reg_to_update-1].sunday = atoi(day);
00423
00424         ESP_LOGI(TAG, "hora:%d, min:%d, day0:%d, day1:%d, day2:%d, day3:%d, day4:%d, day5:%d,
day6:%d", register_readings_from_flash[reg_to_update-1].hour,
register_readings_from_flash[reg_to_update-1].min,
register_readings_from_flash[reg_to_update-1].monday ,
00425         register_readings_from_flash[reg_to_update-1].tuesday ,
register_readings_from_flash[reg_to_update-1].wednesday ,
register_readings_from_flash[reg_to_update-1].thursday ,
register_readings_from_flash[reg_to_update-1].friday
00426         , register_readings_from_flash[reg_to_update-1].saturday,
register_readings_from_flash[reg_to_update-1].sunday);
00427     }
00428
00429
00430 }
00431 void initialize_registers( void ){
00432
00433     char register_information_read[12];
00434     register_information_read[11] = 0x00;
00435     char hora_min_str[3];
00436     hora_min_str[2] = 0x00;

```

```

00437     char day[2];
00438     day[1] = 0x00;
00439
00440
00441     for (int i = 0; i < NUM_REGISTERS_AV; i++){
00442         if ( read_reg_data( &register_information_read[0], i+1 ) == ESP_OK ){
00443
00444             strncpy(&hora_min_str[0], &register_information_read[0], 2);
00445             register_readings_from_flash[i].hour = atoi(hora_min_str);
00446
00447             strncpy(&hora_min_str[0], &register_information_read[2], 2);
00448             register_readings_from_flash[i].min = atoi(hora_min_str);
00449
00450             strncpy(&day[0], &register_information_read[4], 1);
00451             register_readings_from_flash[i].monday = atoi(day);
00452
00453             strncpy(&day[0], &register_information_read[5], 1);
00454             register_readings_from_flash[i].tuesday = atoi(day);
00455
00456             strncpy(&day[0], &register_information_read[6], 1);
00457             register_readings_from_flash[i].wednesday = atoi(day);
00458
00459             strncpy(&day[0], &register_information_read[7], 1);
00460             register_readings_from_flash[i].thursday = atoi(day);
00461
00462             strncpy(&day[0], &register_information_read[8], 1);
00463             register_readings_from_flash[i].friday = atoi(day);
00464
00465             strncpy(&day[0], &register_information_read[9], 1);
00466             register_readings_from_flash[i].saturday = atoi(day);
00467
00468             strncpy(&day[0], &register_information_read[10], 1);
00469             register_readings_from_flash[i].sunday = atoi(day);
00470
00471             //ESP_LOGI(TAG, "hora:%d, min:%d, day0:%d, day1:%d, day2:%d, day3:%d, day4:%d, day5:%d,
day6:%d", register_readings_from_flash[i].hour, register_readings_from_flash[i].min,
register_readings_from_flash[i].monday ,
00472             //register_readings_from_flash[i].tuesday , register_readings_from_flash[i].wednesday ,
register_readings_from_flash[i].thursday , register_readings_from_flash[i].friday
00473             //, register_readings_from_flash[i].saturday, register_readings_from_flash[i].sunday);
00474         }
00475         else{
00476             register_readings_from_flash[i].hour = 99;
00477             register_readings_from_flash[i].min = 99;
00478             register_readings_from_flash[i].monday = 0;
00479             register_readings_from_flash[i].tuesday = 0;
00480             register_readings_from_flash[i].wednesday = 0;
00481             register_readings_from_flash[i].thursday = 0;
00482             register_readings_from_flash[i].friday = 0;
00483             register_readings_from_flash[i].sunday = 0;
00484             register_readings_from_flash[i].saturday = 0;
00485         }
00486     }
00487 }
00488 }
00489 }
00490
00508 void load_wifi_credentials(char *ssid, char *password) {
00509     nvs_handle_t nvs_handle;
00510     ESP_ERROR_CHECK(nvs_open("storage", NVS_READONLY, &nvs_handle));
00511
00512     size_t required_size;
00513
00514     // Get the size of wifi_ssid
00515     ESP_ERROR_CHECK(nvs_get_str(nvs_handle, "wifi_ssid", NULL, &required_size));
00516     // Allocate memory for wifi_ssid
00517     char *ssid_buffer = malloc(required_size);
00518     if (ssid_buffer == NULL) {
00519         // Handle memory allocation error
00520         ESP_LOGE(TAG, "Failed to allocate memory for wifi_ssid");
00521         nvs_close(nvs_handle);
00522         return;
00523     }
00524     // Get wifi_ssid
00525     ESP_ERROR_CHECK(nvs_get_str(nvs_handle, "wifi_ssid", ssid_buffer, &required_size));
00526     // Copy wifi_ssid to the output parameter
00527     strncpy(ssid, ssid_buffer, required_size);
00528
00529     // Repeat the process for wifi_password
00530     ESP_ERROR_CHECK(nvs_get_str(nvs_handle, "wifi_password", NULL, &required_size));
00531     char *password_buffer = malloc(required_size);
00532     if (password_buffer == NULL) {
00533         // Handle memory allocation error
00534         ESP_LOGE(TAG, "Failed to allocate memory for wifi_password");
00535         free(ssid_buffer);
00536         nvs_close(nvs_handle);
00537         return;

```



```

00538     }
00539     ESP_ERROR_CHECK(nvs_get_str(nvs_handle, "wifi_password", password_buffer, &required_size));
00540     strncpy(password, password_buffer, required_size);
00541
00542     // Free the allocated memory
00543     free(ssid_buffer);
00544     free(password_buffer);
00545
00546     nvs_close(nvs_handle);
00547 }
00548 bool nvs_credentials_exist() {
00549     nvs_handle_t nvs_handle;
00550     esp_err_t err = nvs_open("storage", NVS_READONLY, &nvs_handle);
00551     if (err != ESP_OK) {
00552         return false;
00553     }
00554
00555     size_t ssid_size, password_size;
00556     err = nvs_get_str(nvs_handle, "wifi_ssid", NULL, &ssid_size);
00557     if (err != ESP_OK) {
00558         nvs_close(nvs_handle);
00559         return false;
00560     }
00561
00562     err = nvs_get_str(nvs_handle, "wifi_password", NULL, &password_size);
00563     nvs_close(nvs_handle);
00564
00565     return err == ESP_OK;
00566 }
00567
00568 void connect_to_wifi(void) {
00569     if (xSemaphoreTake(mySemaphore, portMAX_DELAY)) { // helpus to not allow multiple calls
00570
00571         char ssid[32];
00572         char password[64];
00573         load_wifi_credentials(&ssid[0], &password[0]);
00574         wifi_config_t* wifi_config = wifi_app_get_wifi_config();
00575         ESP_LOGI(TAG, "saved SSID: %s", ssid);
00576         ESP_LOGI(TAG, "saved password: %s", password);
00577
00578         //memset(wifi_config->sta.ssid, 0x00, sizeof(wifi_config->sta.ssid));
00579         //memset(wifi_config->sta.password, 0x00, sizeof(wifi_config->sta.password));
00580         memset(wifi_config, 0x00, sizeof(wifi_config_t));
00581         strncpy((char*)wifi_config->sta.ssid, ssid, sizeof(wifi_config->sta.ssid));
00582         strncpy((char*)wifi_config->sta.password, password, sizeof(wifi_config->sta.password));
00583         esp_wifi_set_config(WIFI_IF_STA, wifi_config);
00584         wifi_app_connect_sta();
00585
00586         // Do some work
00587         // Release the semaphore (give it back)
00588         xSemaphoreGive(mySemaphore);
00589     }
00590 }
00591
00592 void check_sta_connection_state( void *pvParameters ) {
00593     wifi_ap_record_t ap_info;
00594     esp_err_t ret;
00595     while(true){
00596         ret = esp_wifi_sta_get_ap_info(&ap_info);
00597         ESP_LOGI(TAG, "Checking sta info");
00598         if (ret == ESP_OK) {
00599
00600             if (ap_info.authmode != WIFI_AUTH_MAX) {
00601                 ESP_LOGI(TAG, "Connected to SSID: %s", ap_info.ssid);
00602                 if (get_state_time_was_synchronized() == false)
00603                     obtain_time();
00604             } else {
00605                 ESP_LOGI(TAG, "Not connected to any WiFi network");
00606                 if (nvs_credentials_exist()) {
00607                     // Credentials exist, try to connect
00608                     connect_to_wifi();
00609                     ESP_LOGI(TAG, "CHECKING CONNECTION TO STA_BEFORE_SAVED");
00610                 }
00611
00612                 //return false;
00613             }
00614         } else {
00615             if (nvs_credentials_exist()) {
00616                 // Credentials exist, try to connect
00617                 connect_to_wifi();
00618                 ESP_LOGI(TAG, "CHECKING CONNECTION TO STA_BEFORE_SAVED");
00619             }
00620         }
00621     }
00622 }

```

```

00641             ESP_LOGI(TAG, "Failed to get connection info");
00642             //return false;
00643         }
00644         vTaskDelay(20000 / portTICK_PERIOD_MS);
00645     }
00646 }
00647
00648 // Get the connection info
00649
00650 //static void wifi_app_event_handler(void *arg, esp_event_base_t event_base, int32_t event_id, void
00651 //event_data)
00676 static void wifi_app_event_handler(void *arg, esp_event_base_t event_base, int32_t event_id, void
*event_data)
00677 {
00678     if (event_base == WIFI_EVENT)
00679     {
00680         switch (event_id)
00681         {
00682             case WIFI_EVENT_AP_START:
00683                 ESP_LOGI(TAG, "WIFI_EVENT_AP_START");
00684                 break;
00685
00686             case WIFI_EVENT_AP_STOP:
00687                 ESP_LOGI(TAG, "WIFI_EVENT_AP_STOP");
00688                 break;
00689
00690             case WIFI_EVENT_AP_STA_CONNECTED:
00691                 ESP_LOGI(TAG, "WIFI_EVENT_AP_STA_CONNECTED");
00692                 break;
00693
00694             case WIFI_EVENT_AP_STA_DISCONNECTED:
00695                 ESP_LOGI(TAG, "WIFI_EVENT_AP_STA_DISCONNECTED");
00696                 break;
00697
00698             case WIFI_EVENT_STA_START:
00699                 ESP_LOGI(TAG, "WIFI_EVENT_STA_START");
00700                 break;
00701
00702             case WIFI_EVENT_STA_CONNECTED:
00703                 ESP_LOGI(TAG, "WIFI_EVENT_STA_CONNECTED");
00704                 break;
00705
00706             case WIFI_EVENT_STA_DISCONNECTED:
00707                 ESP_LOGI(TAG, "WIFI_EVENT_STA_DISCONNECTED");
00708
00709                 // wifi_event_sta_disconnected_t *wifi_event_sta_disconnected =
(wifi_event_sta_disconnected_t*)malloc(sizeof(wifi_event_sta_disconnected_t));
00710                 // *wifi_event_sta_disconnected = *((wifi_event_sta_disconnected_t*)event_data);
00711                 // printf("WIFI_EVENT_STA_DISCONNECTED, reason code %d\n",
wifi_event_sta_disconnected->reason);
00712
00713                 // if (g_retry_number < MAX_CONNECTION_RETRIES)
00714                 // {
00715                 //     esp_wifi_connect();
00716                 //     g_retry_number ++;
00717                 // }
00718                 // else
00719                 // {
00720                 //     wifi_app_send_message(WIFI_APP_MSG_STA_DISCONNECTED);
00721                 // }
00722                 break;
00723         }
00724     }
00725 }
00726 else if (event_base == IP_EVENT)
00727 {
00728     switch (event_id)
00729     {
00730         case IP_EVENT_STA_GOT_IP:
00731             ESP_LOGI(TAG, "IP_EVENT_STA_GOT_IP");
00732
00733             wifi_app_send_message(WIFI_APP_MSG_STA_CONNECTED_GOT_IP);
00734
00735             break;
00736         }
00737     }
00738 }
00739
00743 static void wifi_app_event_handler_init(void)
00744 {
00745     // Event loop for the WiFi driver
00746     ESP_ERROR_CHECK(esp_event_loop_create_default());
00747
00748     // Event handler for the connection
00749     esp_event_handler_instance_t instance_wifi_event;
00750     esp_event_handler_instance_t instance_ip_event;
00751     ESP_ERROR_CHECK(esp_event_handler_instance_register(WIFI_EVENT, ESP_EVENT_ANY_ID,

```

```

    &wifi_app_event_handler, NULL, &instance_wifi_event));
00752     ESP_ERROR_CHECK(esp_event_handler_instance_register(IP_EVENT, ESP_EVENT_ANY_ID,
    &wifi_app_event_handler, NULL, &instance_ip_event));
00753 }
00754
00758 static void wifi_app_default_wifi_init(void)
00759 {
00760     // Initialize the TCP stack
00761     ESP_ERROR_CHECK(esp_netif_init());
00762
00763     // Default WiFi config - operations must be in this order!
00764     wifi_init_config_t wifi_init_config = WIFI_INIT_CONFIG_DEFAULT();
00765     ESP_ERROR_CHECK(esp_wifi_init(&wifi_init_config));
00766     ESP_ERROR_CHECK(esp_wifi_set_storage(WIFI_STORAGE_RAM));
00767     esp_netif_sta = esp_netif_create_default_wifi_sta();
00768     esp_netif_ap = esp_netif_create_default_wifi_ap();
00769 }
00770
00792 static void wifi_app_soft_ap_config(void)
00793 {
00794     // SoftAP - WiFi access point configuration
00795     wifi_config_t ap_config =
00796     {
00797         .ap = {
00798             .ssid = WIFI_AP_SSID,
00799             .ssid_len = strlen(WIFI_AP_SSID),
00800             .password = WIFI_AP_PASSWORD,
00801             .channel = WIFI_AP_CHANNEL,
00802             .ssid_hidden = WIFI_AP_SSID_HIDDEN,
00803             .authmode = WIFI_AUTH_WPA2_PSK,
00804             .max_connection = WIFI_AP_MAX_CONNECTIONS,
00805             .beacon_interval = WIFI_AP_BEACON_INTERVAL,
00806         },
00807     };
00808
00809     // Configure DHCP for the AP
00810     esp_netif_ip_info_t ap_ip_info;
00811     memset(&ap_ip_info, 0x00, sizeof(ap_ip_info));
00812
00813     esp_netif_dhcps_stop(esp_netif_ap);
00814     inet_pton(AF_INET, WIFI_AP_IP, &ap_ip_info.ip);
00815     inet_pton(AF_INET, WIFI_AP_GATEWAY, &ap_ip_info.gw);
00816     inet_pton(AF_INET, WIFI_AP_NETMASK, &ap_ip_info.netmask);
00817     ESP_ERROR_CHECK(esp_netif_set_ip_info(esp_netif_ap, &ap_ip_info));
00818     ESP_ERROR_CHECK(esp_netif_dhcps_start(esp_netif_ap));
00819
00820     ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_APSTA));
00821     ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_AP, &ap_config));
00822     ESP_ERROR_CHECK(esp_wifi_set_bandwidth(WIFI_IF_AP, WIFI_AP_BANDWIDTH));
00823     ESP_ERROR_CHECK(esp_wifi_set_ps(WIFI_STA_POWER_SAVE));
00824 }
00825 }
00826
00836 static void wifi_app_connect_sta(void)
00837 {
00838     ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_STA, wifi_app_get_wifi_config()));
00839     ESP_ERROR_CHECK(esp_wifi_connect());
00840 }
00841
00862 static void wifi_app_task(void *pvParameters)
00863 {
00864     wifi_app_queue_message_t msg;
00865
00866     // Initialize the event handler
00867     wifi_app_event_handler_init();
00868
00869     // Initialize the TCP/IP stack and WiFi config
00870     wifi_app_default_wifi_init();
00871
00872     // SoftAP config
00873     wifi_app_soft_ap_config();
00874
00875     // Start WiFi
00876     ESP_ERROR_CHECK(esp_wifi_start());
00877
00878     // Send first event message
00879     wifi_app_send_message(WIFI_APP_MSG_START_HTTP_SERVER);
00880     wifi_app_send_message(WIFI_APP_CONNECT_TO_STA);
00881     initialize_registers(); //Init registers of hours
00882
00883     for (;;)
00884     {
00885         if (xQueueReceive(wifi_app_queue_handle, &msg, portMAX_DELAY))
00886         {
00887             switch (msg.msgID)
00888             {
00889                 case WIFI_APP_CONNECT_TO_STA:

```

```

00890             //ESP_LOGI(TAG, "CHECKING CONNECTION TO STA");
00891             xTaskCreatePinnedToCore(&check_sta_connection_state, "check_sta_connection_state",
WIFI_APP_TASK_STACK_SIZE, NULL, WIFI_APP_TASK_PRIORITY, NULL, WIFI_APP_TASK_CORE_ID);
00892         /*
00893             if (nvs_credentials_exist()) {
00894                 // Credentials exist, try to connect
00895                 connect_to_wifi();
00896                 ESP_LOGI(TAG, "CHECKING CONNECTION TO STA_BEFORE_SAVED");
00897             }
00898             else{
00899                 ESP_LOGI(TAG, "NO INFORMATION WAS FOUND ABOUT CREDENTIALS");
00900             }*/
00901             break;
00902
00903         case WIFI_APP_MSG_START_HTTP_SERVER:
00904             ESP_LOGI(TAG, "WIFI_APP_MSG_START_HTTP_SERVER");
00905
00906             http_server_start();
00907             rgb_led_http_server_started();
00908
00909             break;
00910
00911         case WIFI_APP_MSG_CONNECTING_FROM_HTTP_SERVER:
00912             ESP_LOGI(TAG, "WIFI_APP_MSG_CONNECTING_FROM_HTTP_SERVER");
00913
00914             // Attempt a connection
00915             wifi_app_connect_sta();
00916
00917             // Set current number of retries to zero
00918             g_retry_number = 0;
00919
00920             // Let the HTTP server know about the connection attempt
00921             http_server_monitor_send_message(HTTP_MSG_WIFI_CONNECT_INIT);
00922
00923             break;
00924
00925         case WIFI_APP_MSG_STA_CONNECTED_GOT_IP:
00926             ESP_LOGI(TAG, "WIFI_APP_MSG_STA_CONNECTED_GOT_IP");
00927
00928             rgb_led_wifi_connected();
00929             http_server_monitor_send_message(HTTP_MSG_WIFI_CONNECT_SUCCESS);
00930
00931             break;
00932
00933         case WIFI_APP_MSG_STA_DISCONNECTED:
00934             ESP_LOGI(TAG, "WIFI_APP_MSG_STA_DISCONNECTED");
00935
00936             http_server_monitor_send_message(HTTP_MSG_WIFI_CONNECT_FAIL);
00937
00938             break;
00939
00940         default:
00941             break;
00942     }
00943 }
00944 }
00945 }
00946 }
00947
00948 BaseType_t wifi_app_send_message(wifi_app_message_e msgID)
00949 {
00950     wifi_app_queue_message_t msg;
00951     msg.msgID = msgID;
00952     return xQueueSend(wifi_app_queue_handle, &msg, portMAX_DELAY);
00953 }
00954
00955 wifi_config_t* wifi_app_get_wifi_config(void)
00956 {
00957     return wifi_config;
00958 }
00959
00960 bool compare_hour_day_structs (struct tm timeinfo, register_saved_e aux_reg ){
00961     static const char TAG2 [] = "comparing_app";
00962
00963     // chekcing ll the day
00964     if(timeinfo.tm_wday == 0){
00965         if( aux_reg.sunday != 1 ){
00966             ESP_LOGI(TAG2, "WRONG DAY");
00967             return false;
00968         }
00969     }
00970
00971     if(timeinfo.tm_wday == 1){
00972         if( aux_reg.monday != 1 ){
00973             ESP_LOGI(TAG2, "WRONG DAY");
00974             return false;
00975         }
00976     }
00977
00978     return true;
00979 }

```

```

00983     }
00984
00985     if(timeinfo.tm_wday == 2){
00986         if( aux_reg.tuesday != 1 ){
00987             ESP_LOGI(TAG2, "WRONG DAY");
00988             return false;
00989         }
00990     }
00991
00992     if(timeinfo.tm_wday == 3){
00993         if( aux_reg.wednesday != 1 ){
00994             ESP_LOGI(TAG2, "WRONG DAY");
00995             return false;
00996         }
00997     }
00998
00999     if(timeinfo.tm_wday == 4){
01000         if( aux_reg.thursday != 1 ){
01001             ESP_LOGI(TAG2, "WRONG DAY");
01002             return false;
01003         }
01004     }
01005
01006     if(timeinfo.tm_wday == 5){
01007         if( aux_reg.friday != 1 ){
01008             ESP_LOGI(TAG2, "WRONG DAY");
01009             return false;
01010         }
01011     }
01012
01013     if(timeinfo.tm_wday == 6){
01014         if( aux_reg.saturday != 1 ){
01015             ESP_LOGI(TAG2, "WRONG DAY");
01016             return false;
01017         }
01018     }
01019
01020     if( timeinfo.tm_hour == aux_reg.hour ){
01021         if( timeinfo.tm_min == aux_reg.min ){
01022             // we should activate the motor
01023             toogle_led();
01024             vTaskDelay(40000 / portTICK_PERIOD_MS);
01025             return true;
01026         }
01027         else{
01028             ESP_LOGI(TAG2, "CORRECT DAY CORRECT HOUR WRONG MINUTE");
01029             return false;
01030         }
01031     }
01032     else{
01033         ESP_LOGI(TAG2, "CORRECT DAY WRONG HOUR");
01034         return false;
01035     }
01036 }
01037
01038 }
01039
01058 void task_compare_hour_to_execute_action( void *pvParameters ) {
01059     time_t now;
01060     struct tm timeinfo;
01061     while(get_state_time_was_synchronized() == false){
01062         vTaskDelay(10000 / portTICK_PERIOD_MS);
01063     }
01064     // Asegurar que localtime_r inicialice adecuadamente timeinfo
01065
01066     while (1){
01067
01068         ESP_LOGI(TAG, "COMPARING HOURS");
01069
01070
01071         if (time(&now) != -1 && localtime_r(&now, &timeinfo) != NULL)
01072         {
01073             // Imprimir la hora actual
01074             // ESP_LOGI(TAG, "Día de la semana:%d, Hora:%02d:%02d:%02d", timeinfo.tm_wday,
01075             timeinfo.tm_hour, timeinfo.tm_min, timeinfo.tm_sec);
01076             ESP_LOGI(TAG, "Día de la semana:%d, Hora:%d:%d:%d", timeinfo.tm_wday, timeinfo.tm_hour,
01077             timeinfo.tm_min, timeinfo.tm_sec);
01078         }
01079         else
01080         {
01081             ESP_LOGE(TAG, "Error al obtener la hora actual.");
01082         }
01083
01084         for(int i = 0; i< NUM_REGISTERS_AV; i++){
01085             ESP_LOGI(TAG, "Revisando registro:%d", i);

```

```

01086         compare_hour_day_structs (timeinfo, register_readings_from_flash[i] );
01087     }
01088 }
01089
01090     vTaskDelay(30000 / portTICK_PERIOD_MS);
01091 }
01092
01093 }
01094 }
01095
01115 void wifi_app_start(void)
01116 {
01117     ESP_LOGI(TAG, "STARTING WIFI APPLICATION");
01118
01119     // Start WiFi started LED
01120     rgb_led_wifi_app_started();
01121
01122     // Disable default WiFi logging messages
01123     esp_log_level_set("wifi", ESP_LOG_NONE);
01124
01125     // Allocate memory for the wifi configuration
01126     wifi_config = (wifi_config_t*)malloc(sizeof(wifi_config_t));
01127     memset(wifi_config, 0x00, sizeof(wifi_config_t));
01128
01129     // Create message queue
01130     wifi_app_queue_handle = xQueueCreate(3, sizeof(wifi_app_queue_message_t));
01131     // create semaphore for the wifi connection
01132     mySemaphore = xSemaphoreCreateBinary();
01133     xSemaphoreGive(mySemaphore);
01134     // Start the WiFi application task
01135     xTaskCreatePinnedToCore(&wifi_app_task, "wifi_app_task", WIFI_APP_TASK_STACK_SIZE, NULL,
WIFI_APP_TASK_PRIORITY, NULL, WIFI_APP_TASK_CORE_ID);
01136     xTaskCreatePinnedToCore(&task_compare_hour_to_execute_action, "checking_app_task", 4096, NULL, 5,
NULL, 1);
01137
01138
01139 }
01140
01141
01142
01143
01144
01145
01146
01147
01148

```

9.35. Referencia del archivo wifi_app.h

Gestión completa de WiFi AP/STA con reconexión automática y persistencia.

```

#include "esp_netif.h"
#include "esp_wifi_types.h"
#include "freertos/FreeRTOS.h"

```

Estructuras de datos

- struct `wifi_app_queue_message`
- struct `register_saved`

defines

- #define `WIFI_AP_SSID` "Mi_esp"
Nombre de la red AP.
- #define `WIFI_AP_PASSWORD` "12345678"
Contraseña del AP (mínimo 8 caracteres).
- #define `WIFI_AP_CHANNEL` 1

- Canal WiFi del AP (1-13).
- #define `WIFI_AP_SSID_HIDDEN` 0
 - 0=visible, 1=oculto
- #define `WIFI_AP_MAX_CONNECTIONS` 5
 - Máximo de clientes simultáneos.
- #define `WIFI_AP_BEACON_INTERVAL` 100
 - Intervalo beacon en ms (recomendado 100ms).
- #define `WIFI_AP_IP` "192.168.0.1"
 - IP del AP.
- #define `WIFI_AP_GATEWAY` "192.168.0.1"
 - Gateway del AP (igual que IP).
- #define `WIFI_AP_NETMASK` "255.255.255.0"
 - Máscara de subred.
- #define `WIFI_AP_BANDWIDTH` `WIFI_BW_HT20`
 - Ancho de banda 20 MHz.
- #define `WIFI_STA_POWER_SAVE` `WIFI_PS_NONE`
 - Sin ahorro de energía.
- #define `MAX_SSID_LENGTH` 32
 - Longitud máxima SSID (estándar IEEE).
- #define `MAX_PASSWORD_LENGTH` 64
 - Longitud máxima contraseña (estándar IEEE).
- #define `MAX_CONNECTION_RETRIES` 5
 - Intentos de reconexión antes de fallar.
- #define `NUM_REGISTERS_AV` 10
 - Número máximo de registros programados.

typedefs

- typedef void(* `wifi_connected_event_callback_t`) (void)
- typedef enum `wifi_app_message` `wifi_app_message_e`
- typedef struct `wifi_app_queue_message` `wifi_app_queue_message_t`
- typedef struct `register_saved` `register_saved_e`

Enumeraciones

- enum `wifi_app_message` {
 - `WIFI_APP_MSG_START_HTTP_SERVER` = 0 , `WIFI_APP_MSG_CONNECTING_FROM_HTTP_SERVER` ,
 - `WIFI_APP_MSG_STA_CONNECTED_GOT_IP` , `WIFI_APP_MSG_USER_REQUESTED_STA_DISCONNECT`
 - ,
 - `WIFI_APP_MSG_LOAD_SAVED_CREDENTIALS` , `WIFI_APP_MSG_STA_DISCONNECTED` , `WIFI_APP_CONNECT_TO_STA`

Funciones

- static void [obtain_time](#) (void)
Obtener hora desde servidor SNTP.
- void [connect_to_wifi](#) (void)
Conectar a WiFi usando credenciales guardadas.
- void [initialize_registers](#) (void)
Inicializar sistema de registros programados.
- esp_err_t [read_reg_data](#) (char *str_to_save, uint8_t register_num)
Leer datos de registro desde NVS.
- void [init_obtain_time](#) (void)
Inicializar sincronización de hora SNTP.
- bool [get_state_time_was_synchronized](#) (void)
Verificar si la hora ya fue sincronizada.
- void [save_reg_data](#) (uint8_t register, char *str)
Guardar registro en NVS.
- void [save_wifi_credentials](#) (const char *ssid, const char *password)
Guardar credenciales WiFi en NVS.
- void [load_wifi_credentials](#) (char *ssid, char *password)
Cargar credenciales WiFi desde NVS.
- static void [wifi_app_connect_sta](#) (void)
Conectar ESP32 a red WiFi como Station.
- void [check_sta_connection_state](#) (void *pvParameters)
Tarea que verifica estado de conexión STA.
- void [task_compare_hour_to_execute_action](#) (void *pvParameters)
Tarea que compara hora actual con registros programados.
- void [update_register](#) (int reg_to_update)
Actualizar registro específico.
- BaseType_t [wifi_app_send_message](#) ([wifi_app_message_e](#) msgID)
Enviar mensaje a la cola de la tarea WiFi.
- void [wifi_app_start](#) (void)
Iniciar tarea RTOS de WiFi.
- wifi_config_t * [wifi_app_get_wifi_config](#) (void)
Obtener configuración actual de WiFi.
- void [wifi_app_set_callback](#) ([wifi_connected_event_callback_t](#) cb)
Registrar callback para evento de conexión exitosa.
- void [wifi_app_call_callback](#) (void)
Ejecutar callback registrado.
- int8_t [wifi_app_get_rssi](#) (void)
Obtener intensidad de señal WiFi (RSSI).

Variables

- esp_netif_t * [esp_netif_sta](#)
Handle de interfaz de red Station.
- esp_netif_t * [esp_netif_ap](#)
Handle de interfaz de red Access Point.

9.35.1. Descripción detallada

Gestión completa de WiFi AP/STA con reconexión automática y persistencia.

Autor

Jair Hernan Telpis Cuaran, Luis Fernando Gamba Bedoya

Fecha

2025

Versión

1.0.0

Módulo que gestiona la configuración dual WiFi (Access Point + Station) del ESP32:

- **Access Point (AP):** Crea red WiFi para configuración inicial
- **Station (STA):** Se conecta a redes WiFi existentes
- **Reconexión Automática:** Reintenta conexión hasta `MAX_CONNECTION_RETRIES` veces
- **Persistencia NVS:** Guarda credenciales WiFi para reconexión tras reinicio
- **Sincronización SNTP:** Obtiene hora de servidores NTP cuando se conecta
- **Cola de Mensajes RTOS:** Comunicación thread-safe con el servidor HTTP

El sistema inicia en modo AP+STA, permitiendo configuración web mientras mantiene el AP activo. Las credenciales se guardan en NVS y se cargan automáticamente al reiniciar.

Universidad Nacional de Colombia - Curso RTOS

Definición en el archivo [wifi_app.h](#).

9.35.2. Documentación de «define»

9.35.2.1. NUM_REGISTERS_AV

```
#define NUM_REGISTERS_AV 10
```

Número máximo de registros programados.

Definición en la línea 51 del archivo [wifi_app.h](#).

Referenciado por [initialize_registers\(\)](#) y [task_compare_hour_to_execute_action\(\)](#).

9.35.3. Documentación de «typedef»

9.35.3.1. register_saved_e

```
typedef struct register\_saved register_saved_e
```

9.35.3.2. wifi_app_message_e

```
typedef enum wifi_app_message wifi_app_message_e
```

9.35.3.3. wifi_app_queue_message_t

```
typedef struct wifi_app_queue_message wifi_app_queue_message_t
```

9.35.3.4. wifi_connected_event_callback_t

```
typedef void(* wifi_connected_event_callback_t) (void)
```

Definición en la línea 30 del archivo [wifi_app.h](#).

9.35.4. Documentación de enumeraciones

9.35.4.1. wifi_app_message

```
enum wifi_app_message
```

Valores de enumeraciones

WIFI_APP_MSG_START_HTTP_SERVER	Iniciar servidor HTTP tras conexión WiFi.
WIFI_APP_MSG_CONNECTING_FROM_HTTP_SERVER	Usuario solicitó conexión desde web.
WIFI_APP_MSG_STA_CONNECTED_GOT_IP	STA conectado y obtuvo IP por DHCP.
WIFI_APP_MSG_USER_REQUESTED_STA_DISCONNECT	Usuario solicitó desconexión desde web.
WIFI_APP_MSG_LOAD_SAVED_CREDENTIALS	Cargar credenciales guardadas en NVS.
WIFI_APP_MSG_STA_DISCONNECTED	STA desconectado (evento del sistema).
WIFI_APP_CONNECT_TO_STA	Comando para conectar a STA.

Definición en la línea 65 del archivo [wifi_app.h](#).

9.35.5. Documentación de funciones

9.35.5.1. check_sta_connection_state()

```
void check_sta_connection_state (
    void * pvParameters)
```

Tarea que verifica estado de conexión STA.

Parámetros

<code>pvParameters</code>	Parámetros de la tarea (sin usar)
---------------------------	-----------------------------------

Monitorea conexión y reintenta automáticamente si falla

Tarea que verifica estado de conexión STA.

Cada 30 segundos verifica si está conectado a AP. Si pierde conexión, intenta reconectar automáticamente.

Funcionalidad:

- Llama `esp_wifi_sta_get_ap_info()` cada 30s
- Si falla (`ESP_ERR_WIFI_NOT_CONNECT`), intenta conectar
- Registra RSSI si está conectado

Parámetros

in	<code>pvParameters</code>	Parámetro de tarea (no usado)
----	---------------------------	-------------------------------

Nota

Tarea infinita creada en `wifi_app_start()`

Definición en la línea 611 del archivo `wifi_app.c`.

Hace referencia a `connect_to_wifi()`, `get_state_time_was_synchronized()`, `nvs_credentials_exist()`, `obtain_time()` y `TAG`.

Referenciado por `wifi_app_task()`.

9.35.5.2. `connect_to_wifi()`

```
void connect_to_wifi (
    void )
```

Conectar a WiFi usando credenciales guardadas.

Lee credenciales de NVS e intenta conexión STA

Definición en la línea 569 del archivo `wifi_app.c`.

Hace referencia a `load_wifi_credentials()`, `mySemaphore`, `TAG`, `wifi_app_connect_sta()`, `wifi_app_get_wifi_config()` y `wifi_config`.

Referenciado por `check_sta_connection_state()` y `http_server_wifi_connect_json_handler()`.

9.35.5.3. `get_state_time_was_synchronized()`

```
bool get_state_time_was_synchronized (
    void )
```

Verificar si la hora ya fue sincronizada.

Devuelve

true si el sistema tiene hora válida de SNTP

Verificar si la hora ya fue sincronizada.

Devuelve

bool

Valores devueltos

<i>true</i>	Tiempo sincronizado correctamente con SNTP
<i>false</i>	Aún no sincronizado o fallo

Definición en la línea 124 del archivo [wifi_app.c](#).

Hace referencia a [time_was_synchronized](#).

Referenciado por [check_sta_connection_state\(\)](#) y [task_compare_hour_to_execute_action\(\)](#).

9.35.5.4. [init_obtain_time\(\)](#)

```
void init_obtain_time (
    void )
```

Inicializar sincronización de hora SNTP.

Configura servidor NTP y zona horaria

Inicializar sincronización de hora SNTP.

Marca el tiempo como NO sincronizado. Llamar antes de intentar sincronizar con SNTP.

Definición en la línea 113 del archivo [wifi_app.c](#).

Hace referencia a [time_was_synchronized](#).

9.35.5.5. [initialize_registers\(\)](#)

```
void initialize_registers (
    void )
```

Inicializar sistema de registros programados.

Carga registros desde NVS al arranque < Número máximo de registros programados

Definición en la línea 431 del archivo [wifi_app.c](#).

Hace referencia a [NUM_REGISTERS_AV](#), [read_reg_data\(\)](#) y [register_readings_from_flash](#).

Referenciado por [wifi_app_task\(\)](#).

9.35.5.6. [load_wifi_credentials\(\)](#)

```
void load_wifi_credentials (
    char * ssid,
    char * password)
```

Cargar credenciales WiFi desde NVS.

Parámetros

out	<i>ssid</i>	Buffer para SSID (mínimo MAX_SSID_LENGTH)
out	<i>password</i>	Buffer para contraseña (mínimo MAX_PASSWORD_LENGTH)

Cargar credenciales WiFi desde NVS.

Lee SSID y password almacenados previamente en NVS.

Namespace NVS: "storage" **Keys:**

- "wifi_ssid": SSID de la red
- "wifi_password": Contraseña

Parámetros

out	<i>ssid</i>	Buffer donde se copia el SSID (mínimo 33 bytes)
out	<i>password</i>	Buffer donde se copia el password (mínimo 65 bytes)

Nota

Llama ESP_ERROR_CHECK - programa panic si no existen credenciales

Atención

Validar existencia con [nvs_credentials_exist\(\)](#) antes de llamar

Definición en la línea 508 del archivo [wifi_app.c](#).

Hace referencia a [TAG](#).

Referenciado por [connect_to_wifi\(\)](#).

9.35.5.7. obtain_time()

```
void obtain_time (
    void ) [static]
```

Obtener hora desde servidor SNTP.

Sincroniza reloj del sistema con pool.ntp.org

Nota

Función interna, llamada automáticamente tras conexión WiFi

9.35.5.8. read_reg_data()

```
esp_err_t read_reg_data (
    char * str_to_save,
    uint8_t register_num)
```

Leer datos de registro desde NVS.

Parámetros

out	<i>str_to_save</i>	Buffer donde se escribirá el registro (JSON)
	<i>register_num</i>	Número de registro (1-10)

Devuelve

ESP_OK si se leyó correctamente

Definición en la línea 279 del archivo [wifi_app.c](#).

Hace referencia a [TAG](#).

Referenciado por [initialize_registers\(\)](#) y [update_register\(\)](#).

9.35.5.9. save_reg_data()

```
void save_reg_data (
    uint8_t register_num,
    char * str)
```

Guardar registro en NVS.

Parámetros

<i>register</i>	Número de registro (1-10)
<i>str</i>	Datos del registro en formato string

Guardar registro en NVS.

Función antigua para guardar registros (1-10). **DEPRECADA:** Usar [registers.c](#) (save_register_to_nvs) en su lugar.

Parámetros

in	<i>register_num</i>	Número de registro (1-10)
in	<i>str</i>	Datos del registro como string

Obsoleto Usar API moderna en [registers.c](#)

Nota

Solo para compatibilidad con código antiguo

Definición en la línea 230 del archivo [wifi_app.c](#).

9.35.5.10. save_wifi_credentials()

```
void save_wifi_credentials (
    const char * ssid,
    const char * password)
```

Guardar credenciales WiFi en NVS.

Parámetros

<i>ssid</i>	Nombre de red WiFi
<i>password</i>	Contraseña de red WiFi

Nota

Las credenciales persisten tras reinicios

Guardar credenciales WiFi en NVS.

Almacena SSID y contraseña en memoria no volátil. Sobrevive a reinicios del ESP32.

Claves NVS:

- "wifi_ssid": Nombre de la red WiFi
- "wifi_password": Contraseña

Parámetros

in	<i>ssid</i>	SSID de la red WiFi (máx 32 caracteres)
in	<i>password</i>	Contraseña (máx 64 caracteres)

Atención

ESP_ERROR_CHECK causa panic si falla NVS

Nota

Namespace "storage" debe existir

Definición en la línea 208 del archivo [wifi_app.c](#).

Referenciado por [http_server_wifi_connect_json_handler\(\)](#).

9.35.5.11. task_compare_hour_to_execute_action()

```
void task_compare_hour_to_execute_action (
    void * pvParameters)
```

Tarea que compara hora actual con registros programados.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (sin usar)
---------------------	-----------------------------------

Ejecuta cada minuto, activa ventilador si hay coincidencia horaria

Tarea que compara hora actual con registros programados.

Espera a que SNTP sincronice, luego cada 60s compara hora actual con registros programados. Si coincide, ejecuta acción (ventilador).

Proceso:

1. Espera a `get_state_time_was_synchronized() == true`
2. Cada 60s obtiene hora con `time()` y `localtime_r()`
3. Itera registros y compara con `compare_hour_day_structs()`
4. Si coincide, ejecuta acción del registro

Parámetros

in	<i>pvParameters</i>	Parámetro de tarea (no usado)
----	---------------------	-------------------------------

Nota

Tarea infinita, stack 4096 bytes, prioridad 5

Ver también

[compare_hour_day_structs\(\)](#) Comparación hora/día

< Número máximo de registros programados

Definición en la línea 1058 del archivo [wifi_app.c](#).

Hace referencia a [compare_hour_day_structs\(\)](#), [get_state_time_was_synchronized\(\)](#), [NUM_REGISTERS_AV](#), [register_readings_from_flash](#) y [TAG](#).

Referenciado por [wifi_app_start\(\)](#).

9.35.5.12. update_register()

```
void update_register (
    int reg_to_update)
```

Actualizar registro específico.

Parámetros

<i>reg_to_update</i>	Número de registro a actualizar (1-10)
----------------------	--

Definición en la línea 387 del archivo [wifi_app.c](#).

Hace referencia a [read_reg_data\(\)](#), [register_readings_from_flash](#) y [TAG](#).

9.35.5.13. wifi_app_call_callback()

```
void wifi_app_call_callback (
    void )
```

Ejecutar callback registrado.

Llamado internamente cuando STA obtiene IP

9.35.5.14. wifi_app_connect_sta()

```
void wifi_app_connect_sta (
    void ) [static]
```

Conectar ESP32 a red WiFi como Station.

Nota

Función interna, usa credenciales cargadas previamente

9.35.5.15. `wifi_app_get_rssi()`

```
int8_t wifi_app_get_rssi (
    void )
```

Obtener intensidad de señal WiFi (RSSI).

Devuelve

Nivel RSSI actual en dBm (valores negativos, ej: -50 dBm)

Nota

Solo válido cuando STA está conectado

9.35.5.16. `wifi_app_get_wifi_config()`

```
wifi_config_t * wifi_app_get_wifi_config (
    void )
```

Obtener configuración actual de WiFi.

Devuelve

Puntero a estructura `wifi_config_t` con configuración STA actual

Obtener configuración actual de WiFi.

Devuelve

`wifi_config_t*` Puntero a `wifi_config` global

Nota

Contiene SSID y password cargados de NVS

Definición en la línea [962](#) del archivo [wifi_app.c](#).

Hace referencia a [wifi_config](#).

Referenciado por [connect_to_wifi\(\)](#), [http_server_wifi_connect_json_handler\(\)](#) y [wifi_app_connect_sta\(\)](#).

9.35.5.17. `wifi_app_send_message()`

```
BaseType_t wifi_app_send_message (
    wifi_app_message_e msgID)
```

Enviar mensaje a la cola de la tarea WiFi.

Parámetros

<i>msgID</i>	ID del mensaje desde wifi_app_message_e
--------------	---

Devuelve

pdTRUE si se envió correctamente, pdFALSE si la cola está llena

Nota

Thread-safe, puede llamarse desde cualquier tarea

Definición en la línea 948 del archivo `wifi_app.c`.

Hace referencia a [wifi_app_queue_handle](#).

Referenciado por [wifi_app_event_handler\(\)](#) y [wifi_app_task\(\)](#).

9.35.5.18. `wifi_app_set_callback()`

```
void wifi_app_set_callback (  
    wifi_connected_event_callback_t cb)
```

Registrar callback para evento de conexión exitosa.

Parámetros

<i>cb</i>	Función callback a ejecutar cuando STA se conecta
-----------	---

9.35.5.19. `wifi_app_start()`

```
void wifi_app_start (  
    void )
```

Iniciar tarea RTOS de WiFi.

Crea la tarea principal que gestiona eventos WiFi y mensajes

Nota

Debe llamarse una vez durante la inicialización del sistema

Iniciar tarea RTOS de WiFi.

Punto de entrada principal para WiFi. Secuencia:

1. Inicializa netif TCP/IP
2. Configura event handlers WiFi
3. Inicializa WiFi con config por defecto
4. Configura Access Point
5. Crea cola de mensajes
6. Crea tarea `wifi_app_task`
7. Crea tarea `check_sta_connection_state` (reconexión)

Nota

Llamar una sola vez durante inicialización del sistema

Atención

Requiere NVS inicializado previamente

Ver también

[wifi_app_task\(\)](#) Tarea principal creada

[check_sta_connection_state\(\)](#) Tarea de reconexión

Definición en la línea 1115 del archivo [wifi_app.c](#).

Hace referencia a [mySemaphore](#), [TAG](#), [task_compare_hour_to_execute_action\(\)](#), [wifi_app_queue_handle](#), [wifi_app_task\(\)](#) y [wifi_config](#).

9.35.6. Documentación de variables

9.35.6.1. esp_netif_ap

```
esp_netif_t* esp_netif_ap [extern]
```

Handle de interfaz de red Access Point.

Handle de interfaz de red Access Point.

Definición en la línea 98 del archivo [wifi_app.c](#).

Referenciado por [wifi_app_default_wifi_init\(\)](#) y [wifi_app_soft_ap_config\(\)](#).

9.35.6.2. esp_netif_sta

```
esp_netif_t* esp_netif_sta [extern]
```

Handle de interfaz de red Station.

Handle de interfaz de red Station.

Definición en la línea 95 del archivo [wifi_app.c](#).

Referenciado por [wifi_app_default_wifi_init\(\)](#).

9.36. wifi_app.h

[Ir a la documentación de este archivo.](#)

```

00001
00021
00022 #ifndef MAIN_WIFI_APP_H_
00023 #define MAIN_WIFI_APP_H_
00024
00025 #include "esp_netif.h"
00026 #include "esp_wifi_types.h"
00027 #include "freertos/FreeRTOS.h"
00028
00029 // Callback typedef
00030 typedef void (*wifi_connected_event_callback_t)(void);
00031
00035 #define WIFI_AP_SSID "Mi_esp"
00036 #define WIFI_AP_PASSWORD "12345678"
00037 #define WIFI_AP_CHANNEL 1
00038 #define WIFI_AP_SSID_HIDDEN 0
00039 #define WIFI_AP_MAX_CONNECTIONS 5
00040 #define WIFI_AP_BEACON_INTERVAL 100
00041 #define WIFI_AP_IP "192.168.0.1"
00042 #define WIFI_AP_GATEWAY "192.168.0.1"
00043 #define WIFI_AP_NETMASK "255.255.255.0"
00044 #define WIFI_AP_BANDWIDTH WIFI_BW_HT20
00045 #define WIFI_STA_POWER_SAVE WIFI_PS_NONE
00046 #define MAX_SSID_LENGTH 32
00047 #define MAX_PASSWORD_LENGTH 64
00048 #define MAX_CONNECTION_RETRIES 5
00050
00051 #define NUM_REGISTERS_AV 10
00052
00054 extern esp_netif_t* esp_netif_sta;
00056 extern esp_netif_t* esp_netif_ap;
00057
00065 typedef enum wifi_app_message
00066 {
00067     WIFI_APP_MSG_START_HTTP_SERVER = 0,
00068     WIFI_APP_MSG_CONNECTING_FROM_HTTP_SERVER,
00069     WIFI_APP_MSG_STA_CONNECTED_GOT_IP,
00070     WIFI_APP_MSG_USER_REQUESTED_STA_DISCONNECT,
00071     WIFI_APP_MSG_LOAD_SAVED_CREDENTIALS,
00072     WIFI_APP_MSG_STA_DISCONNECTED,
00073     WIFI_APP_CONNECT_TO_STA,
00074 } wifi_app_message_e;
00075
00080 typedef struct wifi_app_queue_message
00081 {
00082     wifi_app_message_e msgID;
00083 } wifi_app_queue_message_t;
00084
00091 typedef struct register_saved
00092 {
00093     uint8_t hour;
00094     uint8_t min;
00095     uint8_t monday;
00096     uint8_t tuesday;
00097     uint8_t wednesday;
00098     uint8_t thursday;
00099     uint8_t friday;
00100     uint8_t sunday;
00101     uint8_t saturday;
00102 } register_saved_e;
00103
00109 static void obtain_time(void);
00110
00115 void connect_to_wifi(void);
00116
00121 void initialize_registers(void);
00122
00129 esp_err_t read_reg_data(char *str_to_save, uint8_t register_num);
00130
00135 void init_obtain_time(void);
00136
00141 bool get_state_time_was_synchronized(void);
00142
00148 void save_reg_data(uint8_t register, char *str);
00149
00156 void save_wifi_credentials(const char *ssid, const char *password);
00157
00163 void load_wifi_credentials(char *ssid, char *password);
00164
00169 static void wifi_app_connect_sta(void);
00170
00176 void check_sta_connection_state(void *pvParameters);

```

```
00177
00183 void task_compare_hour_to_execute_action(void *pvParameters);
00184
00189 void update_register(int reg_to_update);
00190
00197 BaseType_t wifi_app_send_message(wifi_app_message_e msgID);
00198
00204 void wifi_app_start(void);
00205
00210 wifi_config_t* wifi_app_get_wifi_config(void);
00211
00216 void wifi_app_set_callback(wifi_connected_event_callback_t cb);
00217
00222 void wifi_app_call_callback(void);
00223
00229 int8_t wifi_app_get_rssi(void);
00230
00231 #endif /* MAIN_WIFI_APP_H */
00232
00233
00234
00235
00236
00237
00238
00239
00240
00241
00242
00243
00244
00245
00246
00247
00248
00249
00250
00251
00252
00253
00254
00255
00256
00257
00258
00259
```

9.37. Referencia del archivo

**C:/Users/Igamb/Documents/Uni/RTOS/Repositorio/RTOS_Repo_↵
Limpio/Proyecto_final_TITl/cuna/README.md**